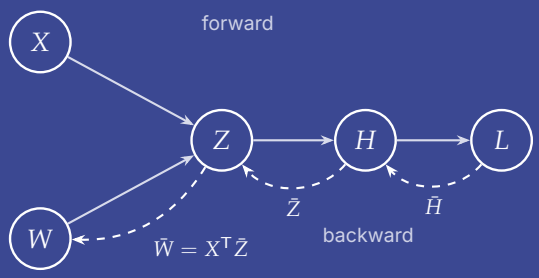
Matrix Calculus for Deep Learning



Functions of matrices, their derivatives, and backpropagation — by hand

Part 1 of a three-part course companion to Bishop & Bishop, *Deep Learning: Foundations and Concepts*

1. **Textbook** (this volume) — derivations by hand, worked examples, exercises with full solutions.
2. **NumPy Lab Manual** — every derivative coded and checked numerically; backprop and autodiff from scratch.
3. **Coding Assignments** — six graded assignments with an automatic `pytest` test suite.

Contents at a glance: shapes and layout conventions · trace, determinant, inverse and spectral functions · matrix identities · gradients, Jacobians and Hessians · curvature and gradient descent · the differential method · Gaussians · backpropagation · finite differences and automatic differentiation · Lagrange multipliers and PCA · Jacobian determinants and normalizing flows · attention and normalisation layers.

Edition of October 2026

How to Use This Course

Bishop & Bishop's *Deep Learning: Foundations and Concepts* is written in the language of vectors and matrices. Its derivations — the least-squares solution, the maximum-likelihood covariance of a Gaussian, the backpropagation equations, the log-determinant in a normalizing flow, the gradient of an attention layer — all depend on one skill: **differentiating functions of vectors and matrices**. Most courses treat this skill as assumed knowledge. This course teaches it directly, from the shapes of derivatives to the backward pass of a transformer layer, and makes you write every result in code.

The three components

Textbook (this volume). Twelve chapters of definitions, rules and, above all, *derivations done by hand*. Every worked example uses small matrices with clean numbers so you can follow each line on paper. Each chapter ends with exercises graded by difficulty; Appendix B solves them all.

NumPy Lab Manual. A separate PDF in which every derivative from this book is implemented in NumPy and *checked numerically*. It builds a multilayer perceptron with a hand-written backward pass, a small automatic-differentiation engine, and the backward passes of attention and layer normalisation. An optional section compares your results with JAX.

Coding Assignments. Six assignments of function stubs for you to complete, with an automatic test suite that checks every gradient against finite differences and prints your score per assignment.

Rules for the code

Everything is NumPy. In this course `numpy.linalg` (`inv`, `solve`, `det`, `slogdet`, `cholesky`, `eigh`) *is* allowed — you built those from scratch in the SVD course, and here the subject is derivatives. What you may not use are libraries that differentiate for you (JAX, PyTorch, TensorFlow, autograd): every gradient is derived by you, coded by you, and checked numerically.

Reading the boxes

Definition: Definitions. Precise statements of new terms.

Theorem: Theorems and rules. Results you will use again and again. Each comes with a short derivation.

Worked Example: Worked examples.
Complete calculations. Cover the solution, try it yourself, then compare line by line.

Key idea. The one thing to remember from a section.

Common pitfall. Mistakes that almost everybody makes once — most of them about shapes and transposes.

Exercises are marked $\star$ routine, $\star\star$ several steps, $\star\star\star$ a proof or a challenge. A tag [Bishop Ch. n] tells you where a result is used in Bishop & Bishop.

Three habits that make matrix calculus easy

1. **Check shapes first.** Before deriving anything, write down the shape of every quantity. The gradient of a scalar with respect to an $m \times n$ matrix is an $m \times n$ matrix; if your answer has another shape, it is wrong — and often a transpose away from right.
2. **Use differentials.** Write how a small change $\mathrm{d}X$ changes f, bring it into the form $\mathrm{d}f = \operatorname{tr}(G^{\mathsf{T}}\mathrm{d}X)$, and read off the gradient G. Chapter 6 turns this into a mechanical procedure that never needs index gymnastics.
3. **Check numerically.** A gradient that has not been compared with finite differences is a guess. The Lab Manual's gradient checker takes five lines of NumPy; use it on everything.

Study Plan

The course is planned as **six weeks at roughly 6–8 hours per week**. Weeks 1–3 are the core technique; weeks 4–6 apply it to the machinery of deep learning. If you are reading Bishop & Bishop at the same time, the table in the next section shows which chapter of this course to finish before each chapter of the book.

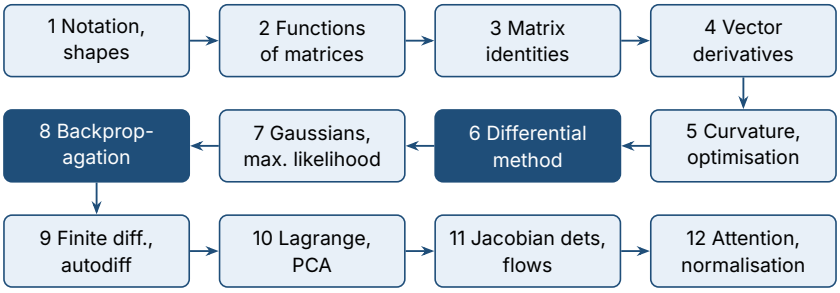


Figure 1: The order of the course. The two dark boxes are the central techniques: the differential method for matrix derivatives, and backpropagation.

Week-by-week plan

Week	Read (Textbook)	Hand practice	NumPy Lab	Assignment
1	Ch. 1 Notation and shapes; Ch. 2 Functions of matrices; Ch. 3 Identities	Ex. 1.1–1.6, 2.1–2.6, 3.1–3.6	Part 0 (NumPy, shapes, `einsum`), Part 1 (gradient checking, matrix functions)	**A1** Matrix functions & checking
2	Ch. 4 Derivatives w.r.t. vectors; Ch. 5 Curvature and optimisation	Ex. 4.1–4.7, 5.1–5.6; one Newton step by hand	Part 2 (gradients, Hessians, gradient descent, Newton)	**A2** Vector derivatives
3	Ch. 6 The differential method; Ch. 7 Gaussians	Ex. 6.1–6.8, 7.1–7.6; derive $\hat{\Sigma}_{\text{ML}}$ without notes	Part 3 (matrix-derivative catalogue, Gaussian fitting)	**A3** Matrix derivatives
4	Ch. 8 Backpropagation	Ex. 8.1–8.7; the two-layer network by hand	Part 4 (an MLP with a hand-written backward pass)	**A4** Backpropagation
5	Ch. 9 Finite differences and autodiff	Ex. 9.1–9.7; a reverse sweep by hand	Part 5 (an autodiff engine; stability; optional JAX check)	**A5** Autodiff engine

Week	Read (Textbook)	Hand practice	NumPy Lab	Assignment
6	Ch. 10 Lagrange and PCA; Ch. 11 Flows; Ch. 12 Attention and normalisation	Ex. 10.1–10.5, 11.1–11.5, 12.1–12.6	Part 6 (attention, layer norm, flows, Hutchinson)	**A6** Deep-learning layers

A suggested weekly rhythm

Day 1. Read the chapters once quickly, then again with pen and paper, redoing every worked example with the solution covered.

Day 2. Do the $\star$ and $\star\star$ exercises; compare with Appendix B only after a genuine attempt.

Day 3. Work through the matching Lab part. Before running each gradient check, predict whether it will pass.

Day 4. Start the assignment. Derive each gradient on paper before you code it.

Day 5. Finish the assignment, attempt the $\star\star\star$ exercises, and do the checkpoint below without notes.

Weekly checkpoints

Week 1 — Shapes and matrix functions

- State the shape of $\partial f/\partial \boldsymbol{x}$, $\partial \boldsymbol{y}/\partial \boldsymbol{x}$ and $\partial f/\partial X$, and say which convention Bishop uses.
- Compute trace, determinant, log-determinant (via Cholesky) and the square root of a 2×2 symmetric matrix.
- Use the cyclic property of the trace, the trace trick $\boldsymbol{x}^\top A\boldsymbol{x} = \operatorname{tr}(A\boldsymbol{x}\boldsymbol{x}^\top)$, and the Sherman–Morrison formula.

Week 2 — Vector derivatives

- Write down without notes the gradients of $\boldsymbol{a}^\top\boldsymbol{x}$, $\boldsymbol{x}^\top A\boldsymbol{x}$, $\|A\boldsymbol{x}-\boldsymbol{b}\|^2$ and $\operatorname{lse}(\boldsymbol{a})$, and the Jacobian of softmax.
- Derive the normal equations and the ridge-regression solution.
- Explain why gradient descent on a quadratic converges only for $\eta < 2/\lambda_{\max}$, and how the condition number sets the speed.

Week 3 — Matrix derivatives

- Derive $\partial/\partial X \operatorname{tr}(AX)$, $\partial/\partial X \ln|X|$ and $\partial/\partial X\, \boldsymbol{a}^\top X^{-1}\boldsymbol{b}$ with the differential method.
- Derive the maximum-likelihood mean and covariance of a Gaussian.
- Compute a conditional Gaussian with the Schur complement, and the KL divergence used in a VAE together with its gradients.

Week 4 — Backpropagation

- Derive $\partial L/\partial W = X^{\mathsf{T}} G$ and $\partial L/\partial X = G W^{\mathsf{T}}$ for $Y = XW$, and the gradient $\boldsymbol{s} - \boldsymbol{t}$ of softmax with cross-entropy.
- Run a forward and backward pass through a two-layer network by hand.

Week 5 — Computing derivatives

- Explain why central differences use $h \approx 10^{-5}$ and forward differences $h \approx 10^{-8}$.
- Differentiate a small function with dual numbers and with a reverse sweep, and say when each mode is cheaper.
- Write a numerically stable log-sum-exp, softmax and log-sigmoid.

Week 6 — Deep-learning applications

- Derive PCA from a Lagrange multiplier.
- Write the change-of-variables formula and the log-determinant of an affine coupling layer.
- Derive the backward pass of a softmax row, of self-attention, and of layer normalisation.

Where Bishop & Bishop uses this course

Bishop & Bishop chapter	What you need	This course
3 Standard Distributions	Gaussian ML for $\boldsymbol{\mu}, \Sigma$; conditionals and marginals; completing the square	Ch. 2, 3, 6, 7
4 Single-layer Networks: Regression	gradient of sum-of-squares; normal equations; pseudo-inverse; regularisation; multiple outputs	Ch. 4, 6
5 Single-layer Networks: Classification	softmax and its Jacobian; cross-entropy gradient; logistic regression Hessian	Ch. 4, 5, 8
6 Deep Neural Networks	Jacobians of layers and activations	Ch. 4, 8
7 Gradient Descent	error surfaces, Hessian eigenvalues, step size, momentum, normalisation	Ch. 5, 12
8 Backpropagation	chain rule, backward pass, Jacobian and Hessian, automatic differentiation	Ch. 8, 9
12 Transformers	softmax attention and its gradients; layer normalisation	Ch. 12
16 Continuous Latent Variables	PCA by Lagrange multipliers; probabilistic PCA	Ch. 7, 10
18 Normalizing Flows	change of variables; log-determinants; coupling layers; trace of the Jacobian	Ch. 11
19 Autoencoders	reparameterisation; KL divergence between Gaussians	Ch. 7
Appendix A, C	matrix identities and derivatives; Lagrange multipliers	Ch. 2, 3, 6, 10

Prerequisite self-check

You need the linear algebra of the SVD course (matrix products, inverse, determinant, eigenvalues of symmetric matrices) and single-variable calculus. Try these:

1. Differentiate $f(x) = x^2 e^{3x}$ and $g(x) = \ln(1 + e^x)$.
2. Compute $\partial/\partial y$ of $f(x, y) = x^2 y + \sin(xy)$.
3. Find the eigenvalues of $\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$.
4. Write the chain rule for $\frac{\mathrm{d}}{\mathrm{d}t} f(g(t))$.
5. What is $\det \left[\begin{smallmatrix} 3 & 1 \\ 2 & 4 \end{smallmatrix}\right]$, and what is the inverse of that matrix?

Answers: (1) $f' = (2x + 3x^2)e^{3x}$; $g' = e^x/(1 + e^x) = \sigma(x)$. (2) $x^2 + x\cos(xy)$. (3) 3 and 1. (4) $f'(g(t))\, g'(t)$. (5) 10; $\frac{1}{10}\left[\begin{smallmatrix} 4 & -1 \\ -2 & 3 \end{smallmatrix}\right]$.

Notation and conventions

x, $\boldsymbol{x}$, X	scalar, column vector, matrix
$\boldsymbol{x}^\top$, $X^\top$	transpose
$\mathbf{1}$, $\mathbf{0}$, I	vector of ones, zero vector, identity
$\partial f/\partial \boldsymbol{x} = \nabla f$	gradient of a scalar f: a **column** vector, same shape as $\boldsymbol{x}$
$\partial \boldsymbol{y}/\partial \boldsymbol{x} = J$	Jacobian of $\boldsymbol{y} \in \mathbb{R}^m$ w.r.t. $\boldsymbol{x} \in \mathbb{R}^n$: an $m \times n$ matrix, $J_{ij} = \partial y_i/\partial x_j$
$\nabla^2 f = H$	Hessian, $H_{ij} = \partial^2 f/\partial x_i \partial x_j$
$\partial f/\partial X$	gradient of a scalar w.r.t. a matrix: same shape as X, entries $\partial f/\partial x_{ij}$
$\mathrm{d}f$, $\mathrm{d}X$	differentials (first-order changes)
$\langle A, B\rangle_F = \operatorname{tr}(A^\top B)$	Frobenius inner product $\sum_{ij} a_{ij} b_{ij}$
$A \odot B$	element-wise (Hadamard) product
$\lvert X\rvert$, $\ln\lvert X\rvert$	determinant and log of its absolute value
$\sigma(a)$, $\operatorname{softmax}(\boldsymbol{a})$	logistic sigmoid; softmax $s_k = e^{a_k}/\sum_j e^{a_j}$
$\operatorname{lse}(\boldsymbol{a})$	log-sum-exp $\ln \sum_k e^{a_k}$
$\bar{v}$	adjoint $\partial L/\partial v$ in reverse-mode differentiation

The Jacobian convention $J_{ij} = \partial y_i/\partial x_j$ ("numerator layout") together with column-vector gradients is the one Bishop & Bishop use; with it, the gradient of a scalar is the transpose of its $1 \times n$ Jacobian. Chapter 1 explains why other books differ.

Contents

How to Use This Course i

Study Plan iii

1 Notation, Shapes and Layout Conventions 1

1.1 Five kinds of derivative . . . 1
1.2 The gradient . . . 2
1.3 The Jacobian . . . 2
1.4 The Hessian . . . 3
1.5 Derivatives with respect to a matrix . . . 3
1.6 Layout conventions — and why books disagree . . . 4
1.7 Rules that carry over from single-variable calculus . . . 5

2 Functions of Matrices 6

2.1 The trace . . . 6
2.2 The determinant and the log-determinant . . . 6
2.3 The inverse and quadratic forms . . . 7
2.4 Spectral functions of symmetric matrices . . . 8
2.5 The matrix exponential . . . 9
2.6 Norms of matrices . . . 9

3 Matrix Identities 11

3.1 Transposes, inverses and traces of products . . . 11
3.2 Determinants . . . 12
3.3 Block matrices and the Schur complement . . . 12
3.4 Low-rank updates: Sherman–Morrison and Woodbury . . . 13
3.5 The push-through identity . . . 14

4 Derivatives with Respect to Vectors 15

4.1 Gradients of linear and quadratic functions . . . 15
4.2 Linear regression: the normal equations . . . 16
4.3 Jacobians of vector functions . . . 16

4.4 The log-sum-exp and the sigmoid . . . 17
4.5 Hessians and stationary points . . . 17

5 Curvature and Optimisation 19
5.1 The local quadratic model . . . 19
5.2 Gradient descent on a quadratic . . . 19
5.3 Why normalising inputs helps . . . 20
5.4 Newton's method . . . 20
5.5 The Gauss–Newton approximation . . . 21

6 The Differential Method: Derivatives with Respect to Matrices 23
6.1 Differentials . . . 23
6.2 The standard matrix derivatives . . . 24
6.3 Multi-output regression . . . 25
6.4 Element-wise functions and Hadamard products . . . 25
6.5 Symmetric matrices . . . 26
6.6 Index notation, as a fallback . . . 26

7 Gaussians and Maximum Likelihood 27
7.1 The log-density . . . 27
7.2 Maximum likelihood for the mean and covariance . . . 27
7.3 Conditional and marginal Gaussians . . . 28
7.4 The KL divergence between Gaussians . . . 29
7.5 The reparameterisation trick . . . 29

8 The Chain Rule and Backpropagation 31
8.1 The chain rule for Jacobians . . . 31
8.2 The backward pass of a linear layer . . . 31
8.3 Activations and losses . . . 32
8.4 A full forward and backward pass by hand . . . 32
8.5 Cost and memory . . . 33
8.6 Forward mode and reverse mode . . . 33

9 Computing Derivatives: Finite Differences, Autodiff and Stability 35
9.1 Finite differences . . . 35
9.2 Gradient checking . . . 36

9.3 Forward mode: dual numbers . . . 36
9.4 Reverse mode: the backward sweep . . . 37
9.5 Numerical stability . . . 37

10 Constrained Optimisation, Eigenproblems and PCA 39
10.1 Lagrange multipliers . . . 39
10.2 Mixture weights: a constraint on a sum . . . 40
10.3 The Rayleigh quotient and the top eigenvector . . . 40
10.4 Principal component analysis . . . 40
10.5 The minimum-norm solution by Lagrange . . . 41

11 Jacobian Determinants and Normalizing Flows 42
11.1 Change of variables . . . 42
11.2 Linear flows . . . 42
11.3 Coupling layers: triangular Jacobians . . . 43
11.4 Planar flows and the determinant lemma . . . 44
11.5 Continuous flows and the trace of the Jacobian . . . 44

12 Attention and Normalisation Layers 46
12.1 The row-wise softmax . . . 46
12.2 Self-attention . . . 46
12.3 Layer normalisation . . . 47

A Formula Sheet 50

B Solutions to the Exercises 52
Chapter 1: Notation, Shapes and Layout Conventions . . . 52
Chapter 2: Functions of Matrices . . . 53
Chapter 3: Matrix Identities . . . 53
Chapter 4: Derivatives with Respect to Vectors . . . 54
Chapter 5: Curvature and Optimisation . . . 54
Chapter 6: The Differential Method . . . 55
Chapter 7: Gaussians and Maximum Likelihood . . . 56
Chapter 8: The Chain Rule and Backpropagation . . . 56
Chapter 9: Computing Derivatives . . . 57
Chapter 10: Constrained Optimisation, Eigenproblems and PCA . . . 58

Chapter 11: Jacobian Determinants and Normalizing Flows 58
Chapter 12: Attention and Normalisation Layers 59

C Further Reading **60**

CHAPTER 1

Notation, Shapes and Layout Conventions

Matrix calculus is ordinary calculus, organised. The derivative of a function of many variables is a collection of partial derivatives; the only questions are *which* partial derivatives, and *how to arrange them* in a vector or matrix. This chapter answers both questions once and for all. Getting the arrangement right is not pedantry: almost every wrong gradient in practice is a correct set of numbers in the wrong shape.

1.1 Five kinds of derivative

A function can take a scalar, a vector or a matrix as input and produce a scalar, a vector or a matrix. Deep learning needs five of the combinations:

Input $\to$ output	Name	Shape of derivative	Example
scalar $\to$ scalar	derivative	1×1	$f(x) = x^2$
vector $\mathbb{R}^n \to$ scalar	gradient ∇f	$n \times 1$ (column)	loss $L(\boldsymbol{w})$
vector $\mathbb{R}^n \to \mathbb{R}^m$	Jacobian J	$m \times n$	a layer $\boldsymbol{y} = \sigma(W\boldsymbol{x})$
vector $\mathbb{R}^n \to$ scalar, twice	Hessian H	$n \times n$	curvature of $L(\boldsymbol{w})$
matrix $\mathbb{R}^{m\times n} \to$ scalar	matrix gradient	$m \times n$	loss $L(W)$

The remaining combinations — the derivative of a matrix with respect to a matrix, for example — are fourth-order arrays. You will almost never need to write them down: Chapter 8 shows that backpropagation only ever needs *products* of such objects with a vector or matrix, and those products have ordinary shapes.

1.2 The gradient

Definition 1.1: Gradient. Let $f : \mathbb{R}^n \to \mathbb{R}$. Its **gradient** at $\boldsymbol{x}$ is the column vector of partial derivatives

$$\nabla f(\boldsymbol{x}) = \frac{\partial f}{\partial \boldsymbol{x}} = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix} \in \mathbb{R}^n.$$

The gradient has the same shape as $\boldsymbol{x}$, which is what lets us write the gradient descent update $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla L(\boldsymbol{w})$ without transposes.

The gradient's defining property is the **first-order (linear) approximation**

$$f(\boldsymbol{x} + \boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^{\mathsf{T}} \boldsymbol{h} \qquad \text{for small } \boldsymbol{h}. \tag{1.1}$$

The number $\nabla f(\boldsymbol{x})^{\mathsf{T}} \boldsymbol{u}$ for a unit vector $\boldsymbol{u}$ is the **directional derivative** — the rate of change of f when moving in direction $\boldsymbol{u}$. By Cauchy–Schwarz it is largest when $\boldsymbol{u}$ points along ∇f: the gradient points uphill, which is why we step against it.

Worked Example 1.1: A gradient and its linear approximation.

$f(x_1, x_2) = x_1^2 x_2 + 3x_2$ at $\boldsymbol{x} = (1, 2)$.

Partial derivatives. $\partial f/\partial x_1 = 2x_1 x_2 = 4$ and $\partial f/\partial x_2 = x_1^2 + 3 = 4$, so $\nabla f(1,2) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$.

Linear approximation. $f(1,2) = 2 + 6 = 8$. For $\boldsymbol{h} = (0.1, 0)$, (1.1) predicts $f(1.1, 2) \approx 8 + 4 \cdot 0.1 = 8.4$.

Exact value. $f(1.1, 2) = 1.21 \cdot 2 + 6 = 8.42$. The error 0.02 is of order $\|\boldsymbol{h}\|^2$, as it should be for a first-order approximation.

1.3 The Jacobian

Definition 1.2: Jacobian. Let $\boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m$. Its **Jacobian** at $\boldsymbol{x}$ is the $m \times n$ matrix

$$J = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{bmatrix} \partial g_1/\partial x_1 & \cdots & \partial g_1/\partial x_n \\ \vdots & & \vdots \\ \partial g_m/\partial x_1 & \cdots & \partial g_m/\partial x_n \end{bmatrix}, \qquad J_{ij} = \frac{\partial g_i}{\partial x_j}.$$

Row i is the (transposed) gradient of output g_i; column j says how all outputs respond to input x_j.

The linear approximation becomes $\boldsymbol{g}(\boldsymbol{x} + \boldsymbol{h}) \approx \boldsymbol{g}(\boldsymbol{x}) + J\boldsymbol{h}$ — a matrix times a vector, with shapes $(m \times n)(n \times 1) = m \times 1$. **The Jacobian is the best linear approximation of $\boldsymbol{g}$ near $\boldsymbol{x}$.** A function $\mathbb{R}^n \to \mathbb{R}$ has a $1 \times n$ Jacobian, and its gradient is the transpose: $\nabla f = J^{\mathsf{T}}$.

Worked Example 1.2: A Jacobian.

$g(x_1, x_2) = \begin{bmatrix} x_1^2 x_2 \\ 5x_1 + \sin x_2 \end{bmatrix}$ at $\boldsymbol{x} = (1, 0)$.

Row 1. $\partial/\partial x_1(x_1^2 x_2) = 2x_1 x_2 = 0, \quad \partial/\partial x_2(x_1^2 x_2) = x_1^2 = 1.$

Row 2. $\partial/\partial x_1(5x_1 + \sin x_2) = 5, \quad \partial/\partial x_2(5x_1 + \sin x_2) = \cos x_2 = 1.$

Result. $J = \begin{bmatrix} 0 & 1 \\ 5 & 1 \end{bmatrix}$. Shape 2×2: two outputs, two inputs.

Three Jacobians you will meet constantly.

- A **linear map** $\boldsymbol{g}(\boldsymbol{x}) = A\boldsymbol{x}$ has Jacobian A (every entry $g_i = \sum_j a_{ij} x_j$ has $\partial g_i/\partial x_j = a_{ij}$).
- An **element-wise function** $\boldsymbol{g}(\boldsymbol{x}) = (h(x_1), \ldots, h(x_n))$ — an activation function — has the *diagonal* Jacobian $\operatorname{diag}(h'(x_1), \ldots, h'(x_n))$, because output i depends only on input i.
- The **softmax** has the dense Jacobian $\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\mathsf{T}$ (Chapter 4).

1.4 The Hessian

Definition 1.3: Hessian. For $f : \mathbb{R}^n \to \mathbb{R}$ with continuous second derivatives, the **Hessian** is the $n \times n$ matrix $H = \nabla^2 f$ with $H_{ij} = \partial^2 f/\partial x_i \partial x_j$. It is symmetric, and it is the Jacobian of the gradient: $H = \partial(\nabla f)/\partial \boldsymbol{x}$.

The Hessian adds the second-order term to the approximation:

$$f(\boldsymbol{x} + \boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\mathsf{T} \boldsymbol{h} + \tfrac{1}{2} \boldsymbol{h}^\mathsf{T} H \boldsymbol{h}. \tag{1.2}$$

This quadratic model of the loss is the starting point of Bishop's analysis of error surfaces [Bishop Ch. 7], and of Chapter 5.

1.5 Derivatives with respect to a matrix

Definition 1.4: Gradient with respect to a matrix. For a scalar function f of an $m \times n$ matrix X, the gradient $\partial f/\partial X$ is the $m \times n$ matrix with entries $(\partial f/\partial X)_{ij} = \partial f/\partial x_{ij}$.

Key idea. **The gradient of a scalar always has the shape of the variable.** If W is 784×128, then $\partial L/\partial W$ is 784×128. This single rule detects most mistakes before you run any code.

The matrix version of the linear approximation uses the Frobenius inner product $\langle A, B\rangle_F = \sum_{ij} a_{ij} b_{ij} = \operatorname{tr}(A^\top B)$:

$$f(X+\Delta) \approx f(X) + \sum_{ij} \frac{\partial f}{\partial x_{ij}} \Delta_{ij} = f(X) + \operatorname{tr}\Big(\Big(\frac{\partial f}{\partial X}\Big)^\top \Delta\Big). \tag{1.3}$$

Equation (1.3) is the seed of the differential method of Chapter 6: whenever you can write the change in f as $\operatorname{tr}(G^\top \Delta)$, the gradient is G.

Worked Example 1.3: A gradient with respect to a matrix, entry by entry.

$f(X) = x_{11}x_{22} - x_{12}x_{21} = \det X$ for a 2×2 matrix X.

Entries. $\partial f/\partial x_{11} = x_{22}$, $\partial f/\partial x_{12} = -x_{21}$, $\partial f/\partial x_{21} = -x_{12}$, $\partial f/\partial x_{22} = x_{11}$.

Arrange in the shape of X. $\frac{\partial f}{\partial X} = \begin{bmatrix} x_{22} & -x_{21} \\ -x_{12} & x_{11} \end{bmatrix}$.

Recognise it. This is the transpose of the adjugate, $\det(X)\, X^{-\top}$. Chapter 6 derives $\partial \det X/\partial X = \det(X) X^{-\top}$ for every size, without computing a single entry.

1.6 Layout conventions — and why books disagree

There are two consistent ways to arrange the derivative of $\boldsymbol{y} \in \mathbb{R}^m$ with respect to $\boldsymbol{x} \in \mathbb{R}^n$:

Numerator layout: $\partial \boldsymbol{y}/\partial \boldsymbol{x}$ is $m \times n$ (rows follow $\boldsymbol{y}$). This is the Jacobian above.

Denominator layout: $\partial \boldsymbol{y}/\partial \boldsymbol{x}$ is $n \times m$ (rows follow $\boldsymbol{x}$) — the transpose.

In numerator layout the derivative of a scalar w.r.t. a column vector is naturally a *row*; in denominator layout it is a column. Many authors mix the two: they use the Jacobian ($m \times n$) for vector outputs but a column gradient for scalar outputs. **That mixed convention is the one used by Bishop & Bishop and by this course**: $J_{ij} = \partial y_i/\partial x_j$, and $\nabla f = \partial f/\partial \boldsymbol{x}$ is a column, equal to the transpose of the $1 \times n$ Jacobian.

Common pitfall. When you copy a formula from another source — the *Matrix Cookbook*, a blog post, a paper — first check its layout. A formula like $\partial(A\boldsymbol{x})/\partial \boldsymbol{x} = A^\top$ is correct in denominator layout and wrong in ours (where it is A). The check is always the same: write the shapes and see which one is consistent.

1.7 Rules that carry over from single-variable calculus

- **Linearity:** $\nabla(\alpha f + \beta g) = \alpha \nabla f + \beta \nabla g$.
- **Product rule:** $\nabla(fg) = f\nabla g + g\nabla f$ for scalar functions.
- **Chain rule:** for $\boldsymbol{z} = \boldsymbol{f}(\boldsymbol{y})$, $\boldsymbol{y} = \boldsymbol{g}(\boldsymbol{x})$: $J_{z,x} = J_{z,y}\, J_{y,x}$ (matrix product in that order). For a scalar loss $L(\boldsymbol{y}(\boldsymbol{x}))$: $\nabla_{\boldsymbol{x}} L = J_{y,x}^{\mathsf{T}}\, \nabla_{\boldsymbol{y}} L$. Chapter 8 builds backpropagation on this one line.

Exercises

Exercise 1.1 $\star$ Compute the gradient of $f(x_1, x_2, x_3) = x_1 x_2 + x_2 x_3^2 - 2x_1$ at $(1, 2, -1)$, and use it to predict $f(1, 2.1, -1)$. Compare with the exact value.

Exercise 1.2 $\star$ Find the Jacobian of $\boldsymbol{g}(\boldsymbol{x}) = \begin{bmatrix} x_1 x_2 x_3 \\ x_1 + x_2 + x_3 \end{bmatrix}$ and state its shape.

Exercise 1.3 $\star$ Write down the Jacobian of $\boldsymbol{g}(\boldsymbol{x}) = \tanh(\boldsymbol{x})$ (element-wise) at $\boldsymbol{x} = (0, \ln 2)$. *Hint:* $\tanh'(x) = 1 - \tanh^2(x)$ and $\tanh(\ln 2) = 3/5$.

Exercise 1.4 $\star\star$ Compute the Hessian of $f(x_1, x_2) = x_1^3 + x_1 x_2^2 - x_2$ and write the quadratic approximation (1.2) at $(1, 1)$.

Exercise 1.5 $\star\star$ For a 2×2 matrix X, let $f(X) = x_{11}^2 + x_{12}x_{21} + 3x_{22}$. Compute $\partial f / \partial X$ entry by entry, then verify that $f(X + \Delta) - f(X) \approx \operatorname{tr}((\partial f/\partial X)^{\mathsf{T}} \Delta)$ for $X = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$, $\Delta = \left[\begin{smallmatrix} 0.01 & 0 \\ 0 & 0.01 \end{smallmatrix}\right]$.

Exercise 1.6 $\star\star\star$ (a) Show that for a linear map $\boldsymbol{g}(\boldsymbol{x}) = A\boldsymbol{x}$ the Jacobian in numerator layout is A. (b) What is it in denominator layout? (c) A source states "$\partial(\boldsymbol{x}^{\mathsf{T}} A \boldsymbol{x})/\partial \boldsymbol{x} = \boldsymbol{x}^{\mathsf{T}}(A + A^{\mathsf{T}})$". Which layout is it using, and how does the formula read in our convention?

CHAPTER 2

Functions of Matrices

Before differentiating functions of matrices we need a stock of such functions. Some turn a matrix into a number — trace, determinant, log-determinant, norms, quadratic forms. Others turn a matrix into a matrix — the inverse, powers, the square root, the exponential. These are exactly the functions that appear in Bishop's formulas: the Gaussian density contains $\ln|\Sigma|$ and Σ^{-1}, sampling uses a square root of Σ, and normalizing flows contain $\ln|\det J|$.

2.1 The trace

Definition 2.1: Trace. The **trace** of a square matrix is the sum of its diagonal entries, $\operatorname{tr} A = \sum_i a_{ii}$.

The trace is linear, $\operatorname{tr}(\alpha A + \beta B) = \alpha \operatorname{tr} A + \beta \operatorname{tr} B$, and unchanged by transposition, $\operatorname{tr} A^\top = \operatorname{tr} A$. Its two most important properties are

$$\operatorname{tr}(AB) = \operatorname{tr}(BA) \qquad \text{and} \qquad \operatorname{tr}(A^\top B) = \sum_{i,j} a_{ij} b_{ij} = \langle A, B \rangle_F. \tag{2.1}$$

The first holds whenever both products are square (even if A and B are rectangular): both sides equal $\sum_{i,j} a_{ij} b_{ji}$. The second says that the trace of $A^\top B$ is the "dot product of matrices" — multiply corresponding entries and add. In particular $\operatorname{tr}(A^\top A) = \sum a_{ij}^2 = \|A\|_F^2$.

For a symmetric matrix with eigenvalues λ_i, $\operatorname{tr} A = \sum_i \lambda_i$ (true for every square matrix, in fact).

2.2 The determinant and the log-determinant

The determinant has three properties worth remembering:

$$\det(AB) = \det A \det B, \qquad \det(A^\top) = \det A, \qquad \det(A^{-1}) = 1/\det A,$$

plus $\det(cA) = c^n \det A$ for $n \times n$ A, and $\det A = \prod_i \lambda_i$. Geometrically $|\det A|$ is the factor by which A scales volumes — the reason it appears in the change-of-variables formula for probability densities (Chapter 11). For a **triangular** matrix the determinant is the product of the diagonal entries.

In probability we almost always need the **log-determinant** $\ln|A|$, for two reasons. Log-likelihoods are sums of logs, so $\ln|\Sigma|$ appears directly in $\ln \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \Sigma)$. And determinants overflow or underflow easily: the determinant of the 1000×1000 matrix $0.5I$ is $0.5^{1000} \approx 10^{-301}$, at the edge of double precision, while its log-determinant $1000 \ln 0.5 \approx -693$ is an ordinary number.

For a symmetric positive definite matrix, the log-determinant is computed from the **Cholesky factorisation**.

Theorem 2.2: Cholesky factorisation. Every symmetric positive definite S can be written uniquely as $S = LL^{\mathsf{T}}$ with L lower triangular and $l_{ii} > 0$. Consequently

$$\ln|S| = \ln\det(L)^2 = 2\sum_i \ln l_{ii}.$$

Cholesky by hand.

Column by column, for $j = 1, \dots, n$:

1. $l_{jj} = \sqrt{s_{jj} - \sum_{k<j} l_{jk}^2}$;
2. $l_{ij} = \left(s_{ij} - \sum_{k<j} l_{ik} l_{jk}\right) / l_{jj}$ for $i > j$.

If a square root of a non-positive number appears, S is not positive definite.

Worked Example 2.1: Trace, determinant, log-determinant.

$S = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$.

Trace and determinant. $\operatorname{tr} S = 7$, $\det S = 12 - 4 = 8$.

Eigenvalues. $\lambda^2 - 7\lambda + 8 = 0$ gives $\lambda = \frac{7 \pm \sqrt{17}}{2} \approx 5.562,\ 1.438$. Check: sum 7, product 8. ✓

Cholesky. $l_{11} = \sqrt{4} = 2$; $l_{21} = 2/2 = 1$; $l_{22} = \sqrt{3 - 1^2} = \sqrt{2}$. So $L = \begin{bmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{bmatrix}$, and $LL^{\mathsf{T}} = \begin{bmatrix} 4 & 2 \\ 2 & 1+2 \end{bmatrix} = S$. ✓

Log-determinant. $2(\ln 2 + \ln\sqrt{2}) = 2\ln 2 + \ln 2 = 3\ln 2 = \ln 8$. ✓

Frobenius norm. $\|S\|_F = \sqrt{16 + 4 + 4 + 9} = \sqrt{33}$.

2.3 The inverse and quadratic forms

For invertible A, $(AB)^{-1} = B^{-1}A^{-1}$ and $(A^{\mathsf{T}})^{-1} = (A^{-1})^{\mathsf{T}}$, written $A^{-\mathsf{T}}$. In code one almost never forms A^{-1} explicitly: to compute $A^{-1}\boldsymbol{b}$, *solve* $A\boldsymbol{x} = \boldsymbol{b}$ (with a Cholesky or LU factorisation). It is faster and more accurate.

A **quadratic form** is $q(\boldsymbol{x}) = \boldsymbol{x}^{\mathsf{T}} A \boldsymbol{x} = \sum_{ij} a_{ij} x_i x_j$. Only the symmetric part of A matters: $\boldsymbol{x}^{\mathsf{T}} A \boldsymbol{x} = \boldsymbol{x}^{\mathsf{T}} \frac{1}{2}(A + A^{\mathsf{T}})\boldsymbol{x}$, because $\boldsymbol{x}^{\mathsf{T}} A \boldsymbol{x}$ is a number and so equals its own transpose $\boldsymbol{x}^{\mathsf{T}} A^{\mathsf{T}} \boldsymbol{x}$. The most important quadratic form in this course is the squared **Mahalanobis distance** $(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}} \Sigma^{-1} (\boldsymbol{x} - \boldsymbol{\mu})$ in the Gaussian density.

2.4 Spectral functions of symmetric matrices

How should we define $\sqrt{S}$, e^S or $\ln S$ for a matrix S? For symmetric matrices the spectral theorem gives a clean answer.

Definition 2.3: Function of a symmetric matrix. Let $S = Q\Lambda Q^\mathsf{T}$ with Q orthogonal and $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. For a scalar function f defined at every λ_i,

$$f(S) = Q\, f(\Lambda)\, Q^\mathsf{T} = Q \operatorname{diag}(f(\lambda_1), \dots, f(\lambda_n))\, Q^\mathsf{T} = \sum_i f(\lambda_i)\, \boldsymbol{q}_i \boldsymbol{q}_i^\mathsf{T}.$$

The definition agrees with every case where we already know the answer: $f(\lambda) = \lambda^2$ gives $Q\Lambda^2 Q^\mathsf{T} = S^2$; $f(\lambda) = 1/\lambda$ gives S^{-1}; $f(\lambda) = \lambda^k$ gives S^k. New cases:

- $S^{1/2} = Q\Lambda^{1/2} Q^\mathsf{T}$ for S positive semidefinite: the unique PSD matrix with $(S^{1/2})^2 = S$.
- $S^{-1/2} = Q\Lambda^{-1/2} Q^\mathsf{T}$: the **whitening** matrix. If $\operatorname{cov}(\boldsymbol{x}) = S$ then $\operatorname{cov}(S^{-1/2}\boldsymbol{x}) = I$ [Bishop Ch. 16].
- $\exp(S) = Q\, e^\Lambda Q^\mathsf{T}$ and, for S positive definite, $\ln S = Q \ln \Lambda\, Q^\mathsf{T}$. Then $\operatorname{tr} \ln S = \sum \ln \lambda_i = \ln |S|$.

Worked Example 2.2: A matrix square root.

Find $\sqrt{S}$ for $S = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$.

Eigen-decomposition. $\lambda = 9$ with $\boldsymbol{q}_1 = \frac{1}{\sqrt{2}}(1, 1)$, and $\lambda = 1$ with $\boldsymbol{q}_2 = \frac{1}{\sqrt{2}}(1, -1)$.

Apply $\sqrt{\ }$ to the eigenvalues. $\sqrt{S} = 3\boldsymbol{q}_1\boldsymbol{q}_1^\mathsf{T} + 1\boldsymbol{q}_2\boldsymbol{q}_2^\mathsf{T} = \frac{3}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \frac{1}{2}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

Check. $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}^2 = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}$. ✓

Two square roots. The symmetric square root $S^{1/2}$ and the Cholesky factor L are different matrices, but both satisfy "$RR^\mathsf{T} = S$". Either can be used to sample from a Gaussian: if $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$ then $\boldsymbol{\mu} + R\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\mu}, RR^\mathsf{T})$ [Bishop Ch. 14,19]. Cholesky is the one used in practice because it is cheaper.

2.5 The matrix exponential

For any square matrix, symmetric or not, the exponential is defined by the power series

$$\exp(A) = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \cdots,$$

which always converges. For symmetric A it agrees with the spectral definition. Two facts: $\exp(A+B) = \exp(A)\exp(B)$ *only if* $AB = BA$; and $\det\exp(A) = e^{\operatorname{tr} A}$, a relation behind the trace formula for continuous normalizing flows (Chapter 11).

Worked Example 2.3: Exponentials of two 2×2 matrices.

Symmetric. $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ has $\lambda = \pm 1$ with $\boldsymbol{q} = \frac{1}{\sqrt{2}}(1, \pm 1)$. So $\exp A = \frac{e}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \frac{e^{-1}}{2}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} \cosh 1 & \sinh 1 \\ \sinh 1 & \cosh 1 \end{bmatrix}$.

Skew-symmetric. $B = \begin{bmatrix} 0 & -\theta \\ \theta & 0 \end{bmatrix}$ satisfies $B^2 = -\theta^2 I$. Grouping even and odd powers of the series, $\exp B = \cos\theta\, I + \frac{\sin\theta}{\theta} B = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$: a rotation. Check $\det\exp B = 1 = e^{\operatorname{tr} B}$. ✓

2.6 Norms of matrices

From the SVD course: the **Frobenius norm** $\|A\|_F = \sqrt{\operatorname{tr}(A^\top A)} = \sqrt{\sum \sigma_i^2}$ and the **spectral norm** $\|A\|_2 = \sigma_1$. The squared Frobenius norm is the matrix version of the sum of squares, and is the loss of a multi-output regression, $\frac{1}{2}\|XW - T\|_F^2$ [Bishop Ch. 4].

Exercises

Exercise 2.1 ⋆ For $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ and $B = \left[\begin{smallmatrix} 0 & 1 \\ 1 & 1 \end{smallmatrix}\right]$ verify $\operatorname{tr}(AB) = \operatorname{tr}(BA)$ and compute $\langle A, B\rangle_F$. Is $\operatorname{tr}(AB) = \langle A, B\rangle_F$ in general?

Exercise 2.2 ⋆ Compute the Cholesky factor of $S = \left[\begin{smallmatrix} 9 & 3 \\ 3 & 5 \end{smallmatrix}\right]$, and use it to find $\ln|S|$. Check against $\det S$.

Exercise 2.3 ⋆ Find the symmetric square root of $\left[\begin{smallmatrix} 10 & 6 \\ 6 & 10 \end{smallmatrix}\right]$ and verify it.

Exercise 2.4 ⋆⋆ Let $S = \left[\begin{smallmatrix} 5 & 4 \\ 4 & 5 \end{smallmatrix}\right]$. Compute $S^{-1/2}$ and verify that $S^{-1/2} S S^{-1/2} = I$. What does this mean for data with covariance S?

Exercise 2.5 ⋆⋆ Prove (a) $\ln|S| = \operatorname{tr}\ln S$ for symmetric positive definite S, and (b) $\det(cA) = c^n \det A$ for $n \times n$ A. (c) Use (b) to compute $\ln\det(0.5 I_{1000})$.

Exercise 2.6 ⋆⋆⋆ Show that for a nilpotent matrix $N = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$, $\exp(tN) = \left[\begin{smallmatrix} 1 & t \\ 0 & 1 \end{smallmatrix}\right]$. Then show that $\exp(A)\exp(B) \neq \exp(A+B)$ for $A = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$, $B = \left[\begin{smallmatrix} 0 & 0 \\ 1 & 0 \end{smallmatrix}\right]$. *Hint:* $A + B$ is the matrix of Worked Example 2.3.

CHAPTER 3

Matrix Identities

Many derivations in deep learning are not about differentiation at all: they rearrange matrix expressions until a derivative becomes obvious, or until an expensive inverse becomes a cheap one. This chapter collects the identities that Bishop & Bishop (Appendix A) and the rest of this course rely on. Learn to recognise them; you will use the cyclic trace and the trace trick in nearly every chapter.

3.1 Transposes, inverses and traces of products

$$(AB)^\top = B^\top A^\top, \qquad (AB)^{-1} = B^{-1}A^{-1}, \qquad (A^{-1})^\top = (A^\top)^{-1} = A^{-\top}.$$

The order reverses, for the same reason that taking off socks and shoes reverses putting them on.

Theorem 3.1: Cyclic property of the trace. $\operatorname{tr}(ABC) = \operatorname{tr}(BCA) = \operatorname{tr}(CAB)$ whenever the products are defined and square. In general $\operatorname{tr}(ABC) \neq \operatorname{tr}(ACB)$: you may *rotate* the factors but not *reorder* them.

Proof. Apply $\operatorname{tr}(XY) = \operatorname{tr}(YX)$ with $X = A$, $Y = BC$, and again with $X = AB$, $Y = C$. □

Theorem 3.2: The trace trick. A scalar equals its own trace, so for any vectors and matrices of compatible shapes

$$\boldsymbol{x}^\top A \boldsymbol{x} = \operatorname{tr}(\boldsymbol{x}^\top A \boldsymbol{x}) = \operatorname{tr}(A \boldsymbol{x}\boldsymbol{x}^\top),$$

$$\sum_{n=1}^{N} (\boldsymbol{x}_n - \boldsymbol{\mu})^\top A (\boldsymbol{x}_n - \boldsymbol{\mu}) = \operatorname{tr}(AS), \qquad S = \sum_n (\boldsymbol{x}_n - \boldsymbol{\mu})(\boldsymbol{x}_n - \boldsymbol{\mu})^\top.$$

The trace trick moves the data into a single matrix S (the scatter matrix). It is how the Gaussian log-likelihood becomes a function of Σ^{-1} that is easy to differentiate (Chapter 7).

Worked Example 3.1: Cyclic trace: rotate yes, reorder no.

Let $\boldsymbol{a} = (1, 2)$, $\boldsymbol{b} = (3, -1)$ and $A = \left[\begin{smallmatrix} 1 & 0 \\ 2 & 1 \end{smallmatrix}\right]$.

Directly. $\boldsymbol{a}^\top A \boldsymbol{b}$: $A\boldsymbol{b} = (3,\ 6 - 1) = (3, 5)$, so $\boldsymbol{a}^\top A \boldsymbol{b} = 3 + 10 = 13$.

Rotated. $\operatorname{tr}(A\boldsymbol{b}\boldsymbol{a}^\top)$: $\boldsymbol{b}\boldsymbol{a}^\top = \left[\begin{smallmatrix} 3 & 6 \\ -1 & -2 \end{smallmatrix}\right]$, $A\boldsymbol{b}\boldsymbol{a}^\top = \left[\begin{smallmatrix} 3 & 6 \\ 5 & 10 \end{smallmatrix}\right]$, trace $3 + 10 = 13$. ✓

Reordered (wrong). $\operatorname{tr}(A\boldsymbol{a}\boldsymbol{b}^\top)$: $\boldsymbol{a}\boldsymbol{b}^\top = \left[\begin{smallmatrix} 3 & -1 \\ 6 & -2 \end{smallmatrix}\right]$, $A\boldsymbol{a}\boldsymbol{b}^\top = \left[\begin{smallmatrix} 3 & -1 \\ 12 & -4 \end{smallmatrix}\right]$, trace $-1 \neq 13$.

3.2 Determinants

$$\det(AB) = \det A \det B, \qquad \det(A^{\mathsf{T}}) = \det A, \qquad \det(A^{-1}) = (\det A)^{-1},$$
$$\det(cA) = c^n \det A \ \ (A \text{ is } n \times n), \qquad \ln|AB| = \ln|A| + \ln|B|.$$

For a block-triangular matrix, $\det \left[\begin{smallmatrix} A & B \\ 0 & D \end{smallmatrix}\right] = \det A \det D$; this is why normalizing flows are built from triangular Jacobians (Chapter 11).

3.3 Block matrices and the Schur complement

Partition a matrix into blocks, $M = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$ with A and D square. Blocks multiply like numbers, as long as you keep the order.

Definition 3.3: Schur complement. If D is invertible, the **Schur complement of D in M** is $M/D = A - BD^{-1}C$. Similarly $M/A = D - CA^{-1}B$.

Theorem 3.4: Partitioned inverse and determinant. If D and M/D are invertible, then with $P = (M/D)^{-1} = (A - BD^{-1}C)^{-1}$,

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} P & -PBD^{-1} \\ -D^{-1}CP & D^{-1} + D^{-1}CPBD^{-1} \end{bmatrix}, \qquad \det M = \det D \, \det(A - BD^{-1}C).$$

Proof sketch. Eliminate C and B by block row and column operations:

$$\begin{bmatrix} I & -BD^{-1} \\ 0 & I \end{bmatrix} \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} I & 0 \\ -D^{-1}C & I \end{bmatrix} = \begin{bmatrix} A - BD^{-1}C & 0 \\ 0 & D \end{bmatrix}.$$

The outer factors have determinant 1, which gives $\det M$; inverting the three factors gives M^{-1}. □

This is exactly the formula Bishop uses for conditional Gaussians [Bishop Ch. 3]: the top-left block of the inverse of Σ is the inverse of a Schur complement, and that Schur complement is the conditional covariance (Chapter 7).

Worked Example 3.2: A partitioned inverse.

$M = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{bmatrix}$ with $A = [2]$, $B = [1 \ \ 0]$, $C = B^{\mathsf{T}}$, $D = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$.

Inverse of D. $D^{-1} = \frac{1}{3}\left[\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right]$.

Schur complement. $BD^{-1}C = \frac{1}{3} \cdot 2 = \frac{2}{3}$, so $M/D = 2 - \frac{2}{3} = \frac{4}{3}$ and $P = \frac{3}{4}$.

Determinant. $\det M = \det D \cdot \frac{4}{3} = 3 \cdot \frac{4}{3} = 4$.
Off-diagonal block. $-PBD^{-1} = -\frac{3}{4} \cdot \frac{1}{3}(2,-1) = (-\frac{1}{2}, \frac{1}{4})$.
Check. The full inverse is $M^{-1} = \frac{1}{4}\begin{bmatrix} 3 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 3 \end{bmatrix}$; its first row is $(\frac{3}{4}, -\frac{1}{2}, \frac{1}{4})$. ✓

3.4 Low-rank updates: Sherman–Morrison and Woodbury

Suppose you already know A^{-1} and A changes by a matrix of low rank. Then the new inverse can be computed cheaply from the old one.

Theorem 3.5: Woodbury identity. For A $(n \times n)$, U $(n \times k)$, C $(k \times k)$, V $(k \times n)$, when the inverses exist,

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\big(C^{-1} + VA^{-1}U\big)^{-1}VA^{-1}.$$

The special case $k = 1$ is the **Sherman–Morrison formula**

$$(A + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}})^{-1} = A^{-1} - \frac{A^{-1}\boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}A^{-1}}{1 + \boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u}},$$

and the matching determinant formula is the **matrix determinant lemma**

$$\det(A + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}) = (1 + \boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u}) \det A.$$

Proof of Sherman–Morrison. Multiply $(A + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}})$ by the claimed inverse. Writing $\beta = \boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u}$ (a number): $I + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}A^{-1} - \frac{\boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}A^{-1} + \boldsymbol{u}(\boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u})\boldsymbol{v}^{\mathsf{T}}A^{-1}}{1+\beta} = I + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}A^{-1} - \frac{(1+\beta)\boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}A^{-1}}{1+\beta} = I$. □

Key idea. Woodbury replaces an $n \times n$ inverse by a $k \times k$ inverse. When $k \ll n$ — a diagonal matrix plus a low-rank correction, as in factor analysis and probabilistic PCA [Bishop Ch. 16], or a Gaussian-process covariance with few inducing points — this turns $O(n^3)$ work into $O(nk^2)$.

Worked Example 3.3: Sherman–Morrison and the determinant lemma.
$A = \operatorname{diag}(2,4)$, $\boldsymbol{u} = \boldsymbol{v} = (1,1)$, so $A + \boldsymbol{u}\boldsymbol{v}^{\mathsf{T}} = \left[\begin{smallmatrix} 3 & 1 \\ 1 & 5 \end{smallmatrix}\right]$.
Ingredients. $A^{-1} = \operatorname{diag}(\frac{1}{2}, \frac{1}{4})$, $A^{-1}\boldsymbol{u} = (\frac{1}{2}, \frac{1}{4})$, $\boldsymbol{v}^{\mathsf{T}}A^{-1}\boldsymbol{u} = \frac{3}{4}$.
Update. $A^{-1} - \dfrac{(\frac{1}{2}, \frac{1}{4})(\frac{1}{2}, \frac{1}{4})^{\mathsf{T}}}{7/4} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{4} \end{bmatrix} - \frac{4}{7}\begin{bmatrix} \frac{1}{4} & \frac{1}{8} \\ \frac{1}{8} & \frac{1}{16} \end{bmatrix} = \frac{1}{14}\begin{bmatrix} 5 & -1 \\ -1 & 3 \end{bmatrix}$.
Check. $\left[\begin{smallmatrix} 3 & 1 \\ 1 & 5 \end{smallmatrix}\right]^{-1} = \frac{1}{14}\left[\begin{smallmatrix} 5 & -1 \\ -1 & 3 \end{smallmatrix}\right]$ since $\det = 14$. ✓
Determinant lemma. $(1 + \frac{3}{4}) \cdot \det A = \frac{7}{4} \cdot 8 = 14$. ✓

3.5 The push-through identity

Theorem 3.6: Push-through identity. For A $(n \times m)$ and B $(m \times n)$, whenever the inverses exist,

$$(I_n + AB)^{-1}A = A(I_m + BA)^{-1}.$$

Proof. $A(I_m + BA) = (I_n + AB)A$; multiply by $(I_n + AB)^{-1}$ on the left and $(I_m + BA)^{-1}$ on the right. □

Applied to ridge regression with design matrix Φ $(N \times D)$:

$$(\Phi^\top \Phi + \lambda I_D)^{-1}\Phi^\top = \Phi^\top(\Phi\Phi^\top + \lambda I_N)^{-1}.$$

The left side inverts a $D \times D$ matrix, the right side an $N \times N$ one: choose whichever is smaller. The right-hand form is the starting point of kernel methods.

Exercises

Exercise 3.1 ⋆ Verify $\operatorname{tr}(ABC) = \operatorname{tr}(CAB)$ for $A = \left[\begin{smallmatrix}1&2\\0&1\end{smallmatrix}\right]$, $B = \left[\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right]$, $C = \left[\begin{smallmatrix}2&0\\0&3\end{smallmatrix}\right]$, and compute $\operatorname{tr}(ACB)$.

Exercise 3.2 ⋆ Use the trace trick to write $\sum_{n=1}^{3} \boldsymbol{x}_n^\top A \boldsymbol{x}_n$ as a single trace, and evaluate it for $A = \left[\begin{smallmatrix}1&1\\0&2\end{smallmatrix}\right]$ and $\boldsymbol{x}_1 = (1,0)$, $\boldsymbol{x}_2 = (0,1)$, $\boldsymbol{x}_3 = (1,1)$.

Exercise 3.3 ⋆⋆ Use the matrix determinant lemma to compute $\det(I_n + \mathbf{1}\mathbf{1}^\top)$, where $\mathbf{1} \in \mathbb{R}^n$ is the vector of ones. Check for $n = 2$.

Exercise 3.4 ⋆⋆ Use Sherman–Morrison to find $(I_n + \mathbf{1}\mathbf{1}^\top)^{-1}$, and check the case $n = 2$.

Exercise 3.5 ⋆⋆ Compute $\det M$ and the top-left entry of M^{-1} for $M = \left[\begin{smallmatrix}4&1&1\\1&3&0\\1&0&2\end{smallmatrix}\right]$ using the Schur complement of $D = \left[\begin{smallmatrix}3&0\\0&2\end{smallmatrix}\right]$.

Exercise 3.6 ⋆⋆⋆ (a) Derive Woodbury in the special case $A = I$, $C = I$: show $(I + UV)^{-1} = I - U(I + VU)^{-1}V$ by multiplying out. (b) Count the cost (in $O(\cdot)$) of computing $(\lambda I_n + UU^\top)^{-1}\boldsymbol{b}$ directly and via Woodbury when U is $n \times k$.

CHAPTER 4

Derivatives with Respect to Vectors

This chapter builds the table of gradients, Jacobians and Hessians that every derivation in Bishop & Bishop uses: linear functions, quadratic forms, least squares, the log-sum-exp, softmax and the sigmoid. We derive each one from partial derivatives once, so that afterwards you can use them as building blocks without thinking about indices.

4.1 Gradients of linear and quadratic functions

Theorem 4.1: Basic gradients. For constant $\boldsymbol{a} \in \mathbb{R}^n$, $A \in \mathbb{R}^{n\times n}$, $B \in \mathbb{R}^{m\times n}$, $\boldsymbol{b} \in \mathbb{R}^m$:

$$\nabla_{\boldsymbol{x}}(\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x}) = \boldsymbol{a}, \qquad \nabla_{\boldsymbol{x}}(\boldsymbol{x}^{\mathsf{T}}A\boldsymbol{x}) = (A + A^{\mathsf{T}})\boldsymbol{x} \quad (= 2A\boldsymbol{x} \text{ if } A = A^{\mathsf{T}}),$$
$$\nabla_{\boldsymbol{x}} \|\boldsymbol{x}\|^2 = 2\boldsymbol{x}, \qquad \nabla_{\boldsymbol{x}} \|B\boldsymbol{x} - \boldsymbol{b}\|^2 = 2B^{\mathsf{T}}(B\boldsymbol{x} - \boldsymbol{b}),$$
$$\nabla_{\boldsymbol{x}} \|\boldsymbol{x}\| = \boldsymbol{x}/\|\boldsymbol{x}\| \ \ (\boldsymbol{x} \neq \boldsymbol{0}).$$

Proof. $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x} = \sum_i a_i x_i$, so $\partial/\partial x_k = a_k$. For the quadratic form $\sum_{ij} a_{ij}x_i x_j$, the product rule gives $\partial/\partial x_k = \sum_j a_{kj}x_j + \sum_i a_{ik}x_i = (A\boldsymbol{x})_k + (A^{\mathsf{T}}\boldsymbol{x})_k$. Then $\|B\boldsymbol{x} - \boldsymbol{b}\|^2 = \boldsymbol{x}^{\mathsf{T}}B^{\mathsf{T}}B\boldsymbol{x} - 2\boldsymbol{b}^{\mathsf{T}}B\boldsymbol{x} + \boldsymbol{b}^{\mathsf{T}}\boldsymbol{b}$, whose gradient is $2B^{\mathsf{T}}B\boldsymbol{x} - 2B^{\mathsf{T}}\boldsymbol{b}$. Finally $\|\boldsymbol{x}\| = (\|\boldsymbol{x}\|^2)^{1/2}$ and the chain rule gives $\frac{1}{2}(\|\boldsymbol{x}\|^2)^{-1/2} \cdot 2\boldsymbol{x}$. □

Common pitfall. $\nabla(\boldsymbol{x}^{\mathsf{T}}A\boldsymbol{x}) = 2A\boldsymbol{x}$ only for symmetric A. For a non-symmetric A the factor is $A + A^{\mathsf{T}}$. Forgetting this is the most common error in hand derivations — and it passes every test that happens to use a symmetric A.

Worked Example 4.1: Gradient and minimum of a quadratic.

$f(\boldsymbol{x}) = \boldsymbol{x}^{\mathsf{T}}A\boldsymbol{x} + \boldsymbol{b}^{\mathsf{T}}\boldsymbol{x}$ with $A = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 3 \end{smallmatrix}\right]$, $\boldsymbol{b} = (1, -1)$.

Gradient formula. $\nabla f = 2A\boldsymbol{x} + \boldsymbol{b}$ (A is symmetric).

At $x = (1, 2)$. $A\boldsymbol{x} = (4, 7)$, so $\nabla f = (8, 14) + (1, -1) = (9, 13)$.

Check with partials. $f = 2x_1^2 + 2x_1x_2 + 3x_2^2 + x_1 - x_2$: $\partial f/\partial x_1 = 4x_1 + 2x_2 + 1 = 9$, $\partial f/\partial x_2 = 2x_1 + 6x_2 - 1 = 13$. ✓

Stationary point. $2A\boldsymbol{x} = -\boldsymbol{b}$ gives $\boldsymbol{x}^\star = -\frac{1}{2}A^{-1}\boldsymbol{b}$. With $A^{-1} = \frac{1}{5}\left[\begin{smallmatrix} 3 & -1 \\ -1 & 2 \end{smallmatrix}\right]$, $A^{-1}\boldsymbol{b} = (\frac{4}{5}, -\frac{3}{5})$ and $\boldsymbol{x}^\star = (-\frac{2}{5}, \frac{3}{10})$.

Minimum or not? The Hessian is $2A$, with eigenvalues $5 \pm \sqrt{5} > 0$: positive definite, so $\boldsymbol{x}^\star$ is the global minimum.

4.2 Linear regression: the normal equations

Bishop's linear regression [Bishop Ch. 4] fits $y(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x})$ by minimising the sum-of-squares error

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left(t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n\right)^2 = \frac{1}{2}\left\|\boldsymbol{t} - \Phi\boldsymbol{w}\right\|^2,$$

where row n of the design matrix Φ is $\boldsymbol{\phi}_n^{\mathsf{T}}$. By Theorem 4.1,

$$\nabla E(\boldsymbol{w}) = \Phi^{\mathsf{T}}(\Phi\boldsymbol{w} - \boldsymbol{t}) = -\sum_n (t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n)\boldsymbol{\phi}_n.$$

Setting it to zero gives the **normal equations** $\Phi^{\mathsf{T}}\Phi\boldsymbol{w} = \Phi^{\mathsf{T}}\boldsymbol{t}$, so $\boldsymbol{w}_{\mathrm{ML}} = (\Phi^{\mathsf{T}}\Phi)^{-1}\Phi^{\mathsf{T}}\boldsymbol{t} = \Phi^{+}\boldsymbol{t}$ — the pseudo-inverse of the SVD course. Adding a weight-decay term $\frac{\lambda}{2}\|\boldsymbol{w}\|^2$ adds $\lambda\boldsymbol{w}$ to the gradient, and the solution becomes $(\Phi^{\mathsf{T}}\Phi + \lambda I)^{-1}\Phi^{\mathsf{T}}\boldsymbol{t}$.

The form $-\sum_n(\text{error}_n)\boldsymbol{\phi}_n$ of the gradient is worth noticing: it is a sum over data points of "error times input". The same shape reappears for logistic regression, for softmax regression, and in backpropagation.

4.3 Jacobians of vector functions

Theorem 4.2: Basic Jacobians.

$$\frac{\partial(A\boldsymbol{x})}{\partial\boldsymbol{x}} = A, \qquad \frac{\partial h(\boldsymbol{x})}{\partial\boldsymbol{x}} = \operatorname{diag}\left(h'(x_1), \ldots, h'(x_n)\right) \ \text{(element-wise } h\text{)},$$
$$\frac{\partial\,\mathrm{softmax}(\boldsymbol{a})}{\partial\boldsymbol{a}} = \operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^{\mathsf{T}}, \quad \frac{\partial}{\partial\boldsymbol{x}}\frac{\boldsymbol{x}}{\|\boldsymbol{x}\|} = \frac{1}{\|\boldsymbol{x}\|}\left(I - \frac{\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}}{\|\boldsymbol{x}\|^2}\right).$$

The softmax Jacobian. With $s_k = e^{a_k}/Z$ and $Z = \sum_j e^{a_j}$, the quotient rule gives

$$\frac{\partial s_k}{\partial a_j} = \frac{e^{a_k}\delta_{kj}Z - e^{a_k}e^{a_j}}{Z^2} = s_k\delta_{kj} - s_k s_j,$$

which is the (k, j) entry of $\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^{\mathsf{T}}$. Every row sums to zero (because $\sum_k s_k = 1$ cannot change), and the matrix is symmetric and positive semidefinite.

Worked Example 4.2: The softmax Jacobian.

Let $\boldsymbol{a} = (0, \ln 2, 0)$, so $e^{\boldsymbol{a}} = (1, 2, 1)$ and $\boldsymbol{s} = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$.

Formula. $\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^{\mathsf{T}} = \frac{1}{16}\begin{bmatrix}4&0&0\\0&8&0\\0&0&4\end{bmatrix} - \frac{1}{16}\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix} = \frac{1}{16}\begin{bmatrix}3&-2&-1\\-2&4&-2\\-1&-2&3\end{bmatrix}.$

Check. Rows sum to 0. Increasing a_2 raises s_2 ($J_{22} > 0$) and lowers the others.

4.4 The log-sum-exp and the sigmoid

The **log-sum-exp** $\operatorname{lse}(\boldsymbol{a}) = \ln \sum_k e^{a_k}$ is the log of softmax's normaliser. Its gradient is the softmax itself and its Hessian is the softmax Jacobian:

$$\nabla \operatorname{lse}(\boldsymbol{a}) = \boldsymbol{s} = \operatorname{softmax}(\boldsymbol{a}), \qquad \nabla^2 \operatorname{lse}(\boldsymbol{a}) = \operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\top \succeq 0.$$

So lse is convex — the reason the cross-entropy loss of softmax regression is convex in the logits.

The **logistic sigmoid** $\sigma(a) = 1/(1+e^{-a})$ satisfies $\sigma'(a) = \sigma(a)(1-\sigma(a))$ and $1-\sigma(a) = \sigma(-a)$. For logistic regression [Bishop Ch. 5] with $y_n = \sigma(\boldsymbol{w}^\top \boldsymbol{\phi}_n)$ and the cross-entropy error $E(\boldsymbol{w}) = -\sum_n \left[t_n \ln y_n + (1-t_n)\ln(1-y_n)\right]$:

$$\nabla E(\boldsymbol{w}) = \sum_n (y_n - t_n)\boldsymbol{\phi}_n = \Phi^\top(\boldsymbol{y} - \boldsymbol{t}).$$

Again "error times input". The derivation: $\partial/\partial a[-t\ln\sigma(a) - (1-t)\ln(1-\sigma(a))] = -t(1-\sigma) + (1-t)\sigma = \sigma - t$, then the chain rule through $a = \boldsymbol{w}^\top\boldsymbol{\phi}$ multiplies by $\boldsymbol{\phi}$.

4.5 Hessians and stationary points

Theorem 4.3: Basic Hessians. $\nabla^2(\boldsymbol{x}^\top A\boldsymbol{x}) = A + A^\top$; $\nabla^2 \frac{1}{2}\|\Phi\boldsymbol{w} - \boldsymbol{t}\|^2 = \Phi^\top\Phi$; $\nabla^2 \operatorname{lse}(\boldsymbol{a}) = \operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\top$; for logistic regression $\nabla^2 E = \Phi^\top R\Phi$ with $R = \operatorname{diag}(y_n(1-y_n))$.

At a stationary point $\nabla f(\boldsymbol{x}^\star) = \boldsymbol{0}$, the quadratic approximation is $f(\boldsymbol{x}^\star + \boldsymbol{h}) \approx f(\boldsymbol{x}^\star) + \frac{1}{2}\boldsymbol{h}^\top H\boldsymbol{h}$, so

- H positive definite (all eigenvalues > 0): local minimum;
- H negative definite: local maximum;
- H with eigenvalues of both signs: saddle point — the typical stationary point of a neural network loss in high dimensions.

Worked Example 4.3: Classifying stationary points.

$f(x,y) = x^3 - 3x + y^2$.

Gradient. $\nabla f = (3x^2 - 3,\ 2y) = \boldsymbol{0}$ at $(1,0)$ and $(-1,0)$.

Hessian. $H = \left[\begin{smallmatrix} 6x & 0 \\ 0 & 2 \end{smallmatrix}\right]$.

At $(1,0)$. $H = \operatorname{diag}(6,2)$, positive definite: local minimum.

At $(-1,0)$. $H = \operatorname{diag}(-6,2)$, indefinite: saddle point. Moving along x decreases f; moving along y increases it.

Exercises

Exercise 4.1 ⋆ Compute $\nabla(\boldsymbol{x}^{\mathsf{T}} A\boldsymbol{x})$ for the non-symmetric $A = \left[\begin{smallmatrix} 1 & 4 \\ 0 & 2 \end{smallmatrix}\right]$ at $\boldsymbol{x} = (1,1)$ in two ways: from the formula $(A + A^{\mathsf{T}})\boldsymbol{x}$, and from the expanded polynomial.

Exercise 4.2 ⋆ Fit a line $y = w_0 + w_1 x$ to $(0,1), (1,3), (2,4)$ by solving the normal equations. Give ∇E at $\boldsymbol{w} = \mathbf{0}$.

Exercise 4.3 ⋆⋆ Derive the gradient of the ridge objective $E(\boldsymbol{w}) = \frac{1}{2}\left\|\boldsymbol{t} - \Phi\boldsymbol{w}\right\|^2 + \frac{\lambda}{2}\left\|\boldsymbol{w}\right\|^2$ and its minimiser. Show that the Hessian is positive definite for every $\lambda > 0$, even when Φ has dependent columns.

Exercise 4.4 ⋆⋆ Show that $\nabla \operatorname{lse}(\boldsymbol{a}) = \operatorname{softmax}(\boldsymbol{a})$, and compute the gradient of the cross-entropy $\ell(\boldsymbol{a}) = -\sum_k t_k \ln s_k$ with $\sum_k t_k = 1$. *Hint:* $\ln s_k = a_k - \operatorname{lse}(\boldsymbol{a})$.

Exercise 4.5 ⋆⋆ Compute the Jacobian of $\boldsymbol{g}(\boldsymbol{x}) = \boldsymbol{x}/\left\|\boldsymbol{x}\right\|$ at $\boldsymbol{x} = (3,4)$, and verify that $J\boldsymbol{x} = \mathbf{0}$. Why must that be so?

Exercise 4.6 ⋆⋆ Find and classify the stationary points of $f(x,y) = x^2 + xy + y^2 - 3x$.

Exercise 4.7 ⋆⋆⋆ For logistic regression, show that $\nabla^2 E = \Phi^{\mathsf{T}} R \Phi$ with $R = \operatorname{diag}(y_n(1 - y_n))$, and conclude that E is convex. When is it strictly convex?

CHAPTER 5

Curvature and Optimisation

Gradients tell us which way is downhill; Hessians tell us how the slope changes, and therefore how large a step we can safely take. This chapter uses the quadratic approximation of a loss to explain three facts that Bishop & Bishop discuss in their chapter on gradient descent [Bishop Ch. 7]: why the learning rate has a hard upper limit, why badly scaled problems converge slowly, and why normalising the inputs helps.

5.1 The local quadratic model

Near a minimum $\boldsymbol{w}^\star$ the gradient vanishes, so the quadratic approximation (1.2) reads

$$E(\boldsymbol{w}) \approx E(\boldsymbol{w}^\star) + \tfrac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^\star)^{\mathsf{T}} H (\boldsymbol{w}-\boldsymbol{w}^\star), \qquad \nabla E(\boldsymbol{w}) \approx H(\boldsymbol{w}-\boldsymbol{w}^\star).$$

Write $H = \sum_i \lambda_i \boldsymbol{q}_i \boldsymbol{q}_i^{\mathsf{T}}$ and expand the displacement in the eigenvectors, $\boldsymbol{w}-\boldsymbol{w}^\star = \sum_i \alpha_i \boldsymbol{q}_i$. Then

$$E(\boldsymbol{w}) - E(\boldsymbol{w}^\star) \approx \tfrac{1}{2}\sum_i \lambda_i \alpha_i^2 .$$

The contours of E are ellipses (ellipsoids) whose axes point along the eigenvectors of H, with half-lengths proportional to $1/\sqrt{\lambda_i}$: a large eigenvalue means a steep, narrow valley.

5.2 Gradient descent on a quadratic

Gradient descent with learning rate η updates $\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \eta \nabla E(\boldsymbol{w}_k) = \boldsymbol{w}_k - \eta H(\boldsymbol{w}_k - \boldsymbol{w}^\star)$. In eigen-coordinates the directions decouple:

$$\alpha_i^{(k+1)} = (1-\eta\lambda_i)\,\alpha_i^{(k)} \qquad \Longrightarrow \qquad \alpha_i^{(k)} = (1-\eta\lambda_i)^k \alpha_i^{(0)}.$$

Theorem 5.1: Convergence of gradient descent on a quadratic. Let H be positive definite with eigenvalues $\lambda_{\min} \le \cdots \le \lambda_{\max}$.

1. Gradient descent converges from every starting point if and only if $0 < \eta < 2/\lambda_{\max}$.
2. The error shrinks by the factor $\rho(\eta) = \max_i |1-\eta\lambda_i|$ per step.
3. The best fixed step is $\eta^\star = \dfrac{2}{\lambda_{\min}+\lambda_{\max}}$, giving $\rho^\star = \dfrac{\kappa-1}{\kappa+1}$ with $\kappa = \lambda_{\max}/\lambda_{\min}$, the condition number of H.

Proof of 3. $|1-\eta\lambda|$ is largest at an end of the spectrum. Balancing the two ends, $1-\eta\lambda_{\min} = -(1-\eta\lambda_{\max})$, gives $\eta^\star$; then $1-\eta^\star\lambda_{\min} = \frac{\lambda_{\max}-\lambda_{\min}}{\lambda_{\max}+\lambda_{\min}} = \frac{\kappa-1}{\kappa+1}$. □

Key idea. The step size is limited by the *steepest* direction ($\eta < 2/\lambda_{\max}$), but the number of steps is set by the *flattest* one. Their ratio, the condition number κ, governs the speed: reducing the error by a factor e takes roughly $\kappa/2$ steps.

Worked Example 5.1: Gradient descent with three learning rates.

$E(\boldsymbol{w}) = \frac{1}{2}\boldsymbol{w}^{\mathsf{T}} H \boldsymbol{w}$ with $H = \mathrm{diag}(1, 10)$, $\boldsymbol{w}^\star = \mathbf{0}$, $\boldsymbol{w}_0 = (1, 1)$. Per-step factors are $1 - \eta$ and $1 - 10\eta$.

$\eta = 0.1$. Factors 0.9 and 0: after one step $\boldsymbol{w}_1 = (0.9, 0)$. The steep direction is solved instantly; the flat one creeps (0.9^k).

$\eta = 0.19$. Factors 0.81 and -0.9: the steep coordinate flips sign every step while shrinking — the familiar zig-zag across a valley.

$\eta = 0.21$. Factor $1 - 2.1 = -1.1$: $|-1.1| > 1$ and the steep coordinate grows by 10% per step. Divergence, because $\eta > 2/\lambda_{\max} = 0.2$.

Optimal. $\eta^\star = 2/11$ gives factors $\pm\frac{9}{11}$, i.e. $\rho^\star = \frac{\kappa-1}{\kappa+1} = \frac{9}{11} \approx 0.82$ with $\kappa = 10$.

Momentum. Adding a fraction of the previous step, $\boldsymbol{w}_{k+1} = \boldsymbol{w}_k - \eta \nabla E(\boldsymbol{w}_k) + \mu(\boldsymbol{w}_k - \boldsymbol{w}_{k-1})$, lets the iterates build up speed along flat directions. With well-chosen η, μ the rate on a quadratic improves from $\frac{\kappa-1}{\kappa+1}$ to $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ — for $\kappa = 100$, from about 0.98 to 0.82 per step.

5.3 Why normalising inputs helps

For linear regression the Hessian is $H = \Phi^{\mathsf{T}}\Phi$. If one feature is measured in metres and another in millimetres, the second column of Φ is a thousand times larger, so one eigenvalue of H is roughly a million times larger: $\kappa \approx 10^6$ and gradient descent crawls. Rescaling each input to zero mean and unit variance makes H close to a multiple of the correlation matrix, typically with small κ. Batch and layer normalisation [Bishop Ch. 7] apply the same idea to the inputs of every layer; Chapter 12 computes their derivatives.

5.4 Newton's method

Minimising the quadratic model $E(\boldsymbol{w}) + \boldsymbol{g}^{\mathsf{T}}\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^{\mathsf{T}} H \boldsymbol{h}$ over the step $\boldsymbol{h}$ gives $H\boldsymbol{h} = -\boldsymbol{g}$:

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} - H^{-1}\nabla E(\boldsymbol{w}).$$

On an exact quadratic, Newton's method lands on the minimum in *one* step, whatever κ is, because H^{-1} undoes the distortion of the ellipses. Near a minimum of a general smooth function it converges quadratically (the number of correct digits doubles each step). Its drawbacks for deep networks are the cost of forming and solving with an $n \times n$ Hessian when n is millions, and the fact that at a saddle point it is attracted to the saddle.

For logistic regression [Bishop Ch. 5], $\nabla E = \Phi^{\mathsf{T}}(\boldsymbol{y} - \boldsymbol{t})$ and $H = \Phi^{\mathsf{T}} R \Phi$, so Newton's method is

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} - (\Phi^{\mathsf{T}} R \Phi)^{-1}\Phi^{\mathsf{T}}(\boldsymbol{y} - \boldsymbol{t}),$$

known as **iteratively reweighted least squares** (IRLS), because each step solves a weighted least-squares problem.

Worked Example 5.2: One Newton step for logistic regression.

One feature, no bias: $x = (1, 2, -1, -2)$ with labels $t = (1, 0, 0, 1)$. Start at $w = 0$, where every $y_n = \sigma(0) = \tfrac{1}{2}$.

Gradient. $g = \sum_n (y_n - t_n) x_n = (-\tfrac{1}{2})(1) + (\tfrac{1}{2})(2) + (\tfrac{1}{2})(-1) + (-\tfrac{1}{2})(-2) = 1.$

Hessian. $H = \sum_n y_n(1 - y_n) x_n^2 = \tfrac{1}{4}(1 + 4 + 1 + 4) = \tfrac{5}{2}.$

Newton step. $w_1 = 0 - g/H = -\tfrac{2}{5}.$

Compare. Gradient descent with $\eta = 0.1$ would move only to $w = -0.1$. Newton rescales the step by the curvature.

5.5 The Gauss–Newton approximation

For a sum-of-squares loss $E = \tfrac{1}{2}\sum_n (y_n(\boldsymbol{w}) - t_n)^2$ the exact Hessian is

$$H = \sum_n \nabla y_n \nabla y_n^{\mathsf{T}} + \sum_n (y_n - t_n) \nabla^2 y_n.$$

Near a good fit the residuals $y_n - t_n$ are small, so the second sum is dropped, leaving the **outer-product** or Gauss–Newton approximation $H \approx \sum_n \nabla y_n \nabla y_n^{\mathsf{T}} = J^{\mathsf{T}} J$, which needs only first derivatives and is always positive semidefinite [Bishop Ch. 8].

Exercises

Exercise 5.1 $\star$ For $H = \mathrm{diag}(2, 50)$, give the largest learning rate for which gradient descent converges, the optimal learning rate, and the resulting convergence factor.

Exercise 5.2 $\star$ Show that one Newton step from any $\boldsymbol{w}_0$ reaches the minimum of $E(\boldsymbol{w}) = \tfrac{1}{2}\boldsymbol{w}^{\mathsf{T}} H \boldsymbol{w} - \boldsymbol{b}^{\mathsf{T}} \boldsymbol{w}$ (H positive definite).

Exercise 5.3 $\star\star$ Run three steps of gradient descent by hand on $E(\boldsymbol{w}) = \tfrac{1}{2}(w_1^2 + 4w_2^2)$ from $\boldsymbol{w}_0 = (1, 1)$ with $\eta = 0.3$. Predict the iterates from the factors $1 - \eta\lambda_i$ first.

Exercise 5.4 $\star\star$ Linear regression with features $x_1 \in [0, 1]$ and $x_2 = 1000 x_1 + \text{noise}$ has an ill-conditioned Hessian. Explain, using $H = \Phi^{\mathsf{T}}\Phi$, why (a) scaling x_2 by $1/1000$ improves κ, but (b) does not fully fix it. What else would?

Exercise 5.5 $\star\star$ Continue Worked Example 5.2: compute the gradient at $w_1 = -0.4$ and say whether a second Newton step will move left or right. *Hint:* $\sigma(0.4) \approx 0.599$, $\sigma(0.8) \approx 0.690$.

Exercise 5.6 $\star\star\star$ For the sum-of-squares loss $E = \frac{1}{2}\sum_n (y_n(\boldsymbol{w}) - t_n)^2$, derive the exact Hessian given in the text. Show that the Gauss–Newton matrix $J^{\mathsf{T}}J$ (rows of J are ∇y_n^{T}) is positive semidefinite, and that the exact Hessian need not be.

CHAPTER 6

The Differential Method: Derivatives with Respect to Matrices

Computing $\partial f / \partial x_{ij}$ entry by entry works for 2×2 examples and fails for everything else: the bookkeeping of indices quickly becomes unmanageable, and the answer is hard to recognise as a matrix formula. The **differential method** avoids indices entirely. You compute how f changes when X changes by a small $\mathrm{d}X$, using a short list of rules, and rearrange the result with the cyclic trace until it has the form $\operatorname{tr}(G^{\mathsf{T}}\mathrm{d}X)$. Then G is the gradient. This chapter derives every matrix derivative in Bishop & Bishop's Appendix A this way.

6.1 Differentials

The **differential** $\mathrm{d}f$ is the first-order part of the change in f: $f(X + \mathrm{d}X) = f(X) + \mathrm{d}f +$ (higher-order terms). By the matrix linear approximation (1.3),

Theorem 6.1: The identification rule. If, for every $\mathrm{d}X$, the differential of a scalar function can be written as

$$\mathrm{d}f = \operatorname{tr}\left(G^{\mathsf{T}}\mathrm{d}X\right) = \langle G, \mathrm{d}X\rangle_F,$$

then $\frac{\partial f}{\partial X} = G$. For vectors the rule reads: if $\mathrm{d}f = \boldsymbol{g}^{\mathsf{T}}\mathrm{d}\boldsymbol{x}$, then $\nabla f = \boldsymbol{g}$.

Theorem 6.2: Rules for differentials. For constant A and matrix functions X, Y of the variable:

$$\begin{aligned}
\mathrm{d}A &= 0, & \mathrm{d}(X+Y) &= \mathrm{d}X + \mathrm{d}Y,\\
\mathrm{d}(\alpha X) &= \alpha\,\mathrm{d}X, & \mathrm{d}(X^{\mathsf{T}}) &= (\mathrm{d}X)^{\mathsf{T}},\\
\mathrm{d}(XY) &= (\mathrm{d}X)Y + X(\mathrm{d}Y), & \mathrm{d}\operatorname{tr}X &= \operatorname{tr}(\mathrm{d}X),\\
\mathrm{d}(X^{-1}) &= -X^{-1}(\mathrm{d}X)X^{-1}, & \mathrm{d}\ln|X| &= \operatorname{tr}(X^{-1}\mathrm{d}X),\\
\mathrm{d}|X| &= |X|\operatorname{tr}(X^{-1}\mathrm{d}X), & \mathrm{d}(X\odot Y) &= \mathrm{d}X\odot Y + X\odot\mathrm{d}Y,\\
\mathrm{d}\,h(X) &= h'(X)\odot\mathrm{d}X \ \text{(element-wise } h).
\end{aligned}$$

Proof of the less obvious rules. Inverse: differentiate $XX^{-1} = I$: $(\mathrm{d}X)X^{-1} + X\,\mathrm{d}(X^{-1}) = 0$, so $\mathrm{d}(X^{-1}) = -X^{-1}(\mathrm{d}X)X^{-1}$.

Log-determinant: $|X + \mathrm{d}X| = |X|\,\left|I + X^{-1}\mathrm{d}X\right|$. For small E, $\det(I+E) = \prod(1+\lambda_i(E)) \approx 1 + \sum\lambda_i(E) = 1 + \operatorname{tr}E$. So $\ln|X+\mathrm{d}X| \approx \ln|X| + \ln(1+\operatorname{tr}(X^{-1}\mathrm{d}X)) \approx \ln|X| + \operatorname{tr}(X^{-1}\mathrm{d}X)$. The rule for $\mathrm{d}|X|$ follows from $\mathrm{d}|X| = |X|\,\mathrm{d}\ln|X|$. □

The differential method.

1. Compute $\mathrm{d}f$ with the rules above. Keep the order of matrix products.
2. If $\mathrm{d}f$ is a scalar but not yet a trace, wrap it: $\mathrm{d}f = \operatorname{tr}(\mathrm{d}f)$.
3. Use the cyclic property and $\operatorname{tr}(M^\mathsf{T}) = \operatorname{tr}(M)$ to move $\mathrm{d}X$ to the far right: $\mathrm{d}f = \operatorname{tr}(M\,\mathrm{d}X)$.
4. Read off $\frac{\partial f}{\partial X} = M^\mathsf{T}$.
5. Check the shape against X.

6.2 The standard matrix derivatives

Theorem 6.3: Matrix derivatives. For constant matrices and vectors of compatible shapes:

$$\frac{\partial}{\partial X}\operatorname{tr}(AX) = A^\mathsf{T}, \qquad \frac{\partial}{\partial X}\operatorname{tr}(XA) = A^\mathsf{T},$$

$$\frac{\partial}{\partial X}\operatorname{tr}(X^\mathsf{T}A) = A, \qquad \frac{\partial}{\partial X}\operatorname{tr}(AXB) = A^\mathsf{T}B^\mathsf{T},$$

$$\frac{\partial}{\partial X}\operatorname{tr}(X^\mathsf{T}AX) = (A + A^\mathsf{T})X, \qquad \frac{\partial}{\partial X}\,\boldsymbol{a}^\mathsf{T}X\boldsymbol{b} = \boldsymbol{a}\boldsymbol{b}^\mathsf{T},$$

$$\frac{\partial}{\partial X}\ln|X| = X^{-\mathsf{T}}, \qquad \frac{\partial}{\partial X}|X| = |X|\,X^{-\mathsf{T}},$$

$$\frac{\partial}{\partial X}\operatorname{tr}(X^{-1}A) = -X^{-\mathsf{T}}A^\mathsf{T}X^{-\mathsf{T}}, \qquad \frac{\partial}{\partial X}\,\boldsymbol{a}^\mathsf{T}X^{-1}\boldsymbol{b} = -X^{-\mathsf{T}}\boldsymbol{a}\boldsymbol{b}^\mathsf{T}X^{-\mathsf{T}},$$

$$\frac{\partial}{\partial X}\|AX - B\|_F^2 = 2A^\mathsf{T}(AX - B), \qquad \frac{\partial}{\partial X}\|XA - B\|_F^2 = 2(XA - B)A^\mathsf{T}.$$

Derivations. Each line follows the recipe.

- $\operatorname{tr}(AXB)$: $\mathrm{d}f = \operatorname{tr}(A\,\mathrm{d}X\,B) = \operatorname{tr}(BA\,\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = (BA)^\mathsf{T} = A^\mathsf{T}B^\mathsf{T}$. With $B = I$: $\operatorname{tr}(AX) \to A^\mathsf{T}$.
- $\operatorname{tr}(X^\mathsf{T}AX)$: $\mathrm{d}f = \operatorname{tr}(\mathrm{d}X^\mathsf{T}AX) + \operatorname{tr}(X^\mathsf{T}A\,\mathrm{d}X) = \operatorname{tr}(X^\mathsf{T}A^\mathsf{T}\mathrm{d}X) + \operatorname{tr}(X^\mathsf{T}A\,\mathrm{d}X)$ (transpose the first trace), so $\frac{\partial f}{\partial X} = \big(X^\mathsf{T}(A^\mathsf{T} + A)\big)^\mathsf{T} = (A + A^\mathsf{T})X$.
- $\boldsymbol{a}^\mathsf{T}X\boldsymbol{b}$: $\mathrm{d}f = \boldsymbol{a}^\mathsf{T}\mathrm{d}X\,\boldsymbol{b} = \operatorname{tr}(\boldsymbol{b}\boldsymbol{a}^\mathsf{T}\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = (\boldsymbol{b}\boldsymbol{a}^\mathsf{T})^\mathsf{T} = \boldsymbol{a}\boldsymbol{b}^\mathsf{T}$.
- $\ln|X|$: $\mathrm{d}f = \operatorname{tr}(X^{-1}\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = X^{-\mathsf{T}}$.
- $\operatorname{tr}(X^{-1}A)$: $\mathrm{d}f = -\operatorname{tr}(X^{-1}\mathrm{d}X\,X^{-1}A) = -\operatorname{tr}(X^{-1}AX^{-1}\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = -(X^{-1}AX^{-1})^\mathsf{T}$.
- $\boldsymbol{a}^\mathsf{T}X^{-1}\boldsymbol{b}$: $\mathrm{d}f = -\boldsymbol{a}^\mathsf{T}X^{-1}\mathrm{d}X\,X^{-1}\boldsymbol{b} = -\operatorname{tr}(X^{-1}\boldsymbol{b}\boldsymbol{a}^\mathsf{T}X^{-1}\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = -X^{-\mathsf{T}}\boldsymbol{a}\boldsymbol{b}^\mathsf{T}X^{-\mathsf{T}}$.
- $\|AX - B\|_F^2 = \operatorname{tr}(R^\mathsf{T}R)$ with $R = AX - B$: $\mathrm{d}f = 2\operatorname{tr}(R^\mathsf{T}\mathrm{d}R) = 2\operatorname{tr}(R^\mathsf{T}A\,\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = 2A^\mathsf{T}R$.

□

Worked Example 6.1: The gradient of the log-determinant, checked entry by entry.

$X = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$, so $\det X = 1$ and $X^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$.

Formula. $\frac{\partial}{\partial X} \ln |X| = X^{-\mathsf{T}} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$.

Check x_{11}. $\det(X + \epsilon E_{11}) = (2+\epsilon)\cdot 1 - 1 = 1 + \epsilon$, so $\ln\det \approx \epsilon$: derivative 1. ✓

Check x_{12}. $\det(X + \epsilon E_{12}) = 2 - (1+\epsilon) = 1 - \epsilon$: derivative -1. ✓

Check x_{22}. $\det = 2(1+\epsilon) - 1 = 1 + 2\epsilon$: derivative 2. ✓

Worked Example 6.2: A trace of a quadratic form.

$f(X) = \operatorname{tr}(X^{\mathsf{T}} A X)$ with $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ (not symmetric) and $X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$.

Formula. $\frac{\partial f}{\partial X} = (A + A^{\mathsf{T}})X = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 4 & 2 \end{bmatrix}$.

Check one entry. Increase x_{21} by ϵ: $X^{\mathsf{T}}AX$ has trace $f = \sum_j \boldsymbol{x}_j^{\mathsf{T}} A \boldsymbol{x}_j$ over the columns $\boldsymbol{x}_j$. Only column 1, $\boldsymbol{x}_1 = (1, 1+\epsilon)$, changes: $\boldsymbol{x}_1^{\mathsf{T}} A \boldsymbol{x}_1 = x_{11}^2 + 2x_{11}x_{21} + x_{21}^2 = 1 + 2(1+\epsilon) + (1+\epsilon)^2$, whose derivative at $\epsilon = 0$ is $2 + 2 = 4$. ✓

6.3 Multi-output regression

Bishop's linear regression with K outputs [Bishop Ch. 4] stacks targets in an $N \times K$ matrix T and weights in a $D \times K$ matrix W, with predictions ΦW. The error and its gradient are

$$E(W) = \tfrac{1}{2}\,\|\Phi W - T\|_F^2, \qquad \frac{\partial E}{\partial W} = \Phi^{\mathsf{T}}(\Phi W - T),$$

so $W_{\mathrm{ML}} = (\Phi^{\mathsf{T}}\Phi)^{-1}\Phi^{\mathsf{T}} T = \Phi^{+} T$: each output column is solved independently with the same pseudo-inverse.

6.4 Element-wise functions and Hadamard products

Neural networks apply functions element by element. Two facts handle all such cases:

$$\operatorname{tr}\left(A^{\mathsf{T}}(B \odot C)\right) = \operatorname{tr}\left((A \odot B)^{\mathsf{T}} C\right), \qquad \sum_{ij} h(x_{ij}) \text{ has gradient } h'(X) \text{ (element-wise)}.$$

The first says a Hadamard factor can be moved from one side of the Frobenius inner product to the other. Example: for $f(X) = \operatorname{tr}(A^{\mathsf{T}} \tanh(X))$, $\mathrm{d}f = \operatorname{tr}(A^{\mathsf{T}}(\tanh'(X) \odot \mathrm{d}X)) = \operatorname{tr}((A \odot \tanh'(X))^{\mathsf{T}} \mathrm{d}X)$, so $\frac{\partial f}{\partial X} = A \odot (1 - \tanh^2(X))$. This is exactly the backward pass of an activation layer (Chapter 8).

6.5 Symmetric matrices

When X is constrained to be symmetric (a covariance), its entries x_{ij} and x_{ji} are the same variable, and some books then write formulas like $\frac{\partial}{\partial X}\ln|X| = 2X^{-1} - \operatorname{diag}(X^{-1})$. For optimisation and maximum likelihood you can avoid this complication: treat all n^2 entries as independent, compute the unconstrained gradient (which is symmetric at a symmetric X for all the functions in this course), and set it to zero. The stationary point you find is symmetric, so it is also the constrained one. This is what Bishop does for the Gaussian covariance, and what Chapter 7 does. In code, a cleaner option is to parameterise a covariance through an unconstrained factor, $\Sigma = LL^{\mathsf{T}}$.

6.6 Index notation, as a fallback

When a function does not fit the patterns above, write it with explicit indices and the *Einstein convention* (sum over repeated indices). For example, for $f = \boldsymbol{a}^{\mathsf{T}} X \boldsymbol{b} = a_i X_{ij} b_j$: $\partial f/\partial X_{kl} = a_i \delta_{ik} \delta_{jl} b_j = a_k b_l$, i.e. $\boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}$. Index notation is mechanical and always works, and NumPy's `einsum` lets you write such formulas directly in code; but the result must still be recognised as a matrix expression, which is where the differential method is better.

Exercises

Exercise 6.1 $\star$ Use the differential method to find $\frac{\partial}{\partial X}\operatorname{tr}(AXB)$ and check its shape when A is $p \times m$, X is $m \times n$ and B is $n \times p$.

Exercise 6.2 $\star$ Find $\nabla_{\boldsymbol{x}}$ of $f(\boldsymbol{x}) = \boldsymbol{x}^{\mathsf{T}} A \boldsymbol{x}$ with the differential method.

Exercise 6.3 $\star$ Find $\frac{\partial}{\partial X}|X|$ for $X = \left[\begin{smallmatrix} 3 & 1 \\ 2 & 4 \end{smallmatrix}\right]$ from the formula, and compare with Worked Example 1.3.

Exercise 6.4 $\star\star$ Derive $\frac{\partial}{\partial X}\|XA - B\|_F^2$.

Exercise 6.5 $\star\star$ Derive $\frac{\partial}{\partial X}\operatorname{tr}(X^2)$ and $\frac{\partial}{\partial X}\operatorname{tr}(X^{\mathsf{T}}X)$. Why are they different?

Exercise 6.6 $\star\star$ Let $f(X) = \ln\left|X^{\mathsf{T}}X\right|$ for an $m \times n$ matrix X with independent columns. Show that $\frac{\partial f}{\partial X} = 2X(X^{\mathsf{T}}X)^{-1}$.

Exercise 6.7 $\star\star$ For $f(X) = \sum_{ij} \sigma(x_{ij})\, a_{ij} = \operatorname{tr}(A^{\mathsf{T}}\sigma(X))$ with the sigmoid applied elementwise, find $\frac{\partial f}{\partial X}$.

Exercise 6.8 $\star\star\star$ Using index notation, derive $\frac{\partial}{\partial X}\operatorname{tr}(X^{-1}A)$ and confirm the formula of Theorem 6.3. *Hint:* $\partial(X^{-1})_{ij}/\partial X_{kl} = -(X^{-1})_{ik}(X^{-1})_{lj}$.

CHAPTER 7

Gaussians and Maximum Likelihood

The multivariate Gaussian is the distribution Bishop & Bishop use most [Bishop Ch. 3], and its formulas are where matrix derivatives first become unavoidable: the log-density contains a log-determinant and an inverse, the maximum-likelihood covariance needs the derivative of both, conditional distributions need the Schur complement, and variational autoencoders need the KL divergence between two Gaussians and its gradient [Bishop Ch. 19]. This chapter derives all of them with the tools of Chapters 3 and 6.

7.1 The log-density

For $\boldsymbol{x} \in \mathbb{R}^D$ with mean $\boldsymbol{\mu}$ and positive definite covariance Σ,

$$\ln \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \Sigma) = -\frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\Sigma| - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu}).$$

It is often easier to work with the **precision matrix** $\Lambda = \Sigma^{-1}$, because then the log-density is linear in Λ apart from $\frac{1}{2}\ln|\Lambda|$ (note $\ln|\Sigma| = -\ln|\Lambda|$).

7.2 Maximum likelihood for the mean and covariance

For data $\boldsymbol{x}_1, \dots, \boldsymbol{x}_N$, the log-likelihood is

$$\ell(\boldsymbol{\mu}, \Sigma) = -\frac{ND}{2}\ln 2\pi - \frac{N}{2}\ln|\Sigma| - \frac{1}{2}\sum_{n=1}^{N}(\boldsymbol{x}_n-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu}).$$

Theorem 7.1: Maximum-likelihood Gaussian. ℓ is maximised by the sample mean and the (biased) sample covariance:

$$\boldsymbol{\mu}_{\mathrm{ML}} = \frac{1}{N}\sum_n \boldsymbol{x}_n, \qquad \Sigma_{\mathrm{ML}} = \frac{1}{N}\sum_n (\boldsymbol{x}_n-\boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n-\boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}} = \frac{1}{N}S.$$

Derivation. Mean. Each quadratic term has gradient $-\Sigma^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu})$ w.r.t. $\boldsymbol{x}_n-\boldsymbol{\mu}$, hence $+\Sigma^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu})$ w.r.t. $\boldsymbol{\mu}$ (the minus signs cancel). So $\nabla_{\boldsymbol{\mu}}\ell = \Sigma^{-1}\sum_n(\boldsymbol{x}_n-\boldsymbol{\mu}) = \mathbf{0}$, and $\boldsymbol{\mu} = \frac{1}{N}\sum \boldsymbol{x}_n$.

Covariance. Use the trace trick (Theorem 3.2) with $S = \sum_n(\boldsymbol{x}_n-\boldsymbol{\mu})(\boldsymbol{x}_n-\boldsymbol{\mu})^{\mathsf{T}}$ and write ℓ in terms of $\Lambda = \Sigma^{-1}$:

$$\ell = \text{const} + \frac{N}{2}\ln|\Lambda| - \frac{1}{2}\operatorname{tr}(\Lambda S).$$

By Theorem 6.3, $\frac{\partial \ell}{\partial \Lambda} = \frac{N}{2}\Lambda^{-\mathsf{T}} - \frac{1}{2}S^{\mathsf{T}} = \frac{N}{2}\Sigma - \frac{1}{2}S$, using the symmetry of Λ and S. Setting this to zero gives $\Sigma = S/N$. □

Differentiating with respect to Σ directly gives the same answer and a formula worth knowing on its own:

$$\frac{\partial}{\partial \Sigma} \ln \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \Sigma) = -\frac{1}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})(\boldsymbol{x}-\boldsymbol{\mu})^{\top}\Sigma^{-1}. \tag{7.1}$$

(Use $\frac{\partial}{\partial \Sigma}\ln|\Sigma| = \Sigma^{-1}$ and $\frac{\partial}{\partial \Sigma}\boldsymbol{a}^{\top}\Sigma^{-1}\boldsymbol{a} = -\Sigma^{-1}\boldsymbol{a}\boldsymbol{a}^{\top}\Sigma^{-1}$.)

Worked Example 7.1: Maximum-likelihood estimates from three points.

Data $\boldsymbol{x}_1 = (1,2)$, $\boldsymbol{x}_2 = (3,4)$, $\boldsymbol{x}_3 = (2,0)$.

Mean. $\boldsymbol{\mu} = \frac{1}{3}(6,6) = (2,2)$.

Deviations. $(-1,0)$, $(1,2)$, $(0,-2)$.

Scatter matrix. $S = \left[\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right] + \left[\begin{smallmatrix}1&2\\2&4\end{smallmatrix}\right] + \left[\begin{smallmatrix}0&0\\0&4\end{smallmatrix}\right] = \left[\begin{smallmatrix}2&2\\2&8\end{smallmatrix}\right]$.

Covariance. $\Sigma_{\mathrm{ML}} = S/3 = \begin{bmatrix}2/3 & 2/3\\ 2/3 & 8/3\end{bmatrix}$.

Check the gradient. At $\Sigma = S/N$, (7.1) summed over the data is $-\frac{N}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}S\Sigma^{-1} = -\frac{N}{2}\Sigma^{-1} + \frac{N}{2}\Sigma^{-1} = 0$. ✓

7.3 Conditional and marginal Gaussians

Partition $\boldsymbol{x} = \left[\begin{smallmatrix}\boldsymbol{x}_a\\ \boldsymbol{x}_b\end{smallmatrix}\right]$, $\boldsymbol{\mu} = \left[\begin{smallmatrix}\boldsymbol{\mu}_a\\ \boldsymbol{\mu}_b\end{smallmatrix}\right]$, $\Sigma = \begin{bmatrix}\Sigma_{aa} & \Sigma_{ab}\\ \Sigma_{ba} & \Sigma_{bb}\end{bmatrix}$ and $\Lambda = \Sigma^{-1} = \begin{bmatrix}\Lambda_{aa} & \Lambda_{ab}\\ \Lambda_{ba} & \Lambda_{bb}\end{bmatrix}$.

Theorem 7.2: Conditional Gaussian. $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b) = \mathcal{N}(\boldsymbol{x}_a \mid \boldsymbol{\mu}_{a|b}, \Sigma_{a|b})$ with

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b), \qquad \Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba} = \Lambda_{aa}^{-1}.$$

The marginal is simply $p(\boldsymbol{x}_a) = \mathcal{N}(\boldsymbol{x}_a \mid \boldsymbol{\mu}_a, \Sigma_{aa})$.

Idea: completing the square. As a function of $\boldsymbol{x}_a$, the exponent $-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\top}\Lambda(\boldsymbol{x}-\boldsymbol{\mu})$ is $-\frac{1}{2}\boldsymbol{x}_a^{\top}\Lambda_{aa}\boldsymbol{x}_a + \boldsymbol{x}_a^{\top}\big(\Lambda_{aa}\boldsymbol{\mu}_a - \Lambda_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)\big) + \text{const}$. A Gaussian $\mathcal{N}(\boldsymbol{\mu}', \Sigma')$ has exponent $-\frac{1}{2}\boldsymbol{x}^{\top}\Sigma'^{-1}\boldsymbol{x} + \boldsymbol{x}^{\top}\Sigma'^{-1}\boldsymbol{\mu}' + \text{const}$. Matching the quadratic terms gives $\Sigma_{a|b} = \Lambda_{aa}^{-1}$, and the linear terms give $\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a - \Lambda_{aa}^{-1}\Lambda_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)$. The partitioned inverse (Theorem 3.4) converts these into the Σ-form: Λ_{aa}^{-1} is the Schur complement $\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ and $-\Lambda_{aa}^{-1}\Lambda_{ab} = \Sigma_{ab}\Sigma_{bb}^{-1}$. □

Worked Example 7.2: A conditional Gaussian.

$\boldsymbol{\mu} = \mathbf{0}$, $\Sigma = \left[\begin{smallmatrix}4&2\\2&3\end{smallmatrix}\right]$; observe $x_2 = 3$.

Mean. $\mu_{1|2} = 0 + \frac{2}{3}(3-0) = 2$.

Variance. $\Sigma_{1|2} = 4 - \frac{2\cdot 2}{3} = \frac{8}{3}$.

Check with the precision. $\Lambda = \frac{1}{8}\left[\begin{smallmatrix}3&-2\\-2&4\end{smallmatrix}\right]$; $\Lambda_{11}^{-1} = \frac{8}{3}$ ✓, and $-\Lambda_{11}^{-1}\Lambda_{12}x_2 = -\frac{8}{3}\cdot\left(-\frac{2}{8}\right)\cdot 3 = 2$. ✓

Interpretation. Observing x_2 shrinks the variance of x_1 from 4 to $\frac{8}{3}$ and shifts its mean toward the observation, because the two are positively correlated.

7.4 The KL divergence between Gaussians

Theorem 7.3: KL divergence of two Gaussians. For D-dimensional Gaussians,

$$\mathrm{KL}(\mathcal{N}(\mu_1,\Sigma_1)\,\|\,\mathcal{N}(\mu_2,\Sigma_2)) = \frac{1}{2}\Big[\operatorname{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_2-\mu_1)^{\mathsf{T}}\Sigma_2^{-1}(\mu_2-\mu_1) - D + \ln\frac{|\Sigma_2|}{|\Sigma_1|}\Big].$$

Derivation. $\mathrm{KL} = \mathbb{E}_1[\ln p_1(x) - \ln p_2(x)]$. Under p_1, $\mathbb{E}_1[(x-\mu_1)^{\mathsf{T}}\Sigma_1^{-1}(x-\mu_1)] = \operatorname{tr}(\Sigma_1^{-1}\mathbb{E}[(x-\mu_1)(x-\mu_1)^{\mathsf{T}}]) = \operatorname{tr}(I) = D$ by the trace trick, and similarly $\mathbb{E}_1[(x-\mu_2)^{\mathsf{T}}\Sigma_2^{-1}(x-\mu_2)] = \operatorname{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_1-\mu_2)^{\mathsf{T}}\Sigma_2^{-1}(\mu_1-\mu_2)$. Subtracting the two log-densities and taking expectations gives the result. □

In a variational autoencoder [Bishop Ch. 19] the encoder outputs a diagonal Gaussian $q = \mathcal{N}(\mu, \operatorname{diag}(\sigma^2))$ and the prior is $\mathcal{N}(0, I)$. The KL term then simplifies to

$$\mathrm{KL}(q\,\|\,p) = \frac{1}{2}\sum_{j=1}^{D}\left(\sigma_j^2 + \mu_j^2 - 1 - \ln\sigma_j^2\right).$$

Networks usually output the log-variance $v_j = \ln\sigma_j^2$, which is unconstrained. The gradients are then

$$\frac{\partial \mathrm{KL}}{\partial \mu_j} = \mu_j, \qquad \frac{\partial \mathrm{KL}}{\partial v_j} = \tfrac{1}{2}\left(e^{v_j} - 1\right).$$

Worked Example 7.3: KL term of a VAE.

$\mu = (1, 0)$, $\sigma^2 = (1, 4)$.

Value. $\tfrac{1}{2}\left[(1+1-1-0) + (4+0-1-\ln 4)\right] = \tfrac{1}{2}(4 - \ln 4) \approx 1.3069$.

Gradients. $\partial\mathrm{KL}/\partial\mu = (1, 0)$; with $v = \ln\sigma^2 = (0, \ln 4)$, $\partial\mathrm{KL}/\partial v = \tfrac{1}{2}(1-1,\ 4-1) = (0, 1.5)$.

Reading. The first coordinate has the right variance but the wrong mean; the second has the right mean but too large a variance. Each gradient points only at what is wrong.

7.5 The reparameterisation trick

To differentiate an expectation $\mathbb{E}_{z\sim\mathcal{N}(\mu,\Sigma)}[f(z)]$ with respect to μ and Σ, write $z = \mu + L\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$ and $LL^{\mathsf{T}} = \Sigma$. The randomness no longer depends on the parameters, so derivatives pass inside the expectation:

$$\nabla_\mu \mathbb{E}[f(z)] = \mathbb{E}[\nabla f(z)], \qquad \frac{\partial}{\partial L}\mathbb{E}[f(z)] = \mathbb{E}\left[\nabla f(z)\,\epsilon^{\mathsf{T}}\right].$$

The second formula is the differential method again: $\mathrm{d}f = \nabla f^{\mathsf{T}}\mathrm{d}L\,\epsilon = \operatorname{tr}(\epsilon\nabla f^{\mathsf{T}}\mathrm{d}L)$. Monte Carlo averages of these quantities are the gradient estimates a VAE is trained with [Bishop Ch. 19].

Exercises

Exercise 7.1 $\star$ Compute $\boldsymbol{\mu}_{\mathrm{ML}}$ and Σ_{ML} for the data $(0,0)$, $(2,0)$, $(1,3)$.

Exercise 7.2 $\star$ For $\boldsymbol{\mu} = (1,1)$, $\Sigma = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$, find $p(x_1 \mid x_2 = 3)$.

Exercise 7.3 $\star\star$ Derive (7.1) with the differential method, and show that summing it over data and setting it to zero gives Σ_{ML}.

Exercise 7.4 $\star\star$ Compute $\mathrm{KL}(\mathcal{N}(\boldsymbol{\mu}_1, \Sigma_1) \| \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_2))$ for $\boldsymbol{\mu}_1 = \mathbf{0}$, $\Sigma_1 = \mathrm{diag}(1,1)$, $\boldsymbol{\mu}_2 = (1,1)$, $\Sigma_2 = \mathrm{diag}(2,2)$. Is it symmetric — what is KL the other way round?

Exercise 7.5 $\star\star$ Show that for the VAE KL term the minimum over (μ_j, v_j) is at $\mu_j = 0$, $v_j = 0$, and that the Hessian there is $\mathrm{diag}(1, \tfrac{1}{2})$ per coordinate.

Exercise 7.6 $\star\star\star$ Derive the reparameterisation gradient $\frac{\partial}{\partial L}\mathbb{E}[f(\boldsymbol{\mu} + L\boldsymbol{\epsilon})] = \mathbb{E}[\nabla f(\boldsymbol{z})\boldsymbol{\epsilon}^{\mathsf{T}}]$ and check it for $f(\boldsymbol{z}) = \boldsymbol{a}^{\mathsf{T}}\boldsymbol{z}$ (where the expectation is independent of L).

CHAPTER 8

The Chain Rule and Backpropagation

A neural network is a composition of simple functions: matrix products, element-wise activations, a loss. Its gradient is therefore given by the chain rule. **Backpropagation** is nothing more than the chain rule evaluated in the right order — from the loss backwards — with each step written as a small matrix formula. This chapter derives those formulas with the differential method, so that you can write the backward pass of any layer yourself [Bishop Ch. 6,8].

8.1 The chain rule for Jacobians

For a composition $\boldsymbol{x} \xrightarrow{g} \boldsymbol{y} \xrightarrow{f} \boldsymbol{z}$, the differentials compose: $\mathrm{d}\boldsymbol{y} = J_g \mathrm{d}\boldsymbol{x}$ and $\mathrm{d}\boldsymbol{z} = J_f \mathrm{d}\boldsymbol{y} = J_f J_g \mathrm{d}\boldsymbol{x}$. Hence

$$J_{\boldsymbol{z},\boldsymbol{x}} = J_f \, J_g.$$

For a chain of K layers, $J = J_K J_{K-1} \cdots J_1$.

When the final output is a scalar loss L, its Jacobian is a $1 \times m$ row, ∇L^T. Then

$$\nabla_{\boldsymbol{x}} L^\mathsf{T} = \nabla_{\boldsymbol{z}} L^\mathsf{T} \, J_K \cdots J_1 \quad \Longleftrightarrow \quad \nabla_{\boldsymbol{x}} L = J_1^\mathsf{T} J_2^\mathsf{T} \cdots J_K^\mathsf{T} \nabla_{\boldsymbol{z}} L.$$

There are two ways to evaluate this product:

- **Forward (right to left in $J_K \cdots J_1$):** multiply Jacobian matrices together — matrix–matrix products, expensive.
- **Reverse (start from ∇L):** multiply the gradient row vector by one Jacobian at a time — only **vector–Jacobian products**, cheap.

For a scalar loss the reverse order is always the cheaper one. That is backpropagation.

> **Key idea.** Backprop never forms a Jacobian. Each layer only needs a function that takes the gradient of the loss with respect to its *output* and returns the gradient with respect to its *input* (and its parameters). We write $\bar{Y} = \partial L / \partial Y$ for this incoming gradient; it always has the shape of Y.

8.2 The backward pass of a linear layer

A fully connected layer processes a mini-batch of N inputs stored as the rows of X ($N \times D$):

$$Y = XW + \mathbf{1}\boldsymbol{b}^\mathsf{T}, \qquad W : \; D \times M, \quad \boldsymbol{b} \in \mathbb{R}^M, \quad Y : \; N \times M.$$

Theorem 8.1: Linear-layer gradients. Given $\bar{Y} = \partial L/\partial Y$,

$$\bar{W} = \frac{\partial L}{\partial W} = X^\top \bar{Y}, \qquad \bar{X} = \frac{\partial L}{\partial X} = \bar{Y} W^\top, \qquad \bar{\boldsymbol{b}} = \frac{\partial L}{\partial \boldsymbol{b}} = \bar{Y}^\top \mathbf{1} = \text{column sums of } \bar{Y}.$$

Derivation. By the identification rule, $\mathrm{d}L = \operatorname{tr}(\bar{Y}^\top \mathrm{d}Y)$. With $\mathrm{d}Y = \mathrm{d}X\, W + X\, \mathrm{d}W + \mathbf{1}\, \mathrm{d}\boldsymbol{b}^\top$:

$$\begin{aligned} \mathrm{d}L &= \operatorname{tr}(\bar{Y}^\top \mathrm{d}X\, W) + \operatorname{tr}(\bar{Y}^\top X\, \mathrm{d}W) + \operatorname{tr}(\bar{Y}^\top \mathbf{1}\, \mathrm{d}\boldsymbol{b}^\top) \\ &= \operatorname{tr}\left((\bar{Y} W^\top)^\top \mathrm{d}X\right) + \operatorname{tr}\left((X^\top \bar{Y})^\top \mathrm{d}W\right) + (\bar{Y}^\top \mathbf{1})^\top \mathrm{d}\boldsymbol{b}. \end{aligned}$$

Read off the three gradients. □

Shape check. $X^\top \bar{Y}$ is $(D \times N)(N \times M) = D \times M$, the shape of W. $\bar{Y} W^\top$ is $(N \times M)(M \times D) = N \times D$, the shape of X. Note how $\bar{W}$ *sums over the batch*: $X^\top \bar{Y} = \sum_n \boldsymbol{x}_n \bar{\boldsymbol{y}}_n^\top$, one outer product ("input times error") per example.

8.3 Activations and losses

Theorem 8.2: Backward passes of common layers.

- **Element-wise activation** $H = h(Z)$: $\bar{Z} = \bar{H} \odot h'(Z)$. For ReLU $h'(z) = [z > 0]$; for tanh, $h' = 1 - H^2$; for the sigmoid, $h' = H \odot (1 - H)$.
- **Mean-squared error** $L = \frac{1}{2N} \|Y - T\|_F^2$: $\bar{Y} = \frac{1}{N}(Y - T)$.
- **Softmax with cross-entropy** $L = -\frac{1}{N} \sum_n \sum_k t_{nk} \ln s_{nk}$, $S = \operatorname{softmax}(A)$ row-wise: $\bar{A} = \frac{1}{N}(S - T)$.
- **Sigmoid with binary cross-entropy**: $\bar{\boldsymbol{a}} = \frac{1}{N}(\sigma(\boldsymbol{a}) - \boldsymbol{t})$.
- **Residual connection** $Y = X + F(X)$: $\bar{X} = \bar{Y} + \bar{X}_F$, where $\bar{X}_F$ is what the backward pass of F returns for $\bar{Y}$.

The softmax–cross-entropy result $S - T$ (Exercise 4.4) is the most important line in classification: the gradient with respect to the logits is "prediction minus target". Computing softmax and cross-entropy as one combined layer is also numerically safer than chaining their separate backward passes (Chapter 9).

8.4 A full forward and backward pass by hand

Backpropagation.

1. **Forward:** evaluate the layers in order and *store* every intermediate value (Z, H, ...).
2. **Seed:** $\bar{L} = 1$; compute the gradient of the loss w.r.t. the network output.
3. **Backward:** visit the layers in reverse order; each turns output into input and the gradients of its parameters.
4. **Check** one parameter with a finite difference.

Worked Example 8.1: A two-layer ReLU network.

Input $\boldsymbol{x} = (1,1)$ (one row), $W_1 = \left[\begin{smallmatrix} 1 & 2 \\ 1 & -1 \end{smallmatrix}\right]$, $W_2 = \left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$, no biases, target $t = 3$, loss $L = \frac{1}{2}(y-t)^2$.

Forward. $\boldsymbol{z} = \boldsymbol{x}W_1 = (1+1,\ 2-1) = (2,1)$; $\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z}) = (2,1)$; $y = \boldsymbol{h}W_2 = 2-1 = 1$; $L = \frac{1}{2}(1-3)^2 = 2$.

Loss. $\bar{y} = y - t = -2$.

Layer 2. $\bar{W}_2 = \boldsymbol{h}^{\mathsf{T}}\bar{y} = \left[\begin{smallmatrix} 2 \\ 1 \end{smallmatrix}\right](-2) = \left[\begin{smallmatrix} -4 \\ -2 \end{smallmatrix}\right]$; $\bar{\boldsymbol{h}} = \bar{y}W_2^{\mathsf{T}} = -2(1,-1) = (-2,2)$.

ReLU. Both $z_i > 0$, so $\bar{\boldsymbol{z}} = \bar{\boldsymbol{h}} = (-2,2)$.

Layer 1. $\bar{W}_1 = \boldsymbol{x}^{\mathsf{T}}\bar{\boldsymbol{z}} = \left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right](-2,\ 2) = \left[\begin{smallmatrix} -2 & 2 \\ -2 & 2 \end{smallmatrix}\right]$; $\bar{\boldsymbol{x}} = \bar{\boldsymbol{z}}W_1^{\mathsf{T}} = (-2,2)\left[\begin{smallmatrix} 1 & 1 \\ 2 & -1 \end{smallmatrix}\right] = (2,-4)$.

Check one entry. Increase $(W_2)_1$ by ϵ: $y = 1 + 2\epsilon$, $L = \frac{1}{2}(2\epsilon - 2)^2 \approx 2 - 4\epsilon$. So $\partial L/\partial (W_2)_1 = -4$. ✓

Worked Example 8.2: Softmax and cross-entropy.

Logits $\boldsymbol{a} = (0, \ln 2, 0)$, one-hot target $\boldsymbol{t} = (0,0,1)$.

Forward. $\boldsymbol{s} = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$; $L = -\ln s_3 = \ln 4 \approx 1.386$.

Backward. $\bar{\boldsymbol{a}} = \boldsymbol{s} - \boldsymbol{t} = (\frac{1}{4}, \frac{1}{2}, -\frac{3}{4})$. The gradient components sum to zero: adding a constant to all logits changes nothing.

Via the Jacobian (the long way). $\bar{\boldsymbol{s}} = -\boldsymbol{t}/\boldsymbol{s} = (0,0,-4)$ and $\bar{\boldsymbol{a}} = J^{\mathsf{T}}\bar{\boldsymbol{s}}$ with J from Worked Example 4.2: $-4 \cdot \frac{1}{16}(-1,-2,3) = (\frac{1}{4}, \frac{1}{2}, -\frac{3}{4})$. ✓

8.5 Cost and memory

Each layer's backward pass costs about twice its forward pass (a linear layer computes two matrix products, $X^{\mathsf{T}}\bar{Y}$ and $\bar{Y}W^{\mathsf{T}}$, against one forward). So a full gradient costs roughly *three* forward passes, independent of the number of parameters — the reason gradient-based training of networks with billions of parameters is possible at all. The price is memory: every intermediate value of the forward pass must be stored until the backward pass reaches it.

8.6 Forward mode and reverse mode

The order in which the chain rule is evaluated is a choice. For a function $\mathbb{R}^n \to \mathbb{R}^m$:

- **Forward mode** pushes a direction $\boldsymbol{v}$ forward and computes a *Jacobian–vector product* $J\boldsymbol{v}$ in one pass. The full Jacobian needs n passes.
- **Reverse mode** pulls a vector $\boldsymbol{u}$ backwards and computes a *vector–Jacobian product* $\boldsymbol{u}^{\mathsf{T}}J$ in one pass. The full Jacobian needs m passes.

A loss has $m = 1$, so reverse mode gives the whole gradient in one pass. Chapter 9 implements both modes.

Exercises

Exercise 8.1 ⋆ For a single input row $\boldsymbol{x}$, $\boldsymbol{y} = \sigma(\boldsymbol{x}W + \boldsymbol{b}^{\mathsf{T}})$ and L a function of $\boldsymbol{y}$, write the backward pass: $\bar{\boldsymbol{a}}$, $\bar{W}$, $\bar{\boldsymbol{b}}$ and $\bar{\boldsymbol{x}}$ in terms of $\bar{\boldsymbol{y}}$.

Exercise 8.2 ⋆ Check the shapes in Theorem 8.1 for $N = 32$, $D = 784$, $M = 128$.

Exercise 8.3 ⋆⋆ Derive $\bar{A} = S - T$ for softmax with cross-entropy, for a single example, from Exercise 4.4. Then show that for the batch-mean loss the gradient is $\frac{1}{N}(S - T)$.

Exercise 8.4 ⋆⋆ Redo Worked Example 8.1 with tanh instead of ReLU, up to $\bar{\boldsymbol{z}}$. Use $\tanh 2 \approx 0.964$, $\tanh 1 \approx 0.762$.

Exercise 8.5 ⋆⋆ Derive the backward pass of an element-wise product layer $Y = A \odot B$ (both inputs depend on the parameters).

Exercise 8.6 ⋆⋆ A network has input dimension $n = 10^6$ (an image) and output a single loss. Compare the number of passes needed for the full gradient in forward and in reverse mode. When would forward mode be preferable?

Exercise 8.7 ⋆⋆⋆ For a residual block $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})$, show that $J_{y,x} = I + J_F$ and explain why stacking such blocks avoids vanishing gradients, compared to $\boldsymbol{y} = \boldsymbol{F}(\boldsymbol{x})$.

CHAPTER 9

Computing Derivatives: Finite Differences, Autodiff and Stability

There are three ways to get a derivative into a computer. You can derive it by hand and code the formula — what the previous chapters taught. You can approximate it by **finite differences**, evaluating the function at nearby points. Or you can let the program **differentiate itself** with automatic differentiation, the technology inside PyTorch and JAX [Bishop Ch. 8]. In practice you use all three: hand derivations for understanding and speed, autodiff for convenience, and finite differences to *check* the other two. This chapter also covers the numerical tricks without which softmax, log-likelihoods and log-determinants overflow.

9.1 Finite differences

Definition 9.1: Difference quotients. For a step $h > 0$:

$$\text{forward:}\quad \frac{f(x+h)-f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2),$$
$$\text{central:}\quad \frac{f(x+h)-f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + O(h^4).$$

The error terms follow from Taylor expansion: in the central difference the even-order terms cancel. So the central difference is far more accurate for the same h.

Worked Example 9.1: Forward versus central.

$f(x) = e^x$ at $x = 0$, exact derivative 1, with $h = 0.1$.

Forward. $(e^{0.1} - 1)/0.1 = 1.05171$: error $5.2 \times 10^{-2} \approx h/2$.

Central. $(e^{0.1} - e^{-0.1})/0.2 = 1.001668$: error $1.7 \times 10^{-3} \approx h^2/6$.

Why h cannot be tiny. Each evaluation of f carries a rounding error of about $\varepsilon\,|f|$, with $\varepsilon \approx 2.2 \times 10^{-16}$. Dividing a difference of two such values by h amplifies the rounding error to about $\varepsilon\,|f|\,/h$. The total error is therefore

$$\text{forward:}\ \tfrac{h}{2}\,|f''| + \tfrac{2\varepsilon|f|}{h}, \qquad \text{central:}\ \tfrac{h^2}{6}\,|f'''| + \tfrac{\varepsilon|f|}{h}.$$

Minimising over h gives $h \approx \sqrt{\varepsilon} \approx 10^{-8}$ with error $\approx 10^{-8}$ for forward differences, and $h \approx \varepsilon^{1/3} \approx 6 \times 10^{-6}$ with error $\approx \varepsilon^{2/3} \approx 4 \times 10^{-11}$ for central differences (for f and its derivatives of size about 1). The Lab Manual plots this V-shaped error curve.

The complex-step derivative. If f is built from analytic operations (polynomials, exp, sin, ...), then $f(x+ih) = f(x) + ihf'(x) - \frac{h^2}{2}f''(x) + \dots$, so

$$f'(x) \approx \frac{\operatorname{Im} f(x+ih)}{h}$$

with error $O(h^2)$ and *no subtraction*. One can take $h = 10^{-20}$ and get the derivative to full machine precision. NumPy handles complex numbers natively, so this is a two-line gradient checker for smooth code (it fails for `abs`, comparisons and `max`, which are not analytic).

9.2 Gradient checking

Gradient check.

1. Use double precision and a smooth test point (avoid ReLU kinks at exactly 0).
2. For each entry x_i (or a random sample of entries), compute $\hat{g}_i = \big(f(\boldsymbol{x} + h\boldsymbol{e}_i) - f(\boldsymbol{x} - h\boldsymbol{e}_i)\big)/2h$ with $h \approx 10^{-6}$.
3. Compare with your analytic g_i via the **relative error** $\dfrac{|g_i - \hat{g}_i|}{\max(|g_i|, |\hat{g}_i|, 10^{-12})}$.
4. Interpret: below 10^{-7} correct; 10^{-5}–10^{-3} suspicious; above 10^{-2} almost certainly a bug.

For large inputs, checking every entry costs two function evaluations per entry. A cheaper test checks a **random direction** v: compare $\big(f(\boldsymbol{x} + h\boldsymbol{v}) - f(\boldsymbol{x} - h\boldsymbol{v})\big)/2h$ with $\nabla f^{\mathsf{T}}\boldsymbol{v}$. Two evaluations test all components at once; a wrong component is detected with probability one.

9.3 Forward mode: dual numbers

A **dual number** is $a + b\epsilon$ with the rule $\epsilon^2 = 0$. Arithmetic follows:

$$(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon, \qquad (a + b\epsilon)(c + d\epsilon) = ac + (ad + bc)\epsilon,$$

and for a smooth function $f(a + b\epsilon) = f(a) + bf'(a)\epsilon$ (Taylor's theorem, with all higher powers of ϵ zero). So if we evaluate any program on $x + 1\epsilon$, the ϵ-part of the result is exactly $f'(x)$ — no approximation, no step size. This is forward-mode automatic differentiation: every value carries its derivative along with it.

Worked Example 9.2: Differentiation with dual numbers.

$f(x) = x^3 + 2x$ at $x = 2$.

Evaluate on $2 + \epsilon$. $(2+\epsilon)^2 = 4 + 4\epsilon$; $(2+\epsilon)^3 = (4+4\epsilon)(2+\epsilon) = 8 + 12\epsilon$; $2(2+\epsilon) = 4 + 2\epsilon$.

Sum. $12 + 14\epsilon$: so $f(2) = 12$ and $f'(2) = 14$. Check: $3 \cdot 4 + 2 = 14$. ✓

For $f : \mathbb{R}^n \to \mathbb{R}^m$, seeding the inputs with $\boldsymbol{x} + \boldsymbol{v}\epsilon$ gives $f(\boldsymbol{x}) + (J\boldsymbol{v})\epsilon$: one forward pass computes a Jacobian–vector product.

9.4 Reverse mode: the backward sweep

Reverse mode first evaluates the function, recording every elementary operation on a **tape** (the computational graph). Then it walks the tape backwards, computing for each intermediate v its **adjoint** $\bar{v} = \partial L/\partial v$ by the rule

$$\bar{v} = \sum_{c \in \text{children of } v} \bar{c}\,\frac{\partial c}{\partial v}.$$

The sum matters: a variable used in several places receives a contribution from each use.

Worked Example 9.3: A reverse sweep by hand.
$f(x_1, x_2) = (x_1x_2 + x_1)^2$ at $(1, 2)$.
Forward (tape). $v_1 = x_1x_2 = 2$; $v_2 = v_1 + x_1 = 3$; $f = v_2^2 = 9$.
Backward. $\bar{f} = 1$. $\bar{v}_2 = 2v_2\bar{f} = 6$. $\bar{v}_1 = \bar{v}_2 \cdot 1 = 6$.
Inputs. x_1 is used twice (in v_1 and v_2): $\bar{x}_1 = \bar{v}_1x_2 + \bar{v}_2 \cdot 1 = 12 + 6 = 18$. $\bar{x}_2 = \bar{v}_1x_1 = 6$.
Check. $\partial f/\partial x_1 = 2(x_1x_2 + x_1)(x_2 + 1) = 2 \cdot 3 \cdot 3 = 18$, $\partial f/\partial x_2 = 2(x_1x_2 + x_1)x_1 = 6$. ✓

Backpropagation (Chapter 8) is reverse mode applied at the level of whole layers instead of scalar operations: each "elementary operation" is a matrix product or an activation, and each has the matrix backward rule we derived.

9.5 Numerical stability

Log-sum-exp and softmax. e^{1000} overflows in double precision. But for any constant m,

$$\operatorname{lse}(\boldsymbol{a}) = m + \ln \sum_k e^{a_k - m}, \qquad \operatorname{softmax}(\boldsymbol{a}) = \operatorname{softmax}(\boldsymbol{a} - m\mathbf{1}),$$

and choosing $m = \max_k a_k$ makes every exponent ≤ 0 and the largest term exactly 1: no overflow, and no underflow of the whole sum to 0.

Worked Example 9.4: A softmax that would overflow.
$\boldsymbol{a} = (1000, 1001, 1002)$. Naively, $e^{1000} = \infty$ and the result is $\infty/\infty =$ `nan`.
Shift. $\boldsymbol{a} - 1002 = (-2, -1, 0)$; $e^{(-2,-1,0)} = (0.1353, 0.3679, 1)$, sum 1.5032.
Result. $\operatorname{softmax}(\boldsymbol{a}) = (0.0900, 0.2447, 0.6652)$, and $\operatorname{lse}(\boldsymbol{a}) = 1002 + \ln 1.5032 = 1002.408$.

Log-softmax and cross-entropy. Compute $\ln s_k = a_k - \operatorname{lse}(\boldsymbol{a})$ directly, never $\ln(\operatorname{softmax}(\boldsymbol{a})_k)$, which gives $\ln 0 = -\infty$ when s_k underflows.

Softplus and log-sigmoid. $\operatorname{softplus}(a) = \ln(1 + e^a)$ overflows for large a; the stable form is $\max(a, 0) + \ln(1 + e^{-|a|})$ (use `log1p`). Then $\ln \sigma(a) = -\operatorname{softplus}(-a)$, and the binary cross-entropy with logits is $\operatorname{softplus}(a) - ta$.

Linear algebra. Compute $\ln|\Sigma|$ as $2\sum \ln l_{ii}$ from the Cholesky factor (or `slogdet`), never as `log(det(S))`. Compute $\Sigma^{-1}\boldsymbol{b}$ with a solve, never with `inv`. Compute Mahalanobis distances as $\left\|L^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\|^2$ with a triangular solve.

Exercises

Exercise 9.1 $\star$ Compute forward and central differences of $f(x) = \sin x$ at $x = 0$ with $h = 0.1$ and give their errors. Compare with the predicted $\frac{h}{2}|f''|$ and $\frac{h^2}{6}|f'''|$.

Exercise 9.2 $\star$ Use dual numbers to compute $f'(1)$ for $f(x) = (x^2+1)(3x-1)$.

Exercise 9.3 $\star\star$ Do a reverse sweep for $f(x,y) = (x+y)^2 y$ at $(1,2)$, writing the tape and adjoints.

Exercise 9.4 $\star\star$ Derive the optimal step h for the central difference by minimising $E(h) = \frac{h^2}{6}M + \frac{\varepsilon F}{h}$, where $M = |f'''|$ and $F = |f|$. Evaluate it for $M = F = 1$.

Exercise 9.5 $\star\star$ Show that $\ln(1+e^a) = \max(a,0) + \ln(1+e^{-|a|})$ for all a. Evaluate softplus(1000) and softplus(-1000) with both formulas in double precision.

Exercise 9.6 $\star\star$ Show that the stable softmax is mathematically identical to the naive one, and that the gradient of $\mathrm{lse}(\boldsymbol{a}) = m + \ln\sum e^{a_k - m}$ with $m = \max a_k$ treated as a constant is still $\mathrm{softmax}(\boldsymbol{a})$. Why is it legitimate to treat m as a constant even though it depends on $\boldsymbol{a}$?

Exercise 9.7 $\star\star\star$ Explain why the complex-step method $\mathrm{Im}\, f(x+ih)/h$ has no cancellation error, and why it gives a wrong answer for $f(x) = |x|$ implemented as `np.abs` (or for code containing `if x > 0`).

CHAPTER 10

Constrained Optimisation, Eigenproblems and PCA

Some quantities in machine learning must satisfy constraints: projection directions have unit length, mixture weights sum to one, a set of basis vectors is orthonormal. The method of **Lagrange multipliers** [Bishop App. C] turns a constrained problem into an unconstrained one that we can attack with gradients. Applied to the variance of projected data it produces an eigenvalue problem — principal component analysis [Bishop Ch. 16] — and connects this course back to the SVD.

10.1 Lagrange multipliers

Theorem 10.1: Lagrange multipliers. If $\boldsymbol{x}^\star$ maximises or minimises $f(\boldsymbol{x})$ subject to $g(\boldsymbol{x}) = 0$, and $\nabla g(\boldsymbol{x}^\star) \neq \mathbf{0}$, then there is a number λ with

$$\nabla f(\boldsymbol{x}^\star) + \lambda \nabla g(\boldsymbol{x}^\star) = \mathbf{0}.$$

Equivalently, $\boldsymbol{x}^\star$ is a stationary point of the **Lagrangian** $\mathcal{L}(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) + \lambda g(\boldsymbol{x})$ with respect to both $\boldsymbol{x}$ and λ.

Why it is true. Moving along the constraint surface means moving in directions $\boldsymbol{v}$ with $\nabla g^{\mathsf{T}} \boldsymbol{v} = 0$. At a constrained optimum f cannot change to first order in any such direction, so $\nabla f^{\mathsf{T}} \boldsymbol{v} = 0$ for every $\boldsymbol{v} \perp \nabla g$. Hence ∇f is parallel to ∇g. □

With several constraints $g_k(\boldsymbol{x}) = 0$ there is one multiplier each: $\nabla f + \sum_k \lambda_k \nabla g_k = \mathbf{0}$. Inequality constraints $g(\boldsymbol{x}) \geq 0$ lead to the KKT conditions: either the constraint is inactive and $\lambda = 0$, or it is active and behaves like an equality ($\lambda g(\boldsymbol{x}) = 0$ in both cases).

Worked Example 10.1: A first Lagrange problem.

Maximise $f = x + y$ subject to $x^2 + y^2 = 2$.

Lagrangian. $\mathcal{L} = x + y + \lambda(2 - x^2 - y^2)$.

Stationarity. $\partial \mathcal{L}/\partial x = 1 - 2\lambda x = 0$ and $\partial \mathcal{L}/\partial y = 1 - 2\lambda y = 0$, so $x = y = \frac{1}{2\lambda}$.

Constraint. $2x^2 = 2$ gives $x = y = \pm 1$, $\lambda = \pm \frac{1}{2}$.

Answer. The maximum is at $(1, 1)$ with $f = 2$; $(-1, -1)$ is the minimum. At both points $\nabla f = (1, 1)$ is parallel to $\nabla g = (2x, 2y)$.

10.2 Mixture weights: a constraint on a sum

In a mixture model the maximum-likelihood mixing coefficients maximise $\sum_k n_k \ln \pi_k$ subject to $\sum_k \pi_k = 1$, where n_k is the (effective) number of points in component k [Bishop Ch. 15]. The Lagrangian $\sum_k n_k \ln \pi_k + \lambda(\sum_k \pi_k - 1)$ gives $n_k/\pi_k + \lambda = 0$, so $\pi_k \propto n_k$; the constraint fixes $\lambda = -N$ and $\pi_k = n_k/N$: each weight is the fraction of the data its component explains.

10.3 The Rayleigh quotient and the top eigenvector

Theorem 10.2: Maximum-variance direction. For a symmetric matrix S, the maximum of $\boldsymbol{u}^\top S\boldsymbol{u}$ subject to $\boldsymbol{u}^\top\boldsymbol{u} = 1$ is the largest eigenvalue λ_1, attained at the corresponding unit eigenvector.

Derivation. $\mathcal{L} = \boldsymbol{u}^\top S\boldsymbol{u} + \lambda(1 - \boldsymbol{u}^\top\boldsymbol{u})$ has gradient $2S\boldsymbol{u} - 2\lambda\boldsymbol{u}$. Setting it to zero: $S\boldsymbol{u} = \lambda\boldsymbol{u}$, so every stationary point is an eigenvector, and there $\boldsymbol{u}^\top S\boldsymbol{u} = \lambda\boldsymbol{u}^\top\boldsymbol{u} = \lambda$. The maximum is the largest eigenvalue. □

The same conclusion appears without a constraint in the gradient of the Rayleigh quotient $R(\boldsymbol{x}) = \boldsymbol{x}^\top S\boldsymbol{x}/\boldsymbol{x}^\top\boldsymbol{x}$:

$$\nabla R(\boldsymbol{x}) = \frac{2}{\boldsymbol{x}^\top\boldsymbol{x}}\big(S\boldsymbol{x} - R(\boldsymbol{x})\boldsymbol{x}\big),$$

which vanishes exactly when $S\boldsymbol{x} = R(\boldsymbol{x})\boldsymbol{x}$.

10.4 Principal component analysis

Let $\boldsymbol{x}_1, \dots, \boldsymbol{x}_N \in \mathbb{R}^D$ have mean $\bar{\boldsymbol{x}}$ and covariance $S = \frac{1}{N}\sum_n (\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^\top$. Projecting the data onto a unit vector $\boldsymbol{u}$ gives scalars $\boldsymbol{u}^\top\boldsymbol{x}_n$ with variance $\boldsymbol{u}^\top S\boldsymbol{u}$ (the trace trick again). PCA chooses $\boldsymbol{u}$ to maximise this variance, which by Theorem 10.2 is the top eigenvector of S.

Theorem 10.3: PCA with M components. Maximising $\operatorname{tr}(U^\top SU)$ over $D \times M$ matrices U with $U^\top U = I_M$ is achieved by the eigenvectors of S for the M largest eigenvalues; the maximum is $\lambda_1 + \cdots + \lambda_M$. Equivalently, the average squared reconstruction error of projecting onto those directions, $J = \frac{1}{N}\sum_n \left\|(\boldsymbol{x}_n - \bar{\boldsymbol{x}}) - UU^\top(\boldsymbol{x}_n - \bar{\boldsymbol{x}})\right\|^2$, is minimised, with minimum $J = \sum_{i>M} \lambda_i$.

Sketch. With a symmetric matrix of multipliers Λ for the constraint $U^\top U = I$, $\mathcal{L} = \operatorname{tr}(U^\top SU) + \operatorname{tr}\left(\Lambda(I - U^\top U)\right)$. By Theorem 6.3, $\partial\mathcal{L}/\partial U = 2SU - 2U\Lambda = 0$, i.e. $SU = U\Lambda$: the columns of U span an invariant subspace of S, and rotating within it we can take them to be eigenvectors. The objective is then the sum of their eigenvalues. For the error: $J = \operatorname{tr}(S) - \operatorname{tr}(U^\top SU)$ by expanding the square and using $U^\top U = I$. □

Worked Example 10.2: PCA by hand.
Data with covariance $S = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$.
Eigenvalues. 3 and 1, with $\boldsymbol{u}_1 = \frac{1}{\sqrt{2}}(1,1)$ and $\boldsymbol{u}_2 = \frac{1}{\sqrt{2}}(1,-1)$.
First component. $\boldsymbol{u}_1$: variance along it $\boldsymbol{u}_1^{\mathsf{T}} S \boldsymbol{u}_1 = \frac{1}{2}(2+1+1+2) = 3$.
Explained variance. $\frac{3}{3+1} = 75\%$; projecting onto $\boldsymbol{u}_1$ leaves error $J = \lambda_2 = 1$.
Check the Lagrange condition. $S\boldsymbol{u}_1 = \frac{1}{\sqrt{2}}(3,3) = 3\boldsymbol{u}_1$. ✓

Connection to the SVD. If the centred data are the rows of X, then $S = \frac{1}{N} X^{\mathsf{T}} X$, whose eigenvectors are the right singular vectors of X with $\lambda_i = \sigma_i^2 / N$. So PCA is the truncated SVD of the data matrix, and the Eckart–Young theorem of the SVD course is the statement that PCA gives the best rank-M reconstruction. Bishop's probabilistic PCA [Bishop Ch. 16] arrives at the same subspace from a maximum-likelihood argument whose derivatives use the identities of Chapters 3 and 6.

10.5 The minimum-norm solution by Lagrange

The pseudo-inverse of the SVD course also has a Lagrange derivation. Minimise $\frac{1}{2}\|\boldsymbol{x}\|^2$ subject to $A\boldsymbol{x} = \boldsymbol{b}$ (full row rank) with a vector of multipliers: $\mathcal{L} = \frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{x} + \boldsymbol{\lambda}^{\mathsf{T}}(\boldsymbol{b} - A\boldsymbol{x})$. Then $\nabla_{\boldsymbol{x}}\mathcal{L} = \boldsymbol{x} - A^{\mathsf{T}}\boldsymbol{\lambda} = \boldsymbol{0}$, so $\boldsymbol{x} = A^{\mathsf{T}}\boldsymbol{\lambda}$; substituting into $A\boldsymbol{x} = \boldsymbol{b}$ gives $\boldsymbol{\lambda} = (AA^{\mathsf{T}})^{-1}\boldsymbol{b}$ and $\boldsymbol{x} = A^{\mathsf{T}}(AA^{\mathsf{T}})^{-1}\boldsymbol{b} = A^{+}\boldsymbol{b}$.

Exercises

Exercise 10.1 ⋆ Minimise $x^2 + y^2$ subject to $x + 2y = 5$ with a Lagrange multiplier. Compare with the minimum-norm formula $A^{\mathsf{T}}(AA^{\mathsf{T}})^{-1}\boldsymbol{b}$.

Exercise 10.2 ⋆ A mixture has effective counts $n = (30, 50, 20)$. Find the maximum-likelihood mixing coefficients and the value of λ.

Exercise 10.3 ⋆⋆ For $S = \left[\begin{smallmatrix} 4 & 2 \\ 2 & 1 \end{smallmatrix}\right]$, find the first principal direction, the variance it explains, and the reconstruction error of a one-component PCA.

Exercise 10.4 ⋆⋆ Derive the gradient of the Rayleigh quotient $R(\boldsymbol{x}) = \boldsymbol{x}^{\mathsf{T}} S \boldsymbol{x} / \boldsymbol{x}^{\mathsf{T}}\boldsymbol{x}$ (S symmetric) and show that $\nabla R(\boldsymbol{x}) \perp \boldsymbol{x}$. Why is that expected?

Exercise 10.5 ⋆⋆⋆ Show that for orthonormal U ($U^{\mathsf{T}}U = I_M$) and centred data with covariance S, the reconstruction error is $J = \operatorname{tr}(S) - \operatorname{tr}(U^{\mathsf{T}} S U)$. Conclude that maximising the projected variance and minimising the reconstruction error are the same problem.

CHAPTER 11

Jacobian Determinants and Normalizing Flows

A normalizing flow [Bishop Ch. 18] builds a flexible probability density by pushing a simple one (a standard Gaussian) through an invertible neural network. The density of the result involves the determinant of the network's Jacobian, and training it by maximum likelihood needs the derivative of a log-determinant. Everything in this chapter is therefore an application of Chapters 2, 3 and 6: the change-of-variables formula, triangular determinants, the matrix determinant lemma, $\mathrm{d}\ln|X| = \operatorname{tr}(X^{-1}\mathrm{d}X)$, and the trace.

11.1 Change of variables

In one dimension, if z has density p_z and $x = g(z)$ with g invertible and $z = f(x) = g^{-1}(x)$, probability conservation $p_x(x)\,|\mathrm{d}x| = p_z(z)\,|\mathrm{d}z|$ gives $p_x(x) = p_z(f(x))\,|f'(x)|$. The factor $|f'(x)|$ accounts for the stretching of the axis.

Theorem 11.1: Change of variables. Let $\boldsymbol{z} \sim p_z$ in $\mathbb{R}^D$ and $\boldsymbol{x} = \boldsymbol{g}(\boldsymbol{z})$ with $\boldsymbol{g}$ invertible and differentiable, $\boldsymbol{z} = \boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{g}^{-1}(\boldsymbol{x})$. Then

$$p_x(\boldsymbol{x}) = p_z(\boldsymbol{f}(\boldsymbol{x}))\,\left|\det J_f(\boldsymbol{x})\right|, \qquad \ln p_x(\boldsymbol{x}) = \ln p_z(\boldsymbol{f}(\boldsymbol{x})) + \ln\left|\det J_f(\boldsymbol{x})\right|,$$

where $J_f = \partial \boldsymbol{f}/\partial \boldsymbol{x}$. Since $J_f(\boldsymbol{x}) = J_g(\boldsymbol{z})^{-1}$, equivalently $\ln p_x(\boldsymbol{x}) = \ln p_z(\boldsymbol{z}) - \ln\left|\det J_g(\boldsymbol{z})\right|$.

$|\det J|$ is the local volume-scaling factor of the map: a small box of volume $\mathrm{d}V$ around $\boldsymbol{x}$ is mapped to a parallelepiped of volume $\left|\det J_f\right|\mathrm{d}V$ around $\boldsymbol{z}$, and the probability inside must be the same.

11.2 Linear flows

The simplest flow is affine: $\boldsymbol{x} = A\boldsymbol{z} + \boldsymbol{b}$ with $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, I)$. Then $\boldsymbol{z} = A^{-1}(\boldsymbol{x} - \boldsymbol{b})$, $J_f = A^{-1}$ and

$$\ln p_x(\boldsymbol{x}) = -\frac{D}{2}\ln 2\pi - \frac{1}{2}\left\|A^{-1}(\boldsymbol{x} - \boldsymbol{b})\right\|^2 - \ln\left|\det A\right|,$$

which is exactly $\ln \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{b}, AA^{\mathsf{T}})$.

Worked Example 11.1: The density of a linear flow.

$A = \left[\begin{smallmatrix} 2 & 0 \\ 1 & 1 \end{smallmatrix}\right]$, $\boldsymbol{b} = \mathbf{0}$, evaluate at $\boldsymbol{x} = (2, 3)$.

Invert. $A^{-1} = \left[\begin{smallmatrix} 1/2 & 0 \\ -1/2 & 1 \end{smallmatrix}\right]$, so $\boldsymbol{z} = A^{-1}\boldsymbol{x} = (1,\ -1+3) = (1, 2)$.

Base density. $\ln p_z(\boldsymbol{z}) = -\ln 2\pi - \frac{1}{2}(1+4) = -\ln 2\pi - 2.5$.

Volume term. $\det A = 2$ (triangular), so subtract $\ln 2$: $\ln p_x(\boldsymbol{x}) = -\ln 2\pi - 2.5 - \ln 2 \approx -5.031$.

Check as a Gaussian. $\Sigma = AA^{\mathsf{T}} = \left[\begin{smallmatrix} 4 & 2 \\ 2 & 2 \end{smallmatrix}\right]$, $|\Sigma| = 4$, $\Sigma^{-1} = \frac{1}{4}\left[\begin{smallmatrix} 2 & -2 \\ -2 & 4 \end{smallmatrix}\right]$, $\boldsymbol{x}^{\mathsf{T}}\Sigma^{-1}\boldsymbol{x} = \frac{1}{4}(8 - 24 + 36) = 5$. So $\ln \mathcal{N} = -\ln 2\pi - \frac{1}{2}\ln 4 - \frac{5}{2}$, the same. ✓

Training a linear flow by maximum likelihood needs $\partial/\partial A$ of the negative log-likelihood. With $\boldsymbol{z} = A^{-1}(\boldsymbol{x} - \boldsymbol{b})$, the differential method gives (Exercise 11.5)

$$\frac{\partial}{\partial A}\left[\tfrac{1}{2}\,\|\boldsymbol{z}\|^2 + \ln|\det A|\right] = A^{-\mathsf{T}}\left(I - \boldsymbol{z}\boldsymbol{z}^{\mathsf{T}}\right).$$

Averaged over data, the gradient vanishes when the average of $\boldsymbol{z}\boldsymbol{z}^{\mathsf{T}}$ is I: the flow is trained exactly when it *whitens* the data.

11.3 Coupling layers: triangular Jacobians

A general $D \times D$ determinant costs $O(D^3)$ — too expensive inside a network. Flows are therefore designed so that their Jacobians are **triangular**, whose determinant is the product of the diagonal. The affine **coupling layer** (RealNVP) splits $\boldsymbol{x} = (\boldsymbol{x}_A, \boldsymbol{x}_B)$ and sets

$$\boldsymbol{z}_A = \boldsymbol{x}_A, \qquad \boldsymbol{z}_B = \exp\big(\boldsymbol{s}(\boldsymbol{x}_A)\big) \odot \boldsymbol{x}_B + \boldsymbol{t}(\boldsymbol{x}_A),$$

where $\boldsymbol{s}$ and $\boldsymbol{t}$ are arbitrary neural networks. Its Jacobian is block lower-triangular,

$$J = \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \begin{bmatrix} I & 0 \\ \partial \boldsymbol{z}_B/\partial \boldsymbol{x}_A & \operatorname{diag}(\exp \boldsymbol{s}(\boldsymbol{x}_A)) \end{bmatrix}, \qquad \ln|\det J| = \sum_j s_j(\boldsymbol{x}_A),$$

and the complicated block $\partial \boldsymbol{z}_B/\partial \boldsymbol{x}_A$ never needs to be computed. The inverse is just as cheap: $\boldsymbol{x}_A = \boldsymbol{z}_A$, $\boldsymbol{x}_B = \big(\boldsymbol{z}_B - \boldsymbol{t}(\boldsymbol{z}_A)\big) \odot \exp(-\boldsymbol{s}(\boldsymbol{z}_A))$. Stacking coupling layers with the roles of A and B alternating gives an expressive flow with a cheap log-determinant [Bishop Ch. 18].

Worked Example 11.2: A coupling layer in two dimensions.

$z_1 = x_1$, $z_2 = x_2 e^{s(x_1)} + t(x_1)$ with $s(x_1) = x_1/2$, $t(x_1) = x_1^2$; evaluate at $\boldsymbol{x} = (2, 1)$.

Forward. $s = 1$, $t = 4$: $\boldsymbol{z} = (2,\ e + 4) \approx (2, 6.718)$.

Jacobian. $\partial z_2/\partial x_1 = x_2 e^s s'(x_1) + t'(x_1) = \frac{e}{2} + 4$, $\partial z_2/\partial x_2 = e^s = e$. So $J = \left[\begin{smallmatrix} 1 & 0 \\ e/2+4 & e \end{smallmatrix}\right]$.

Log-determinant. $\ln|\det J| = \ln e = 1 = s(x_1)$. The off-diagonal entry played no role.

Inverse. $x_2 = (z_2 - t(z_1))e^{-s(z_1)} = (e + 4 - 4)/e = 1$. ✓

11.4 Planar flows and the determinant lemma

A planar flow updates $\boldsymbol{z}' = \boldsymbol{z} + \boldsymbol{u}\, h(\boldsymbol{w}^\top \boldsymbol{z} + b)$ with vectors $\boldsymbol{u}, \boldsymbol{w}$ and a scalar non-linearity h. Its Jacobian is $I + \boldsymbol{u}\, h'(\cdot)\, \boldsymbol{w}^\top$, a rank-one update of the identity, so by the matrix determinant lemma (Theorem 3.5)

$$\det\left(I + h'\, \boldsymbol{u}\boldsymbol{w}^\top\right) = 1 + h'\, \boldsymbol{w}^\top \boldsymbol{u},$$

an $O(D)$ computation instead of $O(D^3)$.

11.5 Continuous flows and the trace of the Jacobian

A continuous flow [Bishop Ch. 18] moves points along a differential equation $\frac{\mathrm{d}\boldsymbol{z}}{\mathrm{d}t} = \boldsymbol{f}(\boldsymbol{z}, t)$. Over a short time δt the map is $\boldsymbol{z} \mapsto \boldsymbol{z} + \boldsymbol{f}\delta t$, with Jacobian $I + J_f \delta t$, and

$$\ln\left|\det(I + J_f \delta t)\right| = \operatorname{tr}(J_f)\delta t + O(\delta t^2)$$

(this is $\mathrm{d}\ln|X| = \operatorname{tr}(X^{-1}\mathrm{d}X)$ at $X = I$). Accumulating these steps gives the **instantaneous change of variables**

$$\frac{\mathrm{d}}{\mathrm{d}t} \ln p(\boldsymbol{z}(t)) = -\operatorname{tr}\left(\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{z}}\right).$$

The determinant has become a trace — cheaper, but computing $\operatorname{tr}(J_f)$ exactly still needs all D diagonal entries of the Jacobian, i.e. D backward passes.

Theorem 11.2: Hutchinson's trace estimator. If $\boldsymbol{\epsilon}$ is a random vector with $\mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top] = I$ (for example standard Gaussian, or Rademacher: independent ± 1 entries), then for any square matrix A

$$\operatorname{tr}(A) = \mathbb{E}\left[\boldsymbol{\epsilon}^\top A \boldsymbol{\epsilon}\right].$$

Proof. $\mathbb{E}[\boldsymbol{\epsilon}^\top A \boldsymbol{\epsilon}] = \mathbb{E}[\operatorname{tr}(A\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top)] = \operatorname{tr}(A\, \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top]) = \operatorname{tr}(A)$ — the trace trick once more. □

For a Jacobian, $\boldsymbol{\epsilon}^\top J_f$ is one vector–Jacobian product (one reverse-mode pass), so each sample of $\boldsymbol{\epsilon}^\top J_f \boldsymbol{\epsilon}$ costs one backward pass regardless of D. The variance with Rademacher vectors is $2\sum_{i \neq j} a_{ij}^{(s)2}$, where $A^{(s)} = \frac{1}{2}(A + A^\top)$; with Gaussian vectors it is $2\left\|A^{(s)}\right\|_F^2$, which is larger.

Worked Example 11.3: Hutchinson by enumeration.

$A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$, $\operatorname{tr} A = 5$, Rademacher $\boldsymbol{\epsilon} \in \{\pm 1\}^2$.

All four cases. $\boldsymbol{\epsilon}^\top A \boldsymbol{\epsilon} = a_{11} + a_{22} + \epsilon_1\epsilon_2(a_{12} + a_{21}) = 5 \pm 5$: the values are 10 for $\boldsymbol{\epsilon} = \pm(1,1)$ and 0 for $\boldsymbol{\epsilon} = \pm(1,-1)$.

Mean. $\frac{1}{4}(10 + 0 + 0 + 10) = 5 = \operatorname{tr} A$. ✓

Variance. 25, matching $2\sum_{i \neq j}(a_{ij}^{(s)})^2 = 2 \cdot 2 \cdot 2.5^2 = 25$. Averaging k samples reduces it to $25/k$.

Exercises

Exercise 11.1 $\star$ Let $z \sim \mathcal{N}(0,1)$ and $x = e^z$. Use the change-of-variables formula to find $p_x(x)$ and evaluate $p_x(1)$.

Exercise 11.2 $\star$ For the linear flow $\boldsymbol{x} = A\boldsymbol{z}$ with $A = \left[\begin{smallmatrix} 1 & 0 \\ 2 & 1 \end{smallmatrix}\right]$, compute $\ln p_x(\boldsymbol{x})$ at $\boldsymbol{x} = (1,3)$. Why is there no volume term?

Exercise 11.3 $\star\star$ For the coupling layer of Worked Example 11.2, compute $\boldsymbol{z}$ and $\ln|\det J|$ at $\boldsymbol{x} = (0,5)$ and at $\boldsymbol{x} = (-2,1)$, and invert both.

Exercise 11.4 $\star\star$ (a) Compute the variance of the Hutchinson estimator with Rademacher vectors for a diagonal matrix. (b) For $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$ compute the variance with Gaussian vectors, and compare.

Exercise 11.5 $\star\star\star$ For a linear flow with base $\mathcal{N}(\mathbf{0}, I)$, the negative log-likelihood of one point (up to a constant) is $\ell(A) = \tfrac{1}{2}\|\boldsymbol{z}\|^2 + \ln|\det A|$ with $\boldsymbol{z} = A^{-1}(\boldsymbol{x} - \boldsymbol{b})$. Derive $\partial \ell / \partial A = A^{-\mathsf{T}}(I - \boldsymbol{z}\boldsymbol{z}^{\mathsf{T}})$, and show that at a stationary point of the average loss, AA^{T} equals the sample covariance.

CHAPTER 12

Attention and Normalisation Layers

Transformers [Bishop Ch. 12] are built from two layer types that are not simple "matrix product plus activation": self-attention, which multiplies data by data and applies a softmax to every row of a matrix, and layer normalisation, which divides each input by a statistic of itself. Their backward passes are still derived with the differential method and the softmax Jacobian — this chapter does it, ending the course where Bishop's book reaches its most modern architectures.

12.1 The row-wise softmax

Attention applies softmax to every row of a matrix S separately: $A = \mathrm{softmax}(S)$ with $A_{ij} = e^{S_{ij}} / \sum_k e^{S_{ik}}$.

Theorem 12.1: Backward pass of a softmax. For one row, $\boldsymbol{s} = \mathrm{softmax}(\boldsymbol{a})$ with incoming gradient $\bar{\boldsymbol{s}}$:

$$\bar{\boldsymbol{a}} = J^{\mathsf{T}}\bar{\boldsymbol{s}} = \big(\mathrm{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^{\mathsf{T}}\big)\bar{\boldsymbol{s}} = \boldsymbol{s} \odot \big(\bar{\boldsymbol{s}} - (\boldsymbol{s}^{\mathsf{T}}\bar{\boldsymbol{s}})\mathbf{1}\big).$$

For a row-wise softmax of a matrix, applied to every row:

$$\bar{S} = A \odot \big(\bar{A} - \mathrm{rowsum}(\bar{A} \odot A)\,\mathbf{1}^{\mathsf{T}}\big).$$

The formula never forms the $n \times n$ Jacobian of each row: it costs $O(n)$ per row instead of $O(n^2)$. The subtracted term $\boldsymbol{s}^{\mathsf{T}}\bar{\boldsymbol{s}}$ is the "average" incoming gradient, weighted by the probabilities; only deviations from it matter, because adding a constant to all logits does nothing.

12.2 Self-attention

For N tokens with embeddings in the rows of X ($N \times D$), one attention head computes [Bishop Ch. 12]

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v, \quad S = \frac{QK^{\mathsf{T}}}{\sqrt{d_k}}, \quad A = \mathrm{softmax}(S)\ (\text{rows}), \quad Y = AV.$$

Q and K are $N \times d_k$, V is $N \times d_v$, the attention matrix A is $N \times N$, and Y is $N \times d_v$.

Theorem 12.2: Backward pass of self-attention. Given $\bar{Y} = \partial L / \partial Y$:

$$\bar{V} = A^{\mathsf{T}}\bar{Y}, \qquad \bar{A} = \bar{Y}V^{\mathsf{T}}, \qquad \bar{S} = A \odot \big(\bar{A} - \mathrm{rowsum}(\bar{A} \odot A)\mathbf{1}^{\mathsf{T}}\big),$$

$$\bar{Q} = \frac{\bar{S}K}{\sqrt{d_k}}, \qquad \bar{K} = \frac{\bar{S}^{\mathsf{T}}Q}{\sqrt{d_k}}, \qquad \bar{W}_\bullet = X^{\mathsf{T}}\bar{\bullet}\ \ (\bullet = Q, K, V),$$

$$\bar{X} = \bar{Q}W_q^{\mathsf{T}} + \bar{K}W_k^{\mathsf{T}} + \bar{V}W_v^{\mathsf{T}}.$$

Derivation. $Y = AV$ is a matrix product, so Theorem 8.1 gives $\bar{A} = \bar{Y}V^\mathsf{T}$ and $\bar{V} = A^\mathsf{T}\bar{Y}$. The softmax step is Theorem 12.1. For the scores, $\mathrm{d}S = (\mathrm{d}Q\,K^\mathsf{T} + Q\,\mathrm{d}K^\mathsf{T})/\sqrt{d_k}$, so

$$\operatorname{tr}(\bar{S}^\mathsf{T}\mathrm{d}S) = \tfrac{1}{\sqrt{d_k}}\big[\operatorname{tr}(K^\mathsf{T}\bar{S}^\mathsf{T}\mathrm{d}Q) + \operatorname{tr}(Q^\mathsf{T}\bar{S}\,\mathrm{d}K)\big],$$

using the cyclic trace and a transpose in the second term; reading off gives $\bar{Q}$ and $\bar{K}$. Finally X feeds three linear maps, so its gradient is the sum of three contributions. □

Key idea. Every line of the attention backward pass is one of two patterns you already know: the backward pass of a matrix product ($\bar{A} = \bar{Y}V^\mathsf{T}$, $\bar{V} = A^\mathsf{T}\bar{Y}$), or the backward pass of a softmax. A variable used in several places (X here) sums its gradients.

Worked Example 12.1: Attention with two tokens, by hand.

$d_k = d_v = 1$, $W_q = W_k = W_v = [1]$, $X = \left[\begin{smallmatrix}0\\1\end{smallmatrix}\right]$, loss $L = y_1$ (so $\bar{Y} = \left[\begin{smallmatrix}1\\0\end{smallmatrix}\right]$).

Forward. $Q = K = V = X$; $S = XX^\mathsf{T} = \left[\begin{smallmatrix}0&0\\0&1\end{smallmatrix}\right]$; row 1 of A is $\mathrm{softmax}(0,0) = (\frac{1}{2},\frac{1}{2})$, row 2 is $\mathrm{softmax}(0,1) \approx (0.269, 0.731)$; $y_1 = \frac{1}{2}\cdot 0 + \frac{1}{2}\cdot 1 = \frac{1}{2}$.

Values and attention. $\bar{V} = A^\mathsf{T}\bar{Y} = (\frac{1}{2},\frac{1}{2})^\mathsf{T}$; $\bar{A} = \bar{Y}V^\mathsf{T} = \left[\begin{smallmatrix}0&1\\0&0\end{smallmatrix}\right]$.

Softmax. Row 1: $(\frac{1}{2},\frac{1}{2}) \odot \big((0,1) - \frac{1}{2}\big) = (-\frac{1}{4},\frac{1}{4})$; row 2 is zero. So $\bar{S} = \left[\begin{smallmatrix}-1/4&1/4\\0&0\end{smallmatrix}\right]$.

Queries and keys. $\bar{Q} = \bar{S}K = (\frac{1}{4},0)^\mathsf{T}$; $\bar{K} = \bar{S}^\mathsf{T}Q = (-\frac{1}{4}\cdot 0 + 0\cdot 1,\ \frac{1}{4}\cdot 0 + 0\cdot 1)^\mathsf{T} = (0,0)^\mathsf{T}$.

Input. $\bar{X} = \bar{Q} + \bar{K} + \bar{V} = (\frac{3}{4},\frac{1}{2})^\mathsf{T}$.

Check $\bar{x}_1$. Replace x_1 by ϵ: row 1 of S is (ϵ^2, ϵ), its softmax $\approx (\frac{1}{2} - \frac{\epsilon}{4}, \frac{1}{2} + \frac{\epsilon}{4})$, and $y_1 \approx (\frac{1}{2} - \frac{\epsilon}{4})\epsilon + (\frac{1}{2} + \frac{\epsilon}{4})\cdot 1 \approx \frac{1}{2} + \frac{3}{4}\epsilon$. ✓

Why divide by $\sqrt{d_k}$? If the entries of $\boldsymbol{q}$ and $\boldsymbol{k}$ are independent with variance 1, then $\boldsymbol{q}^\mathsf{T}\boldsymbol{k}$ has variance d_k. For large d_k the scores in a row differ by many units, the softmax becomes nearly one-hot, and its Jacobian $\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\mathsf{T}$ becomes nearly zero — gradients stop flowing. Dividing by $\sqrt{d_k}$ restores unit variance (Exercise 12.6).

Masking. Causal attention sets $S_{ij} = -\infty$ for $j > i$. Then $A_{ij} = 0$ and, by Theorem 12.1, $\bar{S}_{ij} = 0$: masked positions receive no gradient, as they should.

12.3 Layer normalisation

Layer normalisation [Bishop Ch. 7,12] standardises each row $\boldsymbol{x} \in \mathbb{R}^D$ of its input and then rescales:

$$\mu = \tfrac{1}{D}\mathbf{1}^\mathsf{T}\boldsymbol{x}, \quad v = \tfrac{1}{D}\,\|\boldsymbol{x} - \mu\mathbf{1}\|^2, \quad \tilde{\sigma} = \sqrt{v + \varepsilon}, \quad \hat{\boldsymbol{x}} = \frac{\boldsymbol{x} - \mu\mathbf{1}}{\tilde{\sigma}}, \quad \boldsymbol{y} = \boldsymbol{\gamma} \odot \hat{\boldsymbol{x}} + \boldsymbol{\beta}.$$

Theorem 12.3: Layer-norm Jacobian and backward pass. The Jacobian of the standardisation $\boldsymbol{x} \mapsto \hat{\boldsymbol{x}}$ is

$$\frac{\partial \hat{\boldsymbol{x}}}{\partial \boldsymbol{x}} = \frac{1}{\tilde{\sigma}}\Big(I - \frac{1}{D}\mathbf{1}\mathbf{1}^\top - \frac{1}{D}\hat{\boldsymbol{x}}\hat{\boldsymbol{x}}^\top\Big),$$

a symmetric matrix. Hence, with $\bar{\hat{\boldsymbol{x}}} = \bar{\boldsymbol{y}} \odot \boldsymbol{\gamma}$,

$$\bar{\boldsymbol{x}} = \frac{1}{\tilde{\sigma}}\Big(\bar{\hat{\boldsymbol{x}}} - \operatorname{mean}(\bar{\hat{\boldsymbol{x}}})\mathbf{1} - \hat{\boldsymbol{x}}\operatorname{mean}(\bar{\hat{\boldsymbol{x}}} \odot \hat{\boldsymbol{x}})\Big), \qquad \bar{\boldsymbol{\gamma}} = \sum_{\text{rows}} \bar{\boldsymbol{y}} \odot \hat{\boldsymbol{x}}, \qquad \bar{\boldsymbol{\beta}} = \sum_{\text{rows}} \bar{\boldsymbol{y}}.$$

Derivation. $\mathrm{d}\mu = \frac{1}{D}\mathbf{1}^\top \mathrm{d}\boldsymbol{x}$. Since $\mathbf{1}^\top(\boldsymbol{x} - \mu\mathbf{1}) = 0$, $\mathrm{d}v = \frac{2}{D}(\boldsymbol{x} - \mu\mathbf{1})^\top \mathrm{d}\boldsymbol{x}$ and $\mathrm{d}\tilde{\sigma} = \mathrm{d}v/(2\tilde{\sigma}) = \frac{1}{D}\hat{\boldsymbol{x}}^\top \mathrm{d}\boldsymbol{x}$. Then $\mathrm{d}\hat{\boldsymbol{x}} = \frac{\mathrm{d}\boldsymbol{x} - \mathbf{1}\mathrm{d}\mu}{\tilde{\sigma}} - \frac{(\boldsymbol{x}-\mu\mathbf{1})\mathrm{d}\tilde{\sigma}}{\tilde{\sigma}^2} = \frac{1}{\tilde{\sigma}}\Big(\mathrm{d}\boldsymbol{x} - \frac{1}{D}\mathbf{1}\mathbf{1}^\top \mathrm{d}\boldsymbol{x} - \frac{1}{D}\hat{\boldsymbol{x}}\hat{\boldsymbol{x}}^\top \mathrm{d}\boldsymbol{x}\Big)$. The backward pass is $J^\top \bar{\hat{\boldsymbol{x}}} = J\bar{\hat{\boldsymbol{x}}}$, written without forming J. □

Worked Example 12.2: The layer-norm Jacobian.

$\boldsymbol{x} = (1, 2, 3)$, $\varepsilon = 0$.

Statistics. $\mu = 2$, $v = \frac{2}{3}$, $\tilde{\sigma} = \sqrt{2/3} \approx 0.8165$, $\hat{\boldsymbol{x}} = (-1.2247, 0, 1.2247)$.

Jacobian. $\frac{1}{D}\hat{\boldsymbol{x}}\hat{\boldsymbol{x}}^\top = \frac{1}{2}\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$, so $J = \frac{1}{0.8165}\begin{bmatrix} 1/6 & -1/3 & 1/6 \\ -1/3 & 2/3 & -1/3 \\ 1/6 & -1/3 & 1/6 \end{bmatrix} = \begin{bmatrix} 0.204 & -0.408 & 0.204 \\ -0.408 & 0.816 & -0.408 \\ 0.204 & -0.408 & 0.204 \end{bmatrix}$.

Invariances. $J\mathbf{1} = \mathbf{0}$ (shifting $\boldsymbol{x}$ changes nothing) and $J\hat{\boldsymbol{x}} = \mathbf{0}$ (scaling $\boldsymbol{x}$ changes nothing): rows sum to 0, and $0.204 \cdot (-1.2247) + 0.204 \cdot 1.2247 = 0$. ✓

Batch normalisation uses the same formulas, but standardises each *feature* (column) across the mini-batch instead of each example across its features. The backward pass is Theorem 12.3 applied column-wise, with D replaced by the batch size N. A consequence: in batch norm the gradient for one example depends on every other example in the batch.

Exercises

Exercise 12.1 ⋆ For $\boldsymbol{s} = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4})$ and $\bar{\boldsymbol{s}} = (0, 1, 0)$, compute $\bar{\boldsymbol{a}}$ with Theorem 12.1, and compare with the softmax Jacobian of Worked Example 4.2.

Exercise 12.2 ⋆ A head has $N = 128$ tokens, $D = 512$, $d_k = d_v = 64$. Give the shapes of W_q, Q, S, A, Y and of $\bar{W}_q$ and $\bar{X}$.

Exercise 12.3 ⋆⋆ Derive $\bar{V}$ and $\bar{A}$ for $Y = AV$ with the differential method.

Exercise 12.4 ⋆⋆ Derive the second form of Theorem 12.1: $(\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\top)\bar{\boldsymbol{s}} = \boldsymbol{s} \odot (\bar{\boldsymbol{s}} - (\boldsymbol{s}^\top \bar{\boldsymbol{s}})\mathbf{1})$. Show that $\mathbf{1}^\top \bar{\boldsymbol{a}} = 0$.

Exercise 12.5 $\star\star$ Show that layer normalisation (with $\varepsilon = 0$) is invariant to $\boldsymbol{x} \mapsto c\boldsymbol{x} + b\mathbf{1}$ for $c > 0$, and deduce $J\mathbf{1} = \mathbf{0}$ and $J\boldsymbol{x} = \mathbf{0}$. Verify $J\mathbf{1} = \mathbf{0}$ from the formula of Theorem 12.3.

Exercise 12.6 $\star\star\star$ Let $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^d$ have independent entries with mean 0 and variance 1. (a) Show $\operatorname{Var}(\boldsymbol{q}^\top \boldsymbol{k}) = d$. (b) Show that as the scores in a row spread out, so that $\boldsymbol{s} \to \boldsymbol{e}_j$ (one-hot), the softmax Jacobian $\operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\top \to 0$. (c) Explain the role of $1/\sqrt{d_k}$.

APPENDIX A

Formula Sheet

Conventions	
Gradient	$\nabla f = \partial f/\partial \boldsymbol{x}$: column, shape of $\boldsymbol{x}$; $\partial f/\partial X$: shape of X
Jacobian	$J_{ij} = \partial y_i/\partial x_j$, $m \times n$; $\nabla f = J^\mathsf{T}$ for scalar f
Linear approximation	$f(X+\Delta) \approx f(X) + \operatorname{tr}\left((\partial f/\partial X)^\mathsf{T}\Delta\right)$
Identification rule	$\mathrm{d}f = \operatorname{tr}(G^\mathsf{T}\mathrm{d}X) \;\Rightarrow\; \partial f/\partial X = G$
Identities	
Trace	$\operatorname{tr}(AB) = \operatorname{tr}(BA)$; $\operatorname{tr}(A^\mathsf{T}B) = \sum a_{ij}b_{ij}$; $\boldsymbol{x}^\mathsf{T}A\boldsymbol{x} = \operatorname{tr}(A\boldsymbol{x}\boldsymbol{x}^\mathsf{T})$
Determinant	$\lvert AB\rvert = \lvert A\rvert\,\lvert B\rvert$, $\lvert A^{-1}\rvert = 1/\lvert A\rvert$, $\ln\lvert S\rvert = 2\sum \ln l_{ii}$ $(S = LL^\mathsf{T})$
Block inverse	$P = (A - BD^{-1}C)^{-1}$; $M^{-1} = \begin{bmatrix} P & -PBD^{-1} \\ -D^{-1}CP & D^{-1}+D^{-1}CPBD^{-1} \end{bmatrix}$
Woodbury	$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}$
Determinant lemma	$\left\lvert A + \boldsymbol{u}\boldsymbol{v}^\mathsf{T}\right\rvert = (1+\boldsymbol{v}^\mathsf{T}A^{-1}\boldsymbol{u})\,\lvert A\rvert$
Push-through	$(I+AB)^{-1}A = A(I+BA)^{-1}$
Spectral function	$f(S) = Qf(\Lambda)Q^\mathsf{T}$ for $S = Q\Lambda Q^\mathsf{T}$
Vector derivatives	
Gradients	$\boldsymbol{a}^\mathsf{T}\boldsymbol{x} \to \boldsymbol{a}$; $\boldsymbol{x}^\mathsf{T}A\boldsymbol{x} \to (A+A^\mathsf{T})\boldsymbol{x}$; $\lVert B\boldsymbol{x}-\boldsymbol{b}\rVert^2 \to 2B^\mathsf{T}(B\boldsymbol{x}-\boldsymbol{b})$; $\lVert\boldsymbol{x}\rVert \to \boldsymbol{x}/\lVert\boldsymbol{x}\rVert$
Jacobians	$A\boldsymbol{x} \to A$; $h(\boldsymbol{x}) \to \operatorname{diag}(h'(\boldsymbol{x}))$; $\operatorname{softmax}(\boldsymbol{a}) \to \operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\mathsf{T}$
Log-sum-exp	$\nabla \operatorname{lse}(\boldsymbol{a}) = \operatorname{softmax}(\boldsymbol{a})$; $\nabla^2 \operatorname{lse} = \operatorname{diag}(\boldsymbol{s}) - \boldsymbol{s}\boldsymbol{s}^\mathsf{T}$
Regression	$\nabla\frac{1}{2}\lVert\Phi\boldsymbol{w}-\boldsymbol{t}\rVert^2 = \Phi^\mathsf{T}(\Phi\boldsymbol{w}-\boldsymbol{t})$; $\boldsymbol{w} = (\Phi^\mathsf{T}\Phi+\lambda I)^{-1}\Phi^\mathsf{T}\boldsymbol{t}$
Logistic regression	$\nabla E = \Phi^\mathsf{T}(\boldsymbol{y}-\boldsymbol{t})$, $\nabla^2 E = \Phi^\mathsf{T}R\Phi$, $R = \operatorname{diag}(y_n(1-y_n))$
Differentials and matrix derivatives	
Differentials	$\mathrm{d}(XY) = \mathrm{d}X\,Y + X\mathrm{d}Y$; $\mathrm{d}X^{-1} = -X^{-1}\mathrm{d}X\,X^{-1}$; $\mathrm{d}\ln\lvert X\rvert = \operatorname{tr}(X^{-1}\mathrm{d}X)$
Traces	$\operatorname{tr}(AXB) \to A^\mathsf{T}B^\mathsf{T}$; $\operatorname{tr}(X^\mathsf{T}AX) \to (A+A^\mathsf{T})X$; $\boldsymbol{a}^\mathsf{T}X\boldsymbol{b} \to \boldsymbol{a}\boldsymbol{b}^\mathsf{T}$
Determinants	$\ln\lvert X\rvert \to X^{-\mathsf{T}}$; $\lvert X\rvert \to \lvert X\rvert\,X^{-\mathsf{T}}$
Inverses	$\operatorname{tr}(X^{-1}A) \to -X^{-\mathsf{T}}A^\mathsf{T}X^{-\mathsf{T}}$; $\boldsymbol{a}^\mathsf{T}X^{-1}\boldsymbol{b} \to -X^{-\mathsf{T}}\boldsymbol{a}\boldsymbol{b}^\mathsf{T}X^{-\mathsf{T}}$
Frobenius	$\lVert AX-B\rVert_F^2 \to 2A^\mathsf{T}(AX-B)$; $\lVert XA-B\rVert_F^2 \to 2(XA-B)A^\mathsf{T}$
Hadamard	$\operatorname{tr}(A^\mathsf{T}(B\odot\mathrm{d}X)) = \operatorname{tr}((A\odot B)^\mathsf{T}\mathrm{d}X)$
Optimisation	
Gradient descent	converges iff $\eta < 2/\lambda_{\max}$; best rate $\frac{\kappa-1}{\kappa+1}$ at $\eta = \frac{2}{\lambda_{\min}+\lambda_{\max}}$

Newton	$\boldsymbol{w} \leftarrow \boldsymbol{w} - H^{-1}\nabla E$; Gauss–Newton $H \approx J^{\mathsf{T}}J$
Lagrange	$\nabla f + \lambda \nabla g = \mathbf{0}$; $\max \boldsymbol{u}^{\mathsf{T}} S \boldsymbol{u}$ s.t. $\|\boldsymbol{u}\| = 1$: top eigenvector

Gaussians

Log-density	$-\frac{D}{2}\ln 2\pi - \frac{1}{2}\ln\lvert\Sigma\rvert - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$
Maximum likelihood	$\boldsymbol{\mu} = \bar{\boldsymbol{x}}$, $\Sigma = \frac{1}{N}\sum(\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}$
$\partial/\partial\Sigma \ln\mathcal{N}$	$-\frac{1}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\Sigma^{-1}$
Conditional	$\boldsymbol{\mu}_{a\mid b} = \boldsymbol{\mu}_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)$, $\Sigma_{a\mid b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$
KL (VAE)	$\frac{1}{2}\sum(\sigma^2 + \mu^2 - 1 - \ln\sigma^2)$; $\partial/\partial\mu = \mu$, $\partial/\partial \ln\sigma^2 = \frac{1}{2}(\sigma^2 - 1)$
Reparameterisation	$\boldsymbol{z} = \boldsymbol{\mu} + L\boldsymbol{\epsilon}$: $\nabla_{\boldsymbol{\mu}} = \mathbb{E}[\nabla f]$, $\partial/\partial L = \mathbb{E}[\nabla f \boldsymbol{\epsilon}^{\mathsf{T}}]$

Backpropagation

Linear $Y = XW + \mathbf{1}\boldsymbol{b}^{\mathsf{T}}$	$\bar{W} = X^{\mathsf{T}}\bar{Y}$, $\bar{X} = \bar{Y}W^{\mathsf{T}}$, $\bar{\boldsymbol{b}} = \bar{Y}^{\mathsf{T}}\mathbf{1}$
Activation $H = h(Z)$	$\bar{Z} = \bar{H} \odot h'(Z)$
Softmax + cross-entropy	$\bar{A} = \frac{1}{N}(S - T)$
Softmax row	$\bar{\boldsymbol{a}} = \boldsymbol{s} \odot (\bar{\mathbf{s}} - (\boldsymbol{s}^{\mathsf{T}}\bar{\mathbf{s}})\mathbf{1})$
Attention	$\bar{V} = A^{\mathsf{T}}\bar{Y}$, $\bar{A} = \bar{Y}V^{\mathsf{T}}$, $\bar{Q} = \bar{S}K/\sqrt{d_k}$, $\bar{K} = \bar{S}^{\mathsf{T}}Q/\sqrt{d_k}$
Layer norm	$\bar{\boldsymbol{x}} = \frac{1}{\hat{\sigma}}\left(\bar{\hat{\boldsymbol{x}}} - \mathrm{mean}(\bar{\hat{\boldsymbol{x}}}) - \hat{\boldsymbol{x}}\,\mathrm{mean}(\bar{\hat{\boldsymbol{x}}} \odot \hat{\boldsymbol{x}})\right)$

Flows and numerics

Change of variables	$\ln p_x(\boldsymbol{x}) = \ln p_z(\boldsymbol{f}(\boldsymbol{x})) + \ln\left\lvert\det J_f(\boldsymbol{x})\right\rvert$
Coupling layer	$\ln\lvert\det J\rvert = \sum_j s_j(\boldsymbol{x}_A)$
Continuous flow	$\frac{\mathrm{d}}{\mathrm{d}t}\ln p = -\operatorname{tr}(\partial \boldsymbol{f}/\partial \boldsymbol{z})$; Hutchinson $\operatorname{tr} A = \mathbb{E}[\boldsymbol{\epsilon}^{\mathsf{T}}A\boldsymbol{\epsilon}]$
Finite differences	central: error $\frac{h^2}{6}\lvert f'''\rvert + \frac{\varepsilon\lvert f\rvert}{h}$, $h \approx 10^{-5}$–10^{-6}
Stable forms	$\mathrm{lse}(\boldsymbol{a}) = m + \ln\sum e^{a_k - m}$; $\mathrm{softplus} = \max(a,0) + \ln(1 + e^{-\lvert a\rvert})$

APPENDIX B

Solutions to the Exercises

Every exercise is solved here, in order. For derivations the solutions show each line you should have on paper; for numerical exercises they show the intermediate values so you can find where a calculation went wrong. The page number after each solution points back to the exercise.

Chapter 1 Notation, Shapes and Layout Conventions

Solution 1.1 (p. 5). $\partial f/\partial x_1 = x_2 - 2 = 0$, $\partial f/\partial x_2 = x_1 + x_3^2 = 2$, $\partial f/\partial x_3 = 2x_2x_3 = -4$, so $\nabla f = (0, 2, -4)$. $f(1, 2, -1) = 2 + 2 - 2 = 2$; prediction $2 + 2 \cdot 0.1 = 2.2$. Exact: $f(1, 2.1, -1) = 2.1 + 2.1 - 2 = 2.2$ — exact here because f is linear in x_2.

Solution 1.2 (p. 5). $J = \begin{bmatrix} x_2x_3 & x_1x_3 & x_1x_2 \\ 1 & 1 & 1 \end{bmatrix}$, a 2×3 matrix (2 outputs, 3 inputs).

Solution 1.3 (p. 5). The Jacobian is diagonal: $\operatorname{diag}(1 - \tanh^2 0,\ 1 - \tanh^2 \ln 2) = \operatorname{diag}(1,\ 1 - \frac{9}{25}) = \operatorname{diag}(1,\ \frac{16}{25})$.

Solution 1.4 (p. 5). $\nabla f = (3x_1^2 + x_2^2,\ 2x_1x_2 - 1) = (4, 1)$ at $(1, 1)$. $H = \begin{bmatrix} 6x_1 & 2x_2 \\ 2x_2 & 2x_1 \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ 2 & 2 \end{bmatrix}$. With $f(1, 1) = 1$: $f(1 + h_1, 1 + h_2) \approx 1 + 4h_1 + h_2 + \frac{1}{2}(6h_1^2 + 4h_1h_2 + 2h_2^2) = 1 + 4h_1 + h_2 + 3h_1^2 + 2h_1h_2 + h_2^2$.

Solution 1.5 (p. 5). $\frac{\partial f}{\partial X} = \begin{bmatrix} 2x_{11} & x_{21} \\ x_{12} & 3 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 2 & 3 \end{bmatrix}$ at the given X. Prediction: $\operatorname{tr}\left(\left[\begin{smallmatrix} 2 & 2 \\ 3 & 3 \end{smallmatrix}\right]\left[\begin{smallmatrix} 0.01 & 0 \\ 0 & 0.01 \end{smallmatrix}\right]\right) = 0.02 + 0.03 = 0.05$. Exact: $f(X) = 1 + 6 + 12 = 19$, $f(X + \Delta) = 1.0201 + 6 + 12.03 = 19.0501$; difference 0.0501. ✓

Solution 1.6 (p. 5). (a) $g_i = \sum_k a_{ik}x_k$, so $\partial g_i/\partial x_j = a_{ij}$: the (i, j) entry of the Jacobian is a_{ij}, i.e. $J = A$.
(b) Denominator layout transposes: $A^\top$.
(c) The result is a row vector, so the source uses numerator layout for a scalar output (a $1 \times n$ Jacobian). In our convention the gradient is its transpose, $(A + A^\top)\boldsymbol{x}$.

Chapter 2 Functions of Matrices

Solution 2.1 (p. 9). $AB = \left[\begin{smallmatrix}2&3\\4&7\end{smallmatrix}\right]$, $\operatorname{tr} = 9$; $BA = \left[\begin{smallmatrix}3&4\\4&6\end{smallmatrix}\right]$, $\operatorname{tr} = 9$. ✓ $\langle A, B\rangle_F = 0+2+3+4 = 9$ too, but only because this B is symmetric. In general $\operatorname{tr}(AB) = \sum_{ij} a_{ij}b_{ji} = \langle A, B^{\mathsf{T}}\rangle_F$, which equals $\langle A, B\rangle_F$ when A or B is symmetric. For $B = \left[\begin{smallmatrix}0&1\\0&0\end{smallmatrix}\right]$, $\operatorname{tr}(AB) = 3$ but $\langle A, B\rangle_F = 2$.

Solution 2.2 (p. 9). $l_{11} = 3$, $l_{21} = 3/3 = 1$, $l_{22} = \sqrt{5-1} = 2$: $L = \left[\begin{smallmatrix}3&0\\1&2\end{smallmatrix}\right]$. $\ln|S| = 2(\ln 3 + \ln 2) = \ln 36$. And $\det S = 45 - 9 = 36$. ✓

Solution 2.3 (p. 9). Eigenvalues 16 on $(1,1)/\sqrt{2}$ and 4 on $(1,-1)/\sqrt{2}$. Square roots 4 and 2: $\frac{4}{2}\left[\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right] + \frac{2}{2}\left[\begin{smallmatrix}1&-1\\-1&1\end{smallmatrix}\right] = \left[\begin{smallmatrix}3&1\\1&3\end{smallmatrix}\right]$. Check: $\left[\begin{smallmatrix}3&1\\1&3\end{smallmatrix}\right]^2 = \left[\begin{smallmatrix}10&6\\6&10\end{smallmatrix}\right]$. ✓

Solution 2.4 (p. 9). $S^{-1/2} = \frac{1}{3}\boldsymbol{q}_1\boldsymbol{q}_1^{\mathsf{T}} + 1\boldsymbol{q}_2\boldsymbol{q}_2^{\mathsf{T}} = \frac{1}{6}\left[\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right] + \frac{1}{2}\left[\begin{smallmatrix}1&-1\\-1&1\end{smallmatrix}\right] = \frac{1}{3}\left[\begin{smallmatrix}2&-1\\-1&2\end{smallmatrix}\right]$. Since $S^{-1/2}$ and S share eigenvectors, the product is $Q\Lambda^{-1/2}\Lambda\Lambda^{-1/2}Q^{\mathsf{T}} = I$. Data $\boldsymbol{x}$ with covariance S become *whitened* by $\boldsymbol{x} \mapsto S^{-1/2}\boldsymbol{x}$: uncorrelated, unit variance in every direction.

Solution 2.5 (p. 9). (a) $\operatorname{tr}\ln S = \operatorname{tr}(Q\ln\Lambda Q^{\mathsf{T}}) = \operatorname{tr}(\ln\Lambda\, Q^{\mathsf{T}}Q) = \sum_i \ln\lambda_i = \ln\prod_i\lambda_i = \ln\det S$.
(b) $cA = (cI)A$ and $\det(cI) = c^n$ (diagonal), so $\det(cA) = c^n \det A$.
(c) $1000\ln 0.5 \approx -693.1$.

Solution 2.6 (p. 10). $N^2 = 0$, so the series stops: $\exp(tN) = I + tN = \left[\begin{smallmatrix}1&t\\0&1\end{smallmatrix}\right]$. Similarly $\exp(B) = \left[\begin{smallmatrix}1&0\\1&1\end{smallmatrix}\right]$. Then $\exp(A)\exp(B) = \left[\begin{smallmatrix}1&1\\0&1\end{smallmatrix}\right]\left[\begin{smallmatrix}1&0\\1&1\end{smallmatrix}\right] = \left[\begin{smallmatrix}2&1\\1&1\end{smallmatrix}\right]$, whereas $\exp(A+B) = \left[\begin{smallmatrix}\cosh 1&\sinh 1\\\sinh 1&\cosh 1\end{smallmatrix}\right] \approx \left[\begin{smallmatrix}1.543&1.175\\1.175&1.543\end{smallmatrix}\right]$. They differ because $AB = \left[\begin{smallmatrix}1&0\\0&0\end{smallmatrix}\right] \neq BA = \left[\begin{smallmatrix}0&0\\0&1\end{smallmatrix}\right]$.

Chapter 3 Matrix Identities

Solution 3.1 (p. 14). $AB = \left[\begin{smallmatrix}2&1\\1&0\end{smallmatrix}\right]$, $ABC = \left[\begin{smallmatrix}4&3\\2&0\end{smallmatrix}\right]$, trace 4. $CA = \left[\begin{smallmatrix}2&4\\0&3\end{smallmatrix}\right]$, $CAB = \left[\begin{smallmatrix}4&2\\3&0\end{smallmatrix}\right]$, trace 4. ✓ $AC = \left[\begin{smallmatrix}2&6\\0&3\end{smallmatrix}\right]$, $ACB = \left[\begin{smallmatrix}6&2\\3&0\end{smallmatrix}\right]$, trace $6 \neq 4$.

Solution 3.2 (p. 14). $\sum_n \boldsymbol{x}_n^{\mathsf{T}}A\boldsymbol{x}_n = \operatorname{tr}\left(A\sum_n \boldsymbol{x}_n\boldsymbol{x}_n^{\mathsf{T}}\right) = \operatorname{tr}(AX^{\mathsf{T}}X)$ where the rows of X are the $\boldsymbol{x}_n$. $X^{\mathsf{T}}X = \left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$; $AX^{\mathsf{T}}X = \left[\begin{smallmatrix}3&3\\2&4\end{smallmatrix}\right]$, trace 7. Directly: $1+2+(1+1+0+2) = 7$. ✓

Solution 3.3 (p. 14). $\det(I + \mathbf{1}\mathbf{1}^{\mathsf{T}}) = (1 + \mathbf{1}^{\mathsf{T}}\mathbf{1})\det I = 1 + n$. For $n = 2$: $\det\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right] = 3$. ✓

Solution 3.4 (p. 14). $(I + \mathbf{1}\mathbf{1}^{\mathsf{T}})^{-1} = I - \dfrac{\mathbf{1}\mathbf{1}^{\mathsf{T}}}{1+n}$. For $n = 2$: $I - \frac{1}{3}\left[\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right] = \frac{1}{3}\left[\begin{smallmatrix}2&-1\\-1&2\end{smallmatrix}\right]$, the inverse of $\left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$. ✓

Solution 3.5 (p. 14). $B = (1,1)$, $D^{-1} = \operatorname{diag}(\frac{1}{3}, \frac{1}{2})$, $BD^{-1}B^{\mathsf{T}} = \frac{1}{3} + \frac{1}{2} = \frac{5}{6}$. $M/D = 4 - \frac{5}{6} = \frac{19}{6}$. $\det M = \det D \cdot \frac{19}{6} = 6 \cdot \frac{19}{6} = 19$. Top-left entry of M^{-1}: $(M/D)^{-1} = \frac{6}{19}$. (Check by cofactors: $(M^{-1})_{11} = \det D / \det M = 6/19$.) ✓

Solution 3.6 (p. 14). (a) $(I+UV)\big(I-U(I+VU)^{-1}V\big) = I+UV-U(I+VU)^{-1}V - UVU(I+VU)^{-1}V = I+UV-U(I+VU)(I+VU)^{-1}V = I+UV-UV = I$.
(b) Directly: forming UU^{T} is $O(n^2k)$ and solving an $n\times n$ system $O(n^3)$. Via Woodbury with $A=\lambda I$: $(\lambda I+UU^{\mathsf{T}})^{-1}\boldsymbol{b} = \frac{1}{\lambda}\big(\boldsymbol{b}-U(\lambda I_k+U^{\mathsf{T}}U)^{-1}U^{\mathsf{T}}\boldsymbol{b}\big)$, costing $O(nk^2)$ to form $U^{\mathsf{T}}U$, $O(k^3)$ to solve, and $O(nk)$ for the products — linear in n.

Chapter 4 Derivatives with Respect to Vectors

Solution 4.1 (p. 18). $(A+A^{\mathsf{T}})\boldsymbol{x} = \left[\begin{smallmatrix}2&4\\4&4\end{smallmatrix}\right]\left[\begin{smallmatrix}1\\1\end{smallmatrix}\right] = (6,8)$. Expanded: $\boldsymbol{x}^{\mathsf{T}}A\boldsymbol{x} = x_1^2+4x_1x_2+2x_2^2$; $\partial/\partial x_1 = 2x_1+4x_2 = 6$, $\partial/\partial x_2 = 4x_1+4x_2 = 8$. ✓ (The wrong formula $2A\boldsymbol{x} = (10,4)$ fails.)

Solution 4.2 (p. 18). $\Phi = \left[\begin{smallmatrix}1&0\\1&1\\1&2\end{smallmatrix}\right]$, $\boldsymbol{t} = (1,3,4)$. $\Phi^{\mathsf{T}}\Phi = \left[\begin{smallmatrix}3&3\\3&5\end{smallmatrix}\right]$, $\Phi^{\mathsf{T}}\boldsymbol{t} = (8,11)$. Solve: $\boldsymbol{w} = \frac{1}{6}\left[\begin{smallmatrix}5&-3\\-3&3\end{smallmatrix}\right]\left[\begin{smallmatrix}8\\11\end{smallmatrix}\right] = \frac{1}{6}(7,9) = (\frac{7}{6},\frac{3}{2})$. At $\boldsymbol{w}=\boldsymbol{0}$: $\nabla E = \Phi^{\mathsf{T}}(\boldsymbol{0}-\boldsymbol{t}) = -(8,11)$.

Solution 4.3 (p. 18). $\nabla E = \Phi^{\mathsf{T}}(\Phi\boldsymbol{w}-\boldsymbol{t})+\lambda\boldsymbol{w} = \boldsymbol{0} \Rightarrow (\Phi^{\mathsf{T}}\Phi+\lambda I)\boldsymbol{w} = \Phi^{\mathsf{T}}\boldsymbol{t}$. Hessian $\Phi^{\mathsf{T}}\Phi+\lambda I$: for $\boldsymbol{h}\neq\boldsymbol{0}$, $\boldsymbol{h}^{\mathsf{T}}(\Phi^{\mathsf{T}}\Phi+\lambda I)\boldsymbol{h} = \|\Phi\boldsymbol{h}\|^2+\lambda\|\boldsymbol{h}\|^2 > 0$. So the minimiser is unique.

Solution 4.4 (p. 18). $\partial/\partial a_j \ln\sum_k e^{a_k} = e^{a_j}/\sum_k e^{a_k} = s_j$. Then $\ell = -\sum_k t_k(a_k-\operatorname{lse}(\boldsymbol{a})) = -\boldsymbol{t}^{\mathsf{T}}\boldsymbol{a}+\operatorname{lse}(\boldsymbol{a})\sum_k t_k = -\boldsymbol{t}^{\mathsf{T}}\boldsymbol{a}+\operatorname{lse}(\boldsymbol{a})$, so $\nabla\ell = -\boldsymbol{t}+\boldsymbol{s} = \boldsymbol{s}-\boldsymbol{t}$.

Solution 4.5 (p. 18). $\|\boldsymbol{x}\| = 5$; $J = \frac{1}{5}\big(I-\frac{1}{25}\left[\begin{smallmatrix}9&12\\12&16\end{smallmatrix}\right]\big) = \frac{1}{5}\cdot\frac{1}{25}\left[\begin{smallmatrix}16&-12\\-12&9\end{smallmatrix}\right] = \frac{1}{125}\left[\begin{smallmatrix}16&-12\\-12&9\end{smallmatrix}\right]$. $J\boldsymbol{x} = \frac{1}{125}(48-48,\ -36+36) = \boldsymbol{0}$. Scaling $\boldsymbol{x}$ does not change $\boldsymbol{x}/\|\boldsymbol{x}\|$, so the derivative in the direction of $\boldsymbol{x}$ is zero.

Solution 4.6 (p. 18). $\nabla f = (2x+y-3,\ x+2y) = \boldsymbol{0} \Rightarrow x = -2y$, $-4y+y = 3$, $y=-1$, $x=2$. $H = \left[\begin{smallmatrix}2&1\\1&2\end{smallmatrix}\right]$, eigenvalues $3,1>0$: global minimum at $(2,-1)$, value $4-2+1-6 = -3$.

Solution 4.7 (p. 18). From $\nabla E = \sum_n(y_n-t_n)\boldsymbol{\phi}_n$ and $\nabla_w y_n = \sigma'(a_n)\boldsymbol{\phi}_n = y_n(1-y_n)\boldsymbol{\phi}_n$: $\nabla^2 E = \sum_n y_n(1-y_n)\boldsymbol{\phi}_n\boldsymbol{\phi}_n^{\mathsf{T}} = \Phi^{\mathsf{T}}R\Phi$. For any $\boldsymbol{h}$: $\boldsymbol{h}^{\mathsf{T}}\Phi^{\mathsf{T}}R\Phi\boldsymbol{h} = \sum_n y_n(1-y_n)(\boldsymbol{\phi}_n^{\mathsf{T}}\boldsymbol{h})^2 \geq 0$, so E is convex. Since $0<y_n(1-y_n)$, the form is zero only if $\Phi\boldsymbol{h}=\boldsymbol{0}$; so E is strictly convex exactly when Φ has independent columns.

Chapter 5 Curvature and Optimisation

Solution 5.1 (p. 21). $\eta < 2/50 = 0.04$. $\eta^\star = 2/(2+50) = \frac{1}{26} \approx 0.0385$. $\kappa = 25$, $\rho^\star = \frac{24}{26} \approx 0.923$.

Solution 5.2 (p. 21). $\nabla E = H\boldsymbol{w}-\boldsymbol{b}$, $\nabla^2 E = H$. Newton: $\boldsymbol{w}_1 = \boldsymbol{w}_0 - H^{-1}(H\boldsymbol{w}_0-\boldsymbol{b}) = H^{-1}\boldsymbol{b}$, which is where $\nabla E = \boldsymbol{0}$.

Solution 5.3 (p. 21). Factors $1-0.3 = 0.7$ and $1-1.2 = -0.2$. $\boldsymbol{w}_1 = (0.7,-0.2)$, $\boldsymbol{w}_2 = (0.49,0.04)$, $\boldsymbol{w}_3 = (0.343,-0.008)$. Direct check of step 1: $\nabla E(1,1) = (1,4)$, $\boldsymbol{w}_1 = (1,1)-$

$0.3(1,4) = (0.7, -0.2)$. ✓

Solution 5.4 (p. 21). (a) Scaling a column of Φ by c scales the corresponding row and column of H, changing its eigenvalues; bringing both columns to similar magnitudes removes the factor $\sim 10^6$ in κ. (b) But $x_2 \approx x_1$ after scaling: the columns are nearly parallel, so $\Phi^\top\Phi$ is close to singular and κ stays large. Decorrelating the features (whitening, or PCA, Chapter 10) or adding a ridge term λI fixes the remaining ill-conditioning.

Solution 5.5 (p. 21). $y_n = \sigma(-0.4x_n)$: for $x = 1, 2, -1, -2$: $\sigma(-0.4) \approx 0.401$, $\sigma(-0.8) \approx 0.310$, $\sigma(0.4) \approx 0.599$, $\sigma(0.8) \approx 0.690$. $g = \sum(y_n - t_n)x_n = (-0.599)(1) + (0.310)(2) + (0.599)(-1) + (-0.310)(-2) = -0.599 + 0.620 - 0.599 + 0.620 = 0.042$. Positive gradient, so the next step moves further left, by a small amount: w_1 is already close to the minimum.

Solution 5.6 (p. 21). $\nabla E = \sum_n (y_n - t_n)\nabla y_n$. Differentiating again with the product rule: $\nabla^2 E = \sum_n \nabla y_n \nabla y_n^\top + \sum_n (y_n - t_n)\nabla^2 y_n$. For any $\boldsymbol{h}$, $\boldsymbol{h}^\top J^\top J\boldsymbol{h} = \|J\boldsymbol{h}\|^2 \geq 0$. The second term can be indefinite: e.g. one point, $y(w) = w^2$, $t = 1$: $E = \frac{1}{2}(w^2 - 1)^2$ has $E''(0) = -2 < 0$ while $J^\top J = (2w)^2 = 0$ at $w = 0$.

Chapter 6 The Differential Method

Solution 6.1 (p. 26). $\mathrm{d}f = \mathrm{tr}(A\mathrm{d}XB) = \mathrm{tr}(BA\mathrm{d}X)$, so $\frac{\partial f}{\partial X} = (BA)^\top = A^\top B^\top$. Shapes: $A^\top$ is $m \times p$, $B^\top$ is $p \times n$, product $m \times n$ — the shape of X. ✓

Solution 6.2 (p. 26). $\mathrm{d}f = \mathrm{d}\boldsymbol{x}^\top A\boldsymbol{x} + \boldsymbol{x}^\top A\mathrm{d}\boldsymbol{x} = \boldsymbol{x}^\top A^\top \mathrm{d}\boldsymbol{x} + \boldsymbol{x}^\top A\mathrm{d}\boldsymbol{x}$ (the first term is a scalar, so transpose it). Hence $\mathrm{d}f = \big((A + A^\top)\boldsymbol{x}\big)^\top \mathrm{d}\boldsymbol{x}$ and $\nabla f = (A + A^\top)\boldsymbol{x}$.

Solution 6.3 (p. 26). $|X| = 10$, $X^{-1} = \frac{1}{10}\left[\begin{smallmatrix} 4 & -1 \\ -2 & 3 \end{smallmatrix}\right]$, $X^{-\top} = \frac{1}{10}\left[\begin{smallmatrix} 4 & -2 \\ -1 & 3 \end{smallmatrix}\right]$, so $\frac{\partial |X|}{\partial X} = \left[\begin{smallmatrix} 4 & -2 \\ -1 & 3 \end{smallmatrix}\right]$. Worked Example 1.3 gives $\left[\begin{smallmatrix} x_{22} & -x_{21} \\ -x_{12} & x_{11} \end{smallmatrix}\right] = \left[\begin{smallmatrix} 4 & -2 \\ -1 & 3 \end{smallmatrix}\right]$. ✓

Solution 6.4 (p. 26). $R = XA - B$, $\mathrm{d}R = \mathrm{d}X\,A$. $\mathrm{d}f = 2\,\mathrm{tr}(R^\top \mathrm{d}XA) = 2\,\mathrm{tr}(AR^\top \mathrm{d}X)$, so $\frac{\partial f}{\partial X} = 2(AR^\top)^\top = 2RA^\top = 2(XA - B)A^\top$.

Solution 6.5 (p. 26). $\mathrm{d}\,\mathrm{tr}(X^2) = \mathrm{tr}(\mathrm{d}X\,X + X\mathrm{d}X) = 2\,\mathrm{tr}(X\mathrm{d}X)$, so the gradient is $2X^\top$. $\mathrm{d}\,\mathrm{tr}(X^\top X) = \mathrm{tr}(\mathrm{d}X^\top X + X^\top \mathrm{d}X) = 2\,\mathrm{tr}(X^\top \mathrm{d}X)$, gradient $2X$. They differ because $\mathrm{tr}(X^2) = \sum_{ij} x_{ij}x_{ji}$ pairs x_{ij} with its mirror entry, while $\mathrm{tr}(X^\top X) = \sum_{ij} x_{ij}^2$ is the squared Frobenius norm. They agree for symmetric X.

Solution 6.6 (p. 26). $\mathrm{d}f = \mathrm{tr}\left((X^\top X)^{-1}\mathrm{d}(X^\top X)\right) = \mathrm{tr}\left((X^\top X)^{-1}(\mathrm{d}X^\top X + X^\top \mathrm{d}X)\right)$. The two terms are transposes of each other inside a trace (using symmetry of $(X^\top X)^{-1}$), so $\mathrm{d}f = 2\,\mathrm{tr}((X^\top X)^{-1}X^\top \mathrm{d}X)$ and $\frac{\partial f}{\partial X} = 2\big((X^\top X)^{-1}X^\top\big)^\top = 2X(X^\top X)^{-1}$.

Solution 6.7 (p. 26). $\mathrm{d}f = \mathrm{tr}\left(A^\top(\sigma'(X) \odot \mathrm{d}X)\right) = \mathrm{tr}\left((A \odot \sigma'(X))^\top \mathrm{d}X\right)$, so $\frac{\partial f}{\partial X} = A \odot \sigma(X) \odot (1 - \sigma(X))$.

Solution 6.8 (p. 26). $f = (X^{-1})_{ij}A_{ji}$. Then $\partial f/\partial X_{kl} = -(X^{-1})_{ik}(X^{-1})_{lj}A_{ji} = -(X^{-1}AX^{-1})_{lk}$ (contract j then i: $(X^{-1})_{lj}A_{ji}(X^{-1})_{ik} = (X^{-1}AX^{-1})_{lk}$). The (k,l) entry of the gradient is the (l,k) entry of $-X^{-1}AX^{-1}$, so $\frac{\partial f}{\partial X} = -(X^{-1}AX^{-1})^\top = -X^{-\top}A^\top X^{-\top}$. ✓

Chapter 7 Gaussians and Maximum Likelihood

Solution 7.1 (p. 30). $\boldsymbol{\mu} = (1,1)$. Deviations $(-1,-1)$, $(1,-1)$, $(0,2)$. $S = \left[\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right] + \left[\begin{smallmatrix}1&-1\\-1&1\end{smallmatrix}\right] + \left[\begin{smallmatrix}0&0\\0&4\end{smallmatrix}\right] = \left[\begin{smallmatrix}2&0\\0&6\end{smallmatrix}\right]$, $\Sigma_{\mathrm{ML}} = \left[\begin{smallmatrix}2/3&0\\0&2\end{smallmatrix}\right]$.

Solution 7.2 (p. 30). $\mu_{1|2} = 1 + \frac{1}{2}(3-1) = 2$, $\Sigma_{1|2} = 2 - \frac{1}{2} = \frac{3}{2}$. So $x_1 \mid x_2 = 3 \sim \mathcal{N}(2, \frac{3}{2})$.

Solution 7.3 (p. 30). $\mathrm{d}\ln\mathcal{N} = -\frac{1}{2}\operatorname{tr}(\Sigma^{-1}\mathrm{d}\Sigma) - \frac{1}{2}\boldsymbol{a}^\top \mathrm{d}(\Sigma^{-1})\boldsymbol{a}$ with $\boldsymbol{a} = \boldsymbol{x} - \boldsymbol{\mu}$. Since $\mathrm{d}(\Sigma^{-1}) = -\Sigma^{-1}\mathrm{d}\Sigma\Sigma^{-1}$, the second term is $+\frac{1}{2}\operatorname{tr}(\Sigma^{-1}\boldsymbol{a}\boldsymbol{a}^\top\Sigma^{-1}\mathrm{d}\Sigma)$. Reading off (symmetric matrices): $-\frac{1}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}\boldsymbol{a}\boldsymbol{a}^\top\Sigma^{-1}$. Summing: $-\frac{N}{2}\Sigma^{-1} + \frac{1}{2}\Sigma^{-1}S\Sigma^{-1} = 0$; multiply by 2Σ on both sides: $-NI + S\Sigma^{-1} = 0$, so $\Sigma = S/N$.

Solution 7.4 (p. 30). $\operatorname{tr}(\Sigma_2^{-1}\Sigma_1) = 1$, quadratic term $\frac{1}{2} + \frac{1}{2} = 1$, $\ln(4/1) = \ln 4$. KL $= \frac{1}{2}(1 + 1 - 2 + \ln 4) = \ln 2 \approx 0.693$. Reverse: $\operatorname{tr}(\Sigma_1^{-1}\Sigma_2) = 4$, quadratic 2, $\ln(1/4)$: $\frac{1}{2}(4 + 2 - 2 - \ln 4) = 2 - \ln 2 \approx 1.307$. Not symmetric.

Solution 7.5 (p. 30). $\partial\mathrm{KL}/\partial\mu_j = \mu_j = 0$ and $\partial\mathrm{KL}/\partial v_j = \frac{1}{2}(e^{v_j} - 1) = 0 \Rightarrow v_j = 0$. Second derivatives: $\partial^2\mathrm{KL}/\partial\mu_j^2 = 1$, $\partial^2\mathrm{KL}/\partial v_j^2 = \frac{1}{2}e^{v_j} = \frac{1}{2}$ at 0, mixed 0. Positive definite: a minimum, where $q = p$ and KL $= 0$.

Solution 7.6 (p. 30). For fixed $\boldsymbol{\epsilon}$, $\mathrm{d}f = \nabla f(\boldsymbol{z})^\top \mathrm{d}\boldsymbol{z} = \nabla f^\top(\mathrm{d}L)\boldsymbol{\epsilon} = \operatorname{tr}(\boldsymbol{\epsilon}\nabla f^\top \mathrm{d}L)$, so $\partial f/\partial L = (\boldsymbol{\epsilon}\nabla f^\top)^\top = \nabla f\boldsymbol{\epsilon}^\top$; take expectations (differentiation and $\mathbb{E}$ commute because the distribution of $\boldsymbol{\epsilon}$ does not depend on L). For $f = \boldsymbol{a}^\top\boldsymbol{z}$: $\nabla f = \boldsymbol{a}$ and $\mathbb{E}[\boldsymbol{a}\boldsymbol{\epsilon}^\top] = \boldsymbol{a}\mathbb{E}[\boldsymbol{\epsilon}]^\top = 0$. Indeed $\mathbb{E}[\boldsymbol{a}^\top\boldsymbol{z}] = \boldsymbol{a}^\top\boldsymbol{\mu}$ does not depend on L. ✓

Chapter 8 The Chain Rule and Backpropagation

Solution 8.1 (p. 34). $\boldsymbol{a} = \boldsymbol{x}W + \boldsymbol{b}^\top$. $\bar{\boldsymbol{a}} = \bar{\boldsymbol{y}} \odot \boldsymbol{y} \odot (1 - \boldsymbol{y})$; $\bar{W} = \boldsymbol{x}^\top\bar{\boldsymbol{a}}$; $\bar{\boldsymbol{b}} = \bar{\boldsymbol{a}}^\top$; $\bar{\boldsymbol{x}} = \bar{\boldsymbol{a}}W^\top$.

Solution 8.2 (p. 34). X: 32×784, W: 784×128, $Y, \bar{Y}$: 32×128. $X^\top\bar{Y}$: 784×128 ✓. $\bar{Y}W^\top$: 32×784 ✓. $\bar{Y}^\top\mathbf{1}$: 128 ✓.

Solution 8.3 (p. 34). For one example $\ell = -\boldsymbol{t}^\top\boldsymbol{a} + \operatorname{lse}(\boldsymbol{a})$ (using $\sum_k t_k = 1$), so $\nabla\ell = \boldsymbol{s} - \boldsymbol{t}$. The batch loss is $\frac{1}{N}\sum_n \ell_n$, and ℓ_n depends only on row n of A, so row n of $\bar{A}$ is $\frac{1}{N}(\boldsymbol{s}_n - \boldsymbol{t}_n)$.

Solution 8.4 (p. 34). $\boldsymbol{h} = (0.964, 0.762)$, $y = 0.964 - 0.762 = 0.202$, $\bar{y} = 0.202 - 3 = -2.798$. $\bar{\boldsymbol{h}} = -2.798(1, -1) = (-2.798, 2.798)$. $1 - \boldsymbol{h}^2 = (0.0707, 0.4200)$, so $\bar{\boldsymbol{z}} = (-0.198, 1.175)$. (Note how the saturated unit $z_1 = 2$ passes back much less gradient.)

Solution 8.5 (p. 34). $\mathrm{d}Y = \mathrm{d}A \odot B + A \odot \mathrm{d}B$. $\mathrm{d}L = \operatorname{tr}(\bar{Y}^{\mathsf{T}}(\mathrm{d}A \odot B)) + \operatorname{tr}(\bar{Y}^{\mathsf{T}}(A \odot \mathrm{d}B)) = \operatorname{tr}((\bar{Y} \odot B)^{\mathsf{T}}\mathrm{d}A) + \operatorname{tr}((\bar{Y} \odot A)^{\mathsf{T}}\mathrm{d}B)$. So $\bar{A} = \bar{Y} \odot B$, $\bar{B} = \bar{Y} \odot A$.

Solution 8.6 (p. 34). Forward mode: $n = 10^6$ passes (one per input direction). Reverse mode: $m = 1$ pass. Forward mode wins when there are few inputs and many outputs (e.g. $n = 1$: the derivative of a whole trajectory with respect to one parameter), or when only a single directional derivative Jv is needed.

Solution 8.7 (p. 34). $\mathrm{d}\boldsymbol{y} = \mathrm{d}\boldsymbol{x} + J_F\mathrm{d}\boldsymbol{x}$. For K stacked blocks $\nabla_{\boldsymbol{x}}L = (I + J_{F_1})^{\mathsf{T}} \cdots (I + J_{F_K})^{\mathsf{T}}\nabla_{\boldsymbol{y}}L$. Expanding, the product contains the term I, so the gradient always includes a direct copy of $\nabla_{\boldsymbol{y}}L$, however small the J_{F_k} are. Without the skip connections the product $J_{F_1}^{\mathsf{T}} \cdots J_{F_K}^{\mathsf{T}}$ shrinks geometrically when the J_{F_k} have norm below 1.

Chapter 9 Computing Derivatives

Solution 9.1 (p. 38). Forward: $\sin(0.1)/0.1 = 0.998334$, error 1.67×10^{-3}. Here $f''(0) = 0$, so the leading forward error is the next term $\frac{h^2}{6}|f'''| = 1.67 \times 10^{-3}$ (forward differences are lucky at an inflection point). Central: $(\sin 0.1 - \sin(-0.1))/0.2 = 0.998334$ — the same number, because sin is odd; error $\frac{h^2}{6} = 1.67 \times 10^{-3}$. ✓

Solution 9.2 (p. 38). $(1+\epsilon)^2 + 1 = 2 + 2\epsilon$; $3(1+\epsilon) - 1 = 2 + 3\epsilon$. Product: $4 + (2 \cdot 3 + 2 \cdot 2)\epsilon = 4 + 10\epsilon$. So $f(1) = 4$, $f'(1) = 10$. Check: $f' = 2x(3x-1) + 3(x^2+1) = 4 + 6 = 10$. ✓

Solution 9.3 (p. 38). Tape: $v_1 = x + y = 3$, $v_2 = v_1^2 = 9$, $f = v_2 y = 18$. Backward: $\bar{f} = 1$; $\bar{v}_2 = y = 2$ and the first contribution $\bar{y} \mathrel{+}= v_2 = 9$; $\bar{v}_1 = 2v_1\bar{v}_2 = 12$; $\bar{x} = \bar{v}_1 = 12$; $\bar{y} \mathrel{+}= \bar{v}_1 = 12$, total $\bar{y} = 21$. Check: $\partial f/\partial x = 2(x+y)y = 12$, $\partial f/\partial y = 2(x+y)y + (x+y)^2 = 21$. ✓

Solution 9.4 (p. 38). $E'(h) = \frac{hM}{3} - \frac{\varepsilon F}{h^2} = 0 \Rightarrow h^3 = 3\varepsilon F/M$, $h^\star = (3\varepsilon F/M)^{1/3}$. For $M = F = 1$: $h^\star = (6.7 \times 10^{-16})^{1/3} \approx 8.7 \times 10^{-6}$, and $E(h^\star) \approx 1.3 \times 10^{-11} + 2.5 \times 10^{-11} \approx 4 \times 10^{-11}$.

Solution 9.5 (p. 38). For $a \geq 0$: $\ln(1 + e^a) = \ln(e^a(e^{-a} + 1)) = a + \ln(1 + e^{-a})$. For $a < 0$ the right side is $0 + \ln(1 + e^a)$. Naive: $e^{1000} = \infty$, so $\operatorname{softplus}(1000) = \infty$ (wrong; true value 1000). Stable: $1000 + \ln(1 + e^{-1000}) = 1000$. For -1000: naive $\ln(1 + 0) = 0$, stable $\ln(1 + e^{-1000}) \approx e^{-1000}$, which underflows to 0 in the final result too — harmless, the true value is below the smallest double.

Solution 9.6 (p. 38). $\frac{e^{a_k - m}}{\sum_j e^{a_j - m}} = \frac{e^{-m}e^{a_k}}{e^{-m}\sum_j e^{a_j}}$. For the gradient: $\partial/\partial a_k\left(m + \ln\sum e^{a_j - m}\right) = e^{a_k - m}/\sum e^{a_j - m} = s_k$ when m is constant. Since $\operatorname{lse}(\boldsymbol{a})$ does not depend on the choice of m at all, its derivative with respect to m is zero; so whatever m's dependence on $\boldsymbol{a}$, it contributes nothing.

Solution 9.7 (p. 38). $f(x + ih) = f(x) - \frac{h^2}{2}f''(x) + i\left(hf'(x) - \frac{h^3}{6}f'''(x)\right) + \ldots$: the derivative sits alone in the imaginary part, so no two nearly equal numbers are subtracted, and h

can be as small as 10^{-20}, making the $O(h^2)$ error negligible. The argument requires f to be analytic and implemented with complex arithmetic. `np.abs` of a complex number returns the modulus $\sqrt{x^2+h^2}$, a real number, so the imaginary part is 0 and the computed derivative is 0. Comparisons like `x > 0` are not defined for complex numbers (or silently use only the real part).

Chapter 10 Constrained Optimisation, Eigenproblems and PCA

Solution 10.1 (p. 41). $\mathcal{L} = x^2 + y^2 + \lambda(5 - x - 2y)$: $2x = \lambda$, $2y = 2\lambda$, so $y = 2x$; $x + 4x = 5$, $x = 1$, $y = 2$. Formula: $A = [1\ \ 2]$, $AA^\top = 5$, $A^\top \cdot 5/5 = (1, 2)$. ✓

Solution 10.2 (p. 41). $N = 100$, $\pi = (0.3, 0.5, 0.2)$, $\lambda = -N = -100$.

Solution 10.3 (p. 41). $\operatorname{tr} = 5$, $\det = 0$: eigenvalues 5 and 0. $S - 5I = \left[\begin{smallmatrix} -1 & 2 \\ 2 & -4 \end{smallmatrix}\right]$ gives $\boldsymbol{u}_1 = \frac{1}{\sqrt{5}}(2, 1)$. It explains $5/5 = 100\%$; the error is $\lambda_2 = 0$ — the data lie exactly on a line.

Solution 10.4 (p. 41). Quotient rule: $\nabla R = \frac{2S\boldsymbol{x}\boldsymbol{x}^\top\boldsymbol{x} - \boldsymbol{x}^\top S\boldsymbol{x} \cdot 2\boldsymbol{x}}{(\boldsymbol{x}^\top\boldsymbol{x})^2} = \frac{2}{\boldsymbol{x}^\top\boldsymbol{x}}(S\boldsymbol{x} - R\boldsymbol{x})$. Then $\boldsymbol{x}^\top \nabla R = \frac{2}{\boldsymbol{x}^\top\boldsymbol{x}}(\boldsymbol{x}^\top S\boldsymbol{x} - R\boldsymbol{x}^\top\boldsymbol{x}) = 0$. Expected because $R(c\boldsymbol{x}) = R(\boldsymbol{x})$: the function does not change along $\boldsymbol{x}$, so its gradient has no component in that direction.

Solution 10.5 (p. 41). For one centred point $\boldsymbol{x}$: $\left\|\boldsymbol{x} - UU^\top\boldsymbol{x}\right\|^2 = \boldsymbol{x}^\top\boldsymbol{x} - 2\boldsymbol{x}^\top UU^\top\boldsymbol{x} + \boldsymbol{x}^\top UU^\top UU^\top\boldsymbol{x} = \boldsymbol{x}^\top\boldsymbol{x} - \boldsymbol{x}^\top UU^\top\boldsymbol{x}$. Averaging and using the trace trick: $J = \operatorname{tr}(S) - \operatorname{tr}(UU^\top S) = \operatorname{tr}(S) - \operatorname{tr}(U^\top SU)$. Since $\operatorname{tr}(S)$ does not depend on U, minimising J is maximising $\operatorname{tr}(U^\top SU)$.

Chapter 11 Jacobian Determinants and Normalizing Flows

Solution 11.1 (p. 45). $f(x) = \ln x$, $f'(x) = 1/x$, so $p_x(x) = \mathcal{N}(\ln x \mid 0, 1)/x = \frac{1}{x\sqrt{2\pi}} e^{-(\ln x)^2/2}$ (the log-normal). $p_x(1) = 1/\sqrt{2\pi} \approx 0.399$.

Solution 11.2 (p. 45). $A^{-1} = \left[\begin{smallmatrix} 1 & 0 \\ -2 & 1 \end{smallmatrix}\right]$, $\boldsymbol{z} = (1,\ 3 - 2) = (1, 1)$, $\ln p_z = -\ln 2\pi - 1$. $\det A = 1$, so $\ln|\det A| = 0$: a shear preserves area. $\ln p_x = -\ln 2\pi - 1 \approx -2.838$.

Solution 11.3 (p. 45). At $(0, 5)$: $s = 0$, $t = 0$, $\boldsymbol{z} = (0, 5)$, $\ln|\det J| = 0$; inverse $x_2 = (5 - 0)e^0 = 5$. At $(-2, 1)$: $s = -1$, $t = 4$, $z_2 = e^{-1} + 4 \approx 4.368$, $\ln|\det J| = -1$; inverse $x_2 = (4.368 - 4)e^1 = e^{-1}e = 1$. ✓

Solution 11.4 (p. 45). (a) For diagonal A, $\boldsymbol{\epsilon}^\top A\boldsymbol{\epsilon} = \sum_i a_{ii}\epsilon_i^2 = \operatorname{tr} A$ exactly: variance 0. (b) $A^{(s)} = \left[\begin{smallmatrix} 1 & 2.5 \\ 2.5 & 4 \end{smallmatrix}\right]$, $2\left\|A^{(s)}\right\|_F^2 = 2(1 + 6.25 + 6.25 + 16) = 59$, versus 25 for Rademacher. Gaussian vectors also pay for the randomness of ϵ_i^2 on the diagonal.

Solution 11.5 (p. 45). $\mathrm{d}\boldsymbol{z} = -A^{-1}\mathrm{d}A\,\boldsymbol{z}$, so $\mathrm{d}(\frac{1}{2}\boldsymbol{z}^\mathsf{T}\boldsymbol{z}) = \boldsymbol{z}^\mathsf{T}\mathrm{d}\boldsymbol{z} = -\boldsymbol{z}^\mathsf{T}A^{-1}\mathrm{d}A\,\boldsymbol{z} = -\operatorname{tr}(\boldsymbol{z}\boldsymbol{z}^\mathsf{T}A^{-1}\mathrm{d}A)$, contributing $-(\boldsymbol{z}\boldsymbol{z}^\mathsf{T}A^{-1})^\mathsf{T} = -A^{-\mathsf{T}}\boldsymbol{z}\boldsymbol{z}^\mathsf{T}$. And $\mathrm{d}\ln|\det A| = \operatorname{tr}(A^{-1}\mathrm{d}A)$ contributes $A^{-\mathsf{T}}$. Sum: $A^{-\mathsf{T}}(I - \boldsymbol{z}\boldsymbol{z}^\mathsf{T})$. Averaging over data and setting to zero: $\frac{1}{N}\sum_n \boldsymbol{z}_n\boldsymbol{z}_n^\mathsf{T} = I$, i.e. $A^{-1}\big(\frac{1}{N}\sum_n(\boldsymbol{x}_n - \boldsymbol{b})(\boldsymbol{x}_n - \boldsymbol{b})^\mathsf{T}\big)A^{-\mathsf{T}} = I$, so $AA^\mathsf{T} = \frac{1}{N}\sum_n(\boldsymbol{x}_n - \boldsymbol{b})(\boldsymbol{x}_n - \boldsymbol{b})^\mathsf{T}$ (which is Σ_{ML} when $\boldsymbol{b}$ is the sample mean).

Chapter 12 Attention and Normalisation Layers

Solution 12.1 (p. 48). $\boldsymbol{s}^\mathsf{T}\bar{\boldsymbol{s}} = \frac{1}{2}$. $\bar{\boldsymbol{a}} = (\frac{1}{4}, \frac{1}{2}, \frac{1}{4}) \odot (-\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}) = (-\frac{1}{8}, \frac{1}{4}, -\frac{1}{8})$, which is column 2 of $\frac{1}{16}\begin{bmatrix} 3 & -2 & -1 \\ -2 & 4 & -2 \\ -1 & -2 & 3 \end{bmatrix}$. ✓

Solution 12.2 (p. 48). W_q: 512×64; Q: 128×64; S, A: 128×128; Y: 128×64; $\bar{W}_q$: 512×64; $\bar{X}$: 128×512.

Solution 12.3 (p. 48). $\mathrm{d}Y = \mathrm{d}A\,V + A\,\mathrm{d}V$. $\mathrm{d}L = \operatorname{tr}(\bar{Y}^\mathsf{T}\mathrm{d}A\,V) + \operatorname{tr}(\bar{Y}^\mathsf{T}A\,\mathrm{d}V) = \operatorname{tr}((\bar{Y}V^\mathsf{T})^\mathsf{T}\mathrm{d}A) + \operatorname{tr}((A^\mathsf{T}\bar{Y})^\mathsf{T}\mathrm{d}V)$. So $\bar{A} = \bar{Y}V^\mathsf{T}$, $\bar{V} = A^\mathsf{T}\bar{Y}$.

Solution 12.4 (p. 48). $\operatorname{diag}(\boldsymbol{s})\bar{\boldsymbol{s}} = \boldsymbol{s} \odot \bar{\boldsymbol{s}}$ and $\boldsymbol{s}\boldsymbol{s}^\mathsf{T}\bar{\boldsymbol{s}} = (\boldsymbol{s}^\mathsf{T}\bar{\boldsymbol{s}})\boldsymbol{s} = \boldsymbol{s} \odot ((\boldsymbol{s}^\mathsf{T}\bar{\boldsymbol{s}})\mathbf{1})$; subtract. Then $\mathbf{1}^\mathsf{T}\bar{\boldsymbol{a}} = \sum_i s_i\bar{s}_i - (\boldsymbol{s}^\mathsf{T}\bar{\boldsymbol{s}})\sum_i s_i = 0$ since $\sum s_i = 1$.

Solution 12.5 (p. 48). $\mu \mapsto c\mu + b$, $\boldsymbol{x} - \mu\mathbf{1} \mapsto c(\boldsymbol{x} - \mu\mathbf{1})$, $\tilde{\sigma} \mapsto c\tilde{\sigma}$, so $\hat{\boldsymbol{x}}$ is unchanged. Differentiating $\hat{\boldsymbol{x}}(\boldsymbol{x} + t\mathbf{1}) = \hat{\boldsymbol{x}}(\boldsymbol{x})$ at $t = 0$ gives $J\mathbf{1} = \mathbf{0}$; differentiating $\hat{\boldsymbol{x}}((1+t)\boldsymbol{x}) = \hat{\boldsymbol{x}}(\boldsymbol{x})$ gives $J\boldsymbol{x} = \mathbf{0}$. Formula: $J\mathbf{1} = \frac{1}{\tilde{\sigma}}(\mathbf{1} - \mathbf{1} - \frac{1}{D}\hat{\boldsymbol{x}}(\hat{\boldsymbol{x}}^\mathsf{T}\mathbf{1})) = \mathbf{0}$ because $\hat{\boldsymbol{x}}$ has mean zero.

Solution 12.6 (p. 49). (a) $\boldsymbol{q}^\mathsf{T}\boldsymbol{k} = \sum_i q_i k_i$ with independent terms of mean 0 and variance $\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$; the variance of the sum is d. (b) With $\boldsymbol{s} = \boldsymbol{e}_j$: $\operatorname{diag}(\boldsymbol{e}_j) - \boldsymbol{e}_j\boldsymbol{e}_j^\mathsf{T} = 0$; by continuity the Jacobian tends to 0. (c) Unscaled scores have standard deviation $\sqrt{d_k}$ (e.g. 8 for $d_k = 64$), pushing softmax toward one-hot and its Jacobian toward 0, so almost no gradient reaches Q and K. Dividing by $\sqrt{d_k}$ gives scores of unit variance, keeping the softmax in its sensitive range.

APPENDIX C

Further Reading

- **Christopher M. Bishop and Hugh Bishop**, *Deep Learning: Foundations and Concepts*. The book this course accompanies. Its Appendix A summarises the matrix identities and derivatives used here; Appendix C covers Lagrange multipliers.
- **Kaare Brandt Petersen and Michael Syskind Pedersen**, *The Matrix Cookbook*. A large free reference of identities and derivatives. Check its layout conventions before copying formulas (Chapter 1).
- **Jan R. Magnus and Heinz Neudecker**, *Matrix Differential Calculus with Applications in Statistics and Econometrics*. The rigorous treatment of the differential method of Chapter 6.
- **Terence Parr and Jeremy Howard**, *The Matrix Calculus You Need for Deep Learning*. A short, gentle online introduction with a deep-learning focus.
- **Atılım Güneş Baydin, Barak Pearlmutter, Alexey Radul and Jeffrey Siskind**, *Automatic Differentiation in Machine Learning: a Survey*. Forward and reverse mode in depth, the background to Chapter 9.
- **Jorge Nocedal and Stephen Wright**, *Numerical Optimization*. Gradient descent, Newton and Gauss–Newton methods, and the role of the Hessian (Chapter 5).
- **George Papamakarios et al.**, *Normalizing Flows for Probabilistic Modeling and Inference*. A review of the flows of Chapter 11.